

A codelab in 23 steps · companion deck

Zero to Hero: Generative UI

When the agent's answer is an interface, not a paragraph. Seven sub-themes, every step with theory, a code walkthrough and a recorded result.

Heading

Card

BarChart

Button

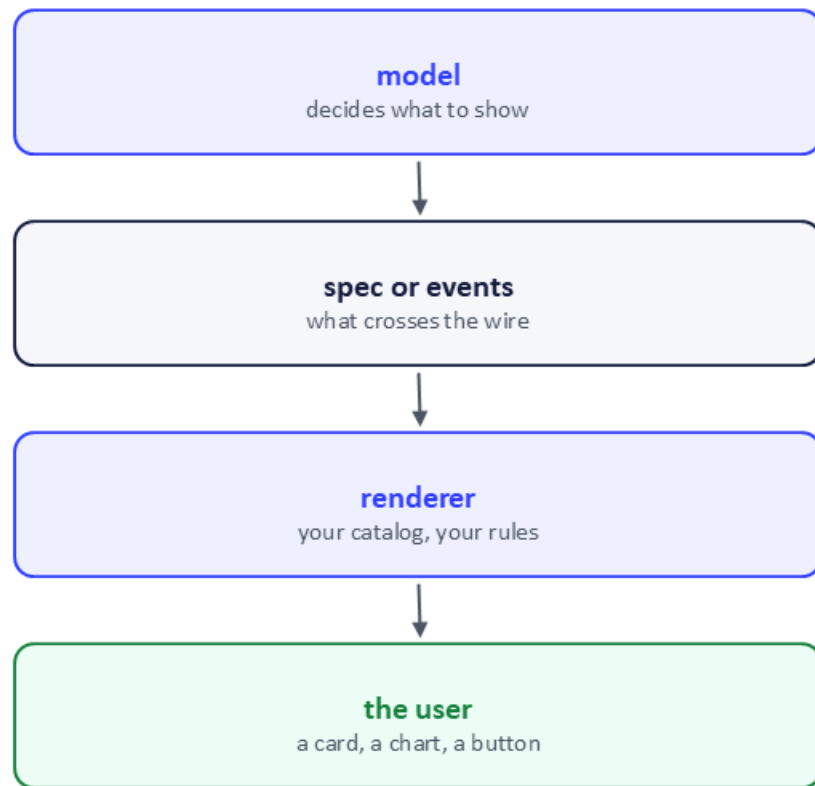
GeneratedView · sandboxed iframe

github.com/dlmastery/simple-coding-harness · [genui/README.md](https://github.com/dlmastery/genui/blob/main/README.md) is the written codelab; this deck follows it step by step

A chat box is the wrong shape for most answers

Ask an agent how sales went this week. The best reply is **three numbers, a chart and a button**, not four paragraphs.

- Generative UI is the layer where an agent's output becomes an interface composed at run time.
- The agent still calls a model in a loop; what changes is what the loop emits and what renders it.
- The harness codelab built the loop. This series builds what the user sees.
- Zero: a page that draws one metric card when the model asks for it.
- Hero: the same agent in a browser, a terminal and a host you do not control, on three open formats, with the costs measured.

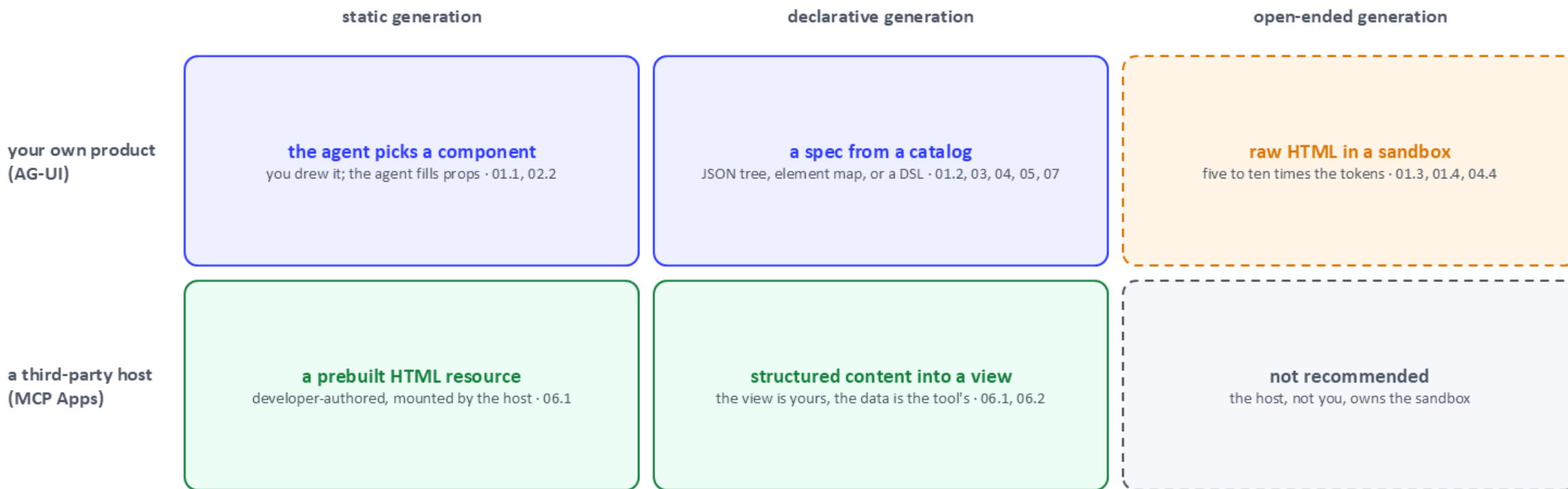


The loop did not change. The output did, and so did the thing that renders it.

Takeaway Every step is a directory you can run, with an offline test and a recorded demo.

Two choices, not one: where the UI appears, and what the agent emits

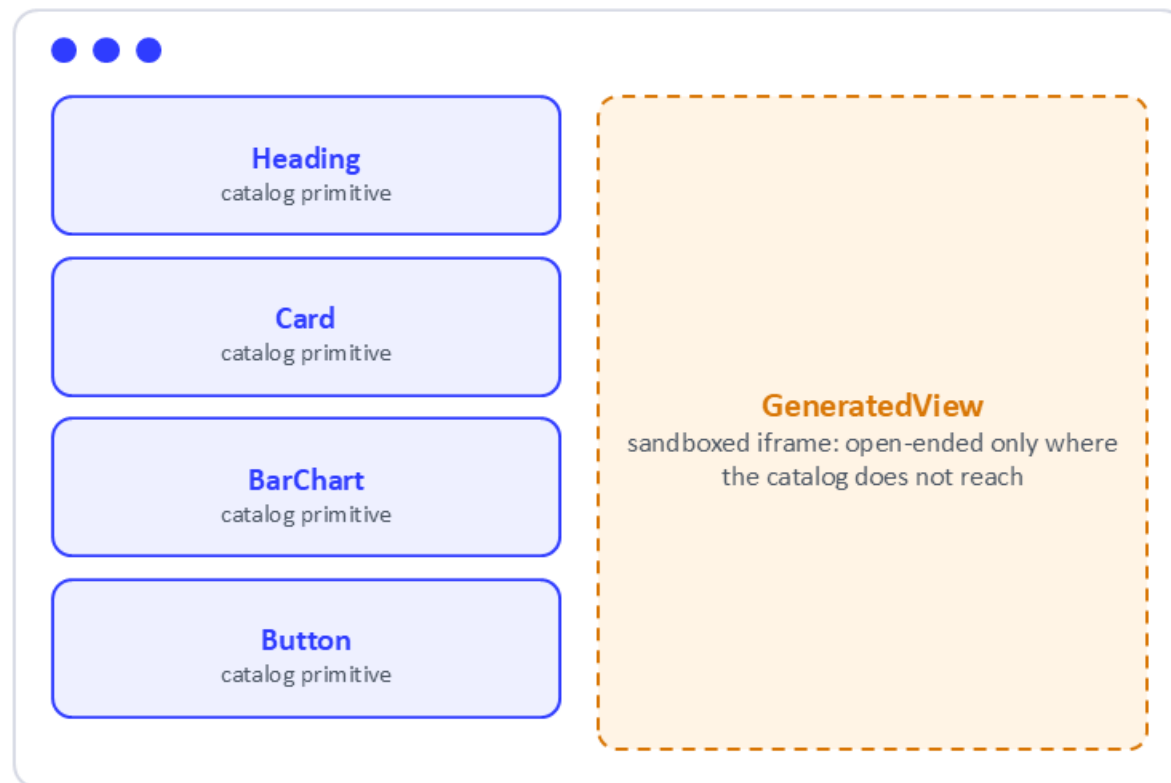
The State of Generative UI report (June 2026) separates **transport** from **generation**. Every step in this series sits in one cell.



Takeaway Choose the transport first, then the generation mode. They are independent.

Static, declarative, open-ended: who decides the layout

- **Static.** Components exist before the model runs. The agent selects one and supplies its props. Lowest cost, safest.
- **Declarative.** The agent composes a spec from a catalog: a JSON tree, a flat element map, or a line-oriented language. The renderer walks the spec.
- **Open-ended.** No catalog. The model writes HTML, CSS and JavaScript; the host runs it in a sandboxed iframe. Five to ten times the tokens.
- **Hybrid.** A catalog with one open-ended component in it. The report's recommended default.
- The declarative middle is where the open formats compete: A2UI, OpenUI Lang, json-render.



The report's diagram: catalog primitives by default

Takeaway Sub-theme 01 builds all four by hand; sub-theme 04 builds the hybrid on OpenUI's reference implementation.

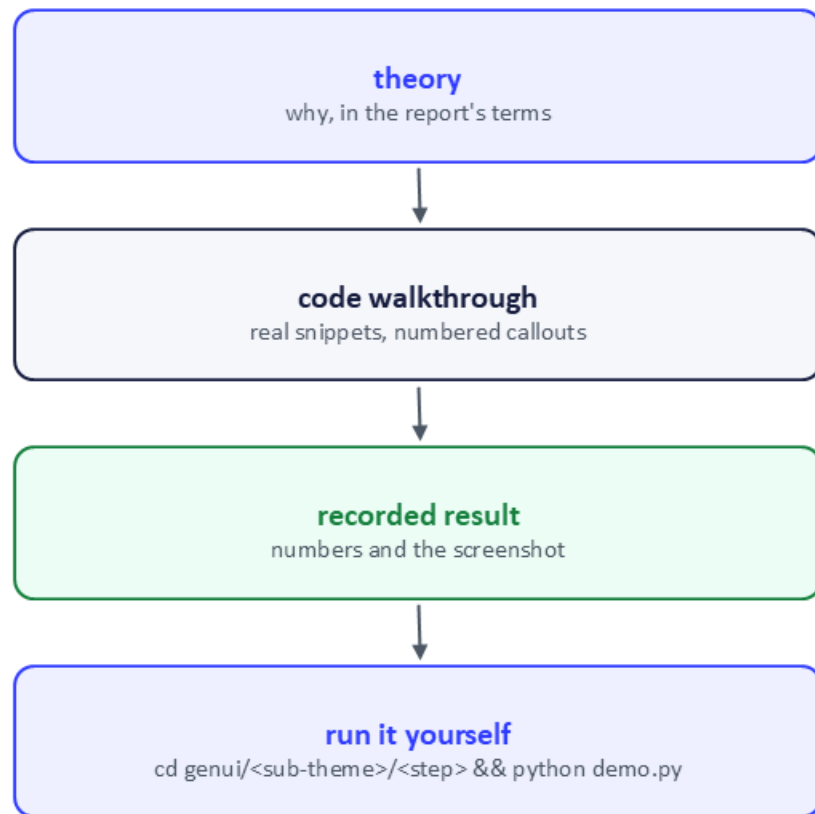
Seven sub-themes, 23 steps, in the order to read them

Sub-theme	Steps	You learn
01 Foundations	static · declarative tree · open-ended HTML · hybrid	the three modes with no framework, measured in tokens and first paint
02 AG-UI	server · tools and state · from the harness	the transport for your own product: runs, events, state patches, client tools
03 A2UI	by hand · from a model · Lit over AG-UI	Google's protocol: surfaces, flat components, a separate data model, actions
04 OpenUI Lang	parser · React renderer · benchmark · html artifact	the DSL with forward references; the report's token table reproduced; the hybrid on OpenUI's example
05 json-render	catalog · streaming patches · actions and targets	Vercel's element map: one catalog for prompt, validator and registry; one spec, two renderers
06 MCP Apps	an app resource · in a real host · hybrid inside an app	UI inside a host you do not control: a tool result that carries HTML, a bridge, a CSP
07 In the harness	render_ui tool · TrueForge generative UI · hybrid on TrueForge	one agent with two surfaces; a hosted harness's UI language parsed by your own page

Read the sub-themes in order the first time. Each one is self-contained after that.

Every step has the same shape, in the deck and in the README

- **Theory.** The idea in the report's terms, why it comes now, and a diagram.
- **Code walkthrough.** One or two snippets read from the real source at build time, with numbered callouts.
- **Recorded result.** The demo output, its real numbers, and the screenshot the demo script took headlessly.
- **Checkpoint and takeaway** in the README: one thing to try, one sentence to keep.
- Every step directory has an offline test that needs no key, and a Files section that names every file.



Takeaway Numbers on these slides come from recorded runs, never from estimates.

What you need to run every step

- Python 3.10 or newer and Node 22. `pip install -r requirements.txt` from the repository root.
- A key in `API_KEY`, or `OPENAI_API_KEY=...` in `~/.simple-harness/env`. Every step's `llm.py` reads both; `BASE_URL` and `MODEL` are optional.
- `python -m playwright install chromium` once, for the demo screenshots.
- Steps with a browser part have a `package.json`; `npm install` there fetches one or two packages. A few steps run one `esbuild` command for a library that ships no browser build.
- `python run_tests.py genui` runs every offline test; `python check_snippets.py genui` verifies every README snippet against the code.

23

steps, each a directory
you can run

30

recorded screenshots

0

keys needed for the tests

3

declarative formats
measured side by side

2

transports: AG-UI and
MCP Apps

656

README snippets verified
against the code in CI

Foundations: static, declarative and open-ended

The report separates two choices: transport is where the UI appears, generation is what the model emits. This sub-theme is about generation. It builds the report's three modes with no framework, so the mechanism stays visible: one FastAPI server, one page with a hand-written renderer, and one prompt, a dashboard for a lemonade stand. Each step copies the previous one and adds one idea, and the last step lands on the report's recommended default with a click that drives the next model turn.

01.1 Static components: the model selects, the page draws

01.2 Declarative tree: the model composes from a catalog

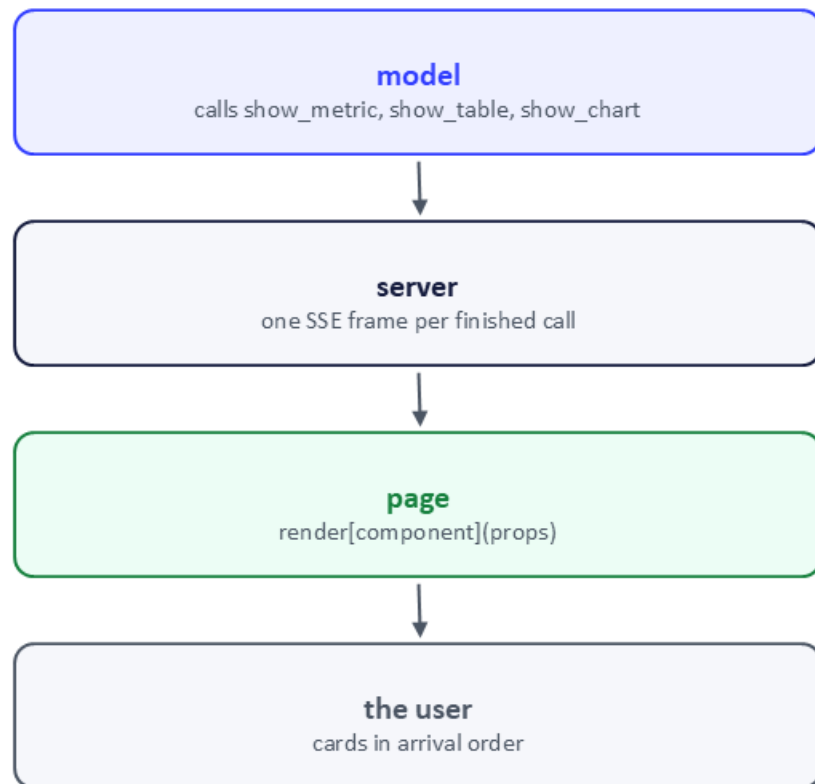
01.3 Open-ended HTML: the model writes the page, in a box

01.4 Hybrid escape hatch: the sandbox as a component

The smallest generative UI: components exist first, the model only selects and fills

In **static generation** the components are built ahead of time. The model sees each one as a tool and calls it with typed props.

- Three components, **Metric**, **Table**, **Chart**, each a strict function-calling schema with every property required.
- The API enforces the schema before the server sees the call, so the renderer never meets a prop it did not expect.
- Each finished tool call becomes one `{component, props}` message on a server-sent event stream.
- The page holds three pure functions from props to HTML and appends each frame as it arrives.
- The model gives up layout: it cannot nest a metric in a card or put two charts side by side. The page decides that.
- It gains safety and cost: every message is small, every prop is typed, every render is one function call.



The page never learns that a model exists; it renders messages.

Takeaway Static generation trades layout for safety and cost; the model chooses only the order of its calls.

One model turn becomes a stream of messages: component, note, or done

```

1 def components(prompt):
2     """Run one model turn and yield one message per component the model chose."""
3     messages = [{"role": "system", "content": SYSTEM_PROMPT}, {"role": "user",
"content": prompt}]
4     for event in llm.stream_chat(messages, tools=catalog.TOOL_SCHEMAS):
[1]         if event["type"] == "tool_call":
6             message = catalog.message_from_call(event["name"], event["arguments"])
7             if message is not None:
8                 yield message
9             elif event["type"] == "text":
[2]                 yield {"note": event["text"]} # the model spoke instead of calling a
tool
11            elif event["type"] == "usage":
[3]                yield {"done": True, "usage": {k: v for k, v in event.items() if k !=
"type"}}

```

01_foundations/step_01_static_components/server.py

- 1 A finished **tool call** becomes a **{ component, props }** message. A call that does not fit the catalog becomes **None** and is dropped, never a broken frame.
- 2 Prose the model wrote instead of calling a tool becomes a **note**, so nothing the model said is lost.
- 3 The **usage** event closes the stream with **done** and the token counts the demo reports.

`/api/run` wraps each message as one SSE frame: a **data:** line with the JSON, then a blank line.

The contract is the schema: strict, every prop required, nothing extra allowed

```
1 def tool(name, description, properties):
2     """One function-calling schema. Every property is required; nothing else is
   allowed."""
3     return {
4         "type": "function",
5         "function": {
6             "name": name,
7             "description": description,
8             [1] "strict": True,
9             "parameters": {
10                 "type": "object",
11                 "properties": properties,
12                 [2] "required": list(properties),
13                 [3] "additionalProperties": False,
14             },
15         },
16     }
```

01_foundations/step_01_static_components/catalog.py

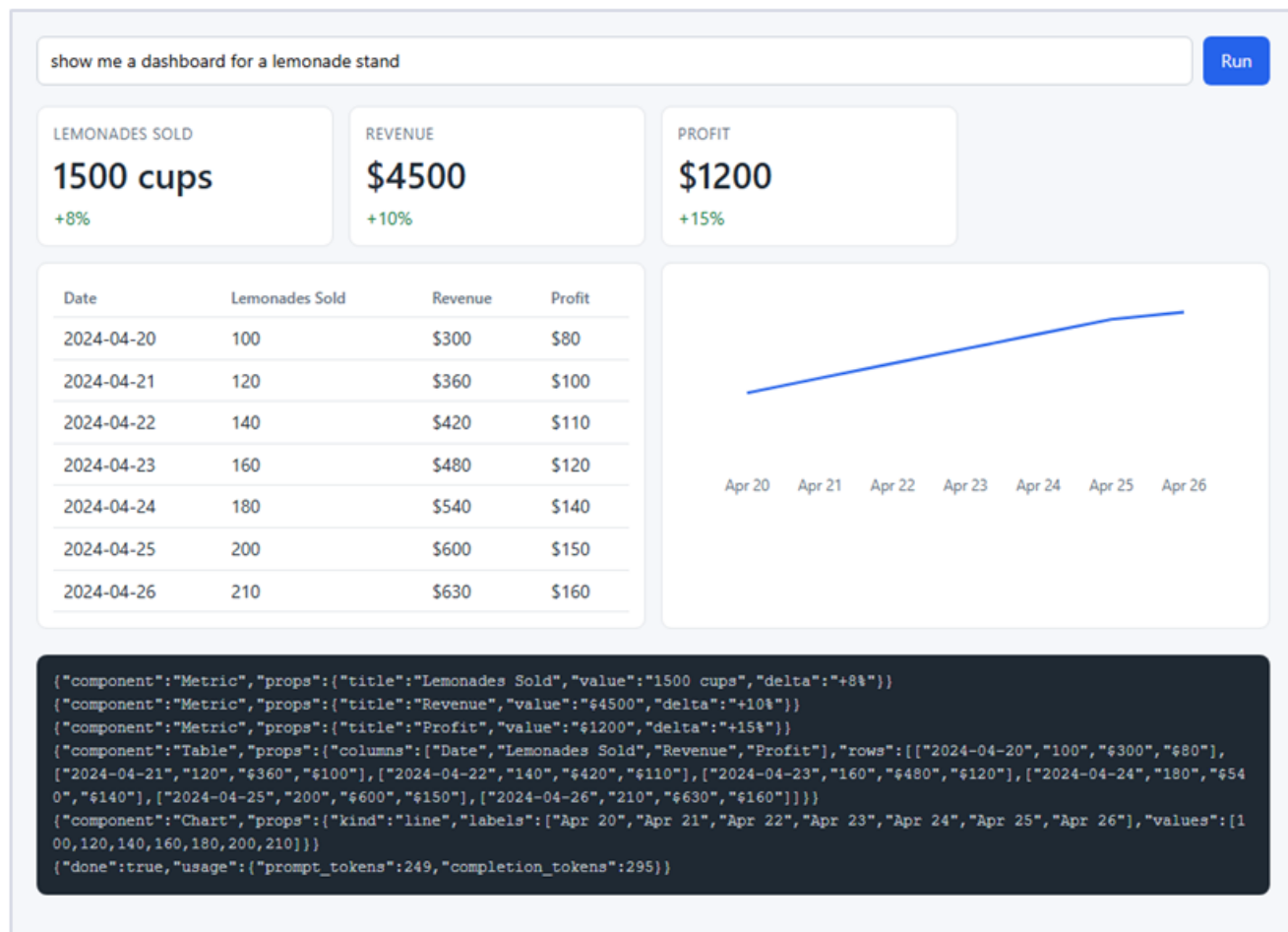
1 **strict: True** makes the API reject any call that does not match the schema, before the server sees it.

2 Every property is **required**: the required list is derived from the properties, so the two cannot drift.

3 **additionalProperties: false** closes the contract. The renderer receives exactly the props it was written for.

Adding a fourth component is one schema here and one function in `render.mjs`; nothing else changes.

Six SSE frames in 7.9 seconds: three metrics, a table, a chart, then done



6

SSE messages in 7.9s

249 / 295

prompt / completion tokens

- Three **Metric** frames, one **Table**, one **Chart**, then **done** with the usage: the wire panel at the bottom lists all six.
- The first metric was drawn while the chart was still streaming: a new tool-call index means every earlier call is complete.
- The model chose only the order of its calls. The page put the metrics in a row; the model could not ask for that.

Prompt: show me a dashboard for a lemonade stand. Recorded with `python demo.py`.

The model composes a layout as data, and the shape of that data decides how it streams

In **declarative generation** the components stay prebuilt and typed, but the model decides which ones, in what order, nested how. The output is data, not code.

- A catalog of eight components drives three things: the prompt the model reads, the JSON Schema the server checks, the renderer table the page uses.
- The page does not wait for the document: a tolerant partial-JSON parser re-parses after every chunk and re-renders.
- Two shapes of the same document: a nested tree, and a flat element map with ids where the root is written first.
- The tree's root closes last, so its layout is known only at the final chunk.
- The flat map puts the root and its child ids first, so the layout is known before any detail arrives.
- That ordering is why A2UI and json-render chose the flat shape.

```
root = "card"
card: Card [metricsRow, dataRow]
metricsRow: Row [sales, visitors, profit]
sales: Metric {title, value, delta}
visitors: Metric {...}
profit: Metric {...}
dataRow: Row [table, chart]
```

**Card: Lemonade Stand
Dashboard**

Row: three metrics, one arrived
waiting for its statement

Row: table and chart
waiting for its statement

A flat map, part way through the stream: the layout is fixed, the details fill in.

Takeaway Declarative generation makes the catalog the single source of truth, and a flat element map puts the layout before the details.

The prompt asks for the root first and the elements top-down; that ordering is the trick

```
1 SHAPE_RULES = {  
[1]     "tree": "Answer with one JSON object: the root node. A node is {\"type\",  
    \"props\", \"children\"}; "  
3         "children is a list of nodes and is only allowed on components that take  
children.",  
[2]     "flat": "Answer with one JSON object {\"root\", \"elements\"}. Write \"root\"  
first. \"elements\" maps an id to a node "  
5         \"{\"type\", \"props\", \"children\"}, where children is a list of ids.  
Write the root element first, "  
[3]     "then its children in order, then their children: top-down, so the  
layout is known before the details.",  
7 }
```

01_foundations/step_02_declarative_tree/catalog.py

- 1 The **tree** rule: the answer is the root node, and children are nodes inside it. The root object cannot close until every descendant has.
- 2 The **flat** rule: **root** is written first, and **elements** maps an id to a node whose children are ids, not nodes.
- 3 Top-down: the root element, then its children in order, then theirs. The layout is on the wire before any detail.

The same catalog generates both prompts; only these rules and the child type in the schema differ.

A child id whose element has not arrived renders as a pending slot

```

1 export function renderFlat(spec, id = spec?.root, seen = new Set()) {
2   // A flat element map. A child id whose element has not arrived yet renders
3   // as a pending slot, so the layout holds still while the details stream in.
4   const node = spec?.elements?.[id];
5   [1] if (node === undefined || seen.has(id)) return PENDING;
6   seen.add(id);
7   if (!isComponent(node)) return PENDING;
8   [2] const children = (node.children ?? []).map((child) => renderFlat(spec, child,
9   seen)).join("");
10  const html = render(node.type, node.props, children);
11  [3] return isPartial(node) ? `

${html}</div>` : html;
12 }

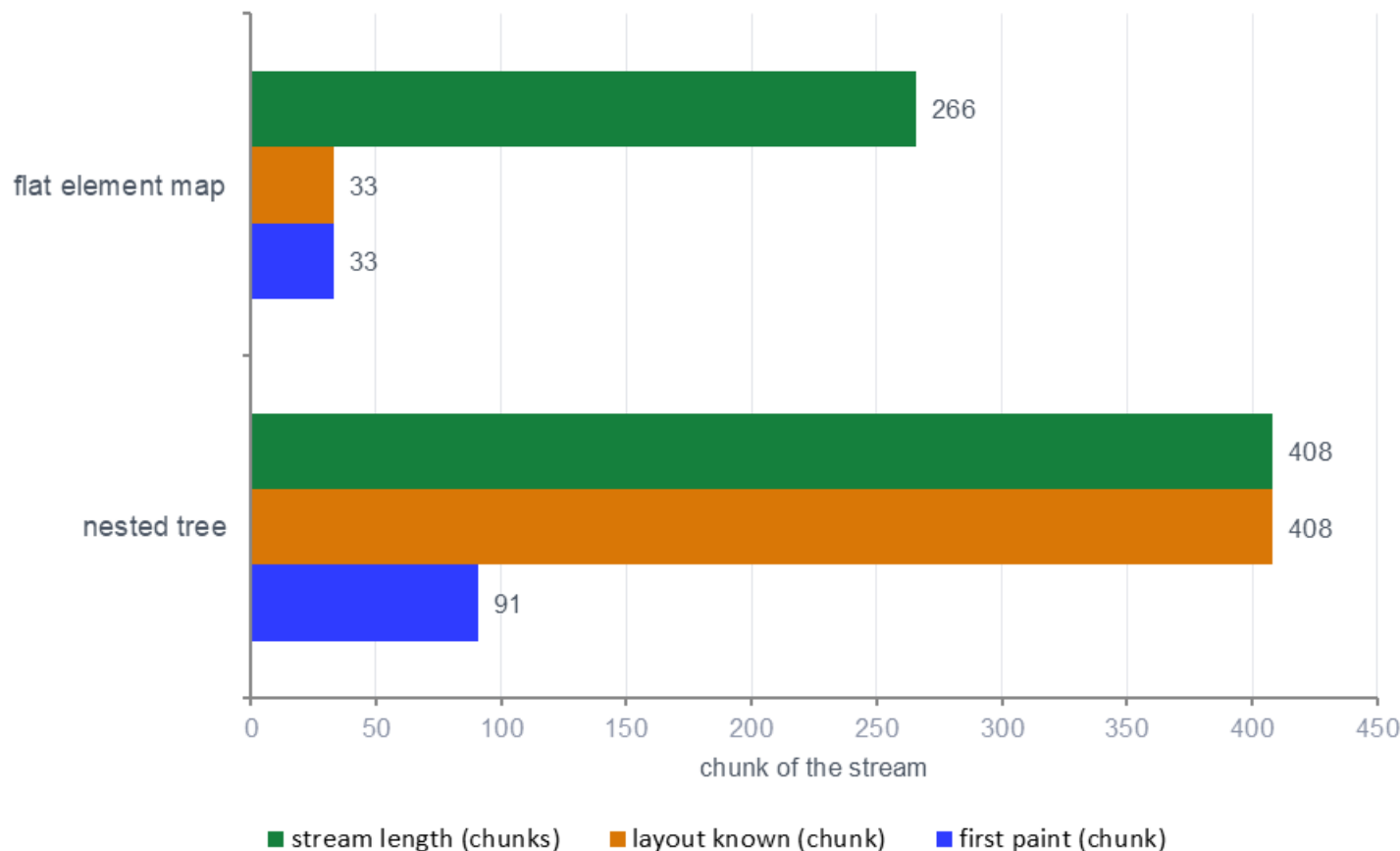

```

01_foundations/step_02_declarative_tree/page/render.mjs

- 1 An id with no element yet, or one already seen, returns **PENDING**: an empty slot at the right place, so the layout holds still.
- 2 Recursion is by **id**, not by nested object. The child is looked up in the map, which is what lets it arrive later.
- 3 A node the parser left open is wrapped as pending; a node the model closed renders plain. The parser types tell them apart.

`renderTree` is the same walk over nested nodes; an open node renders with whatever closed children it already has.

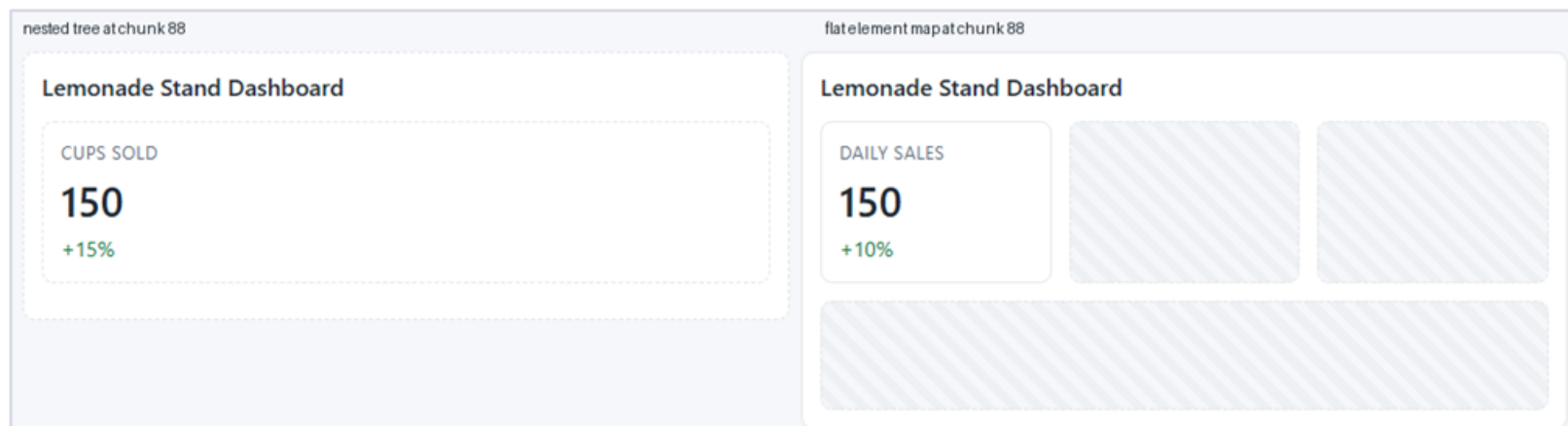
A flat element map paints and fixes its layout at chunk 33; a tree waits for chunk 408



- Same prompt, same model, same eight components. Only the JSON shape differs.
- **Tree:** first paint at chunk 91 (3.22s), layout known at chunk 408 of 408 (6.10s), the last one.
- **Flat:** first paint and layout known both at chunk 33 of 266 (1.08s); done at 3.41s.
- The flat root element is short and closes early; the tree root closes last because everything is inside it.
- Both documents validated.

Recorded by `progress.py` during `python demo.py`: the chunk at which the first component closed and the root's children became final.

At the same chunk, the tree has one metric in an open card; the flat map has the whole layout



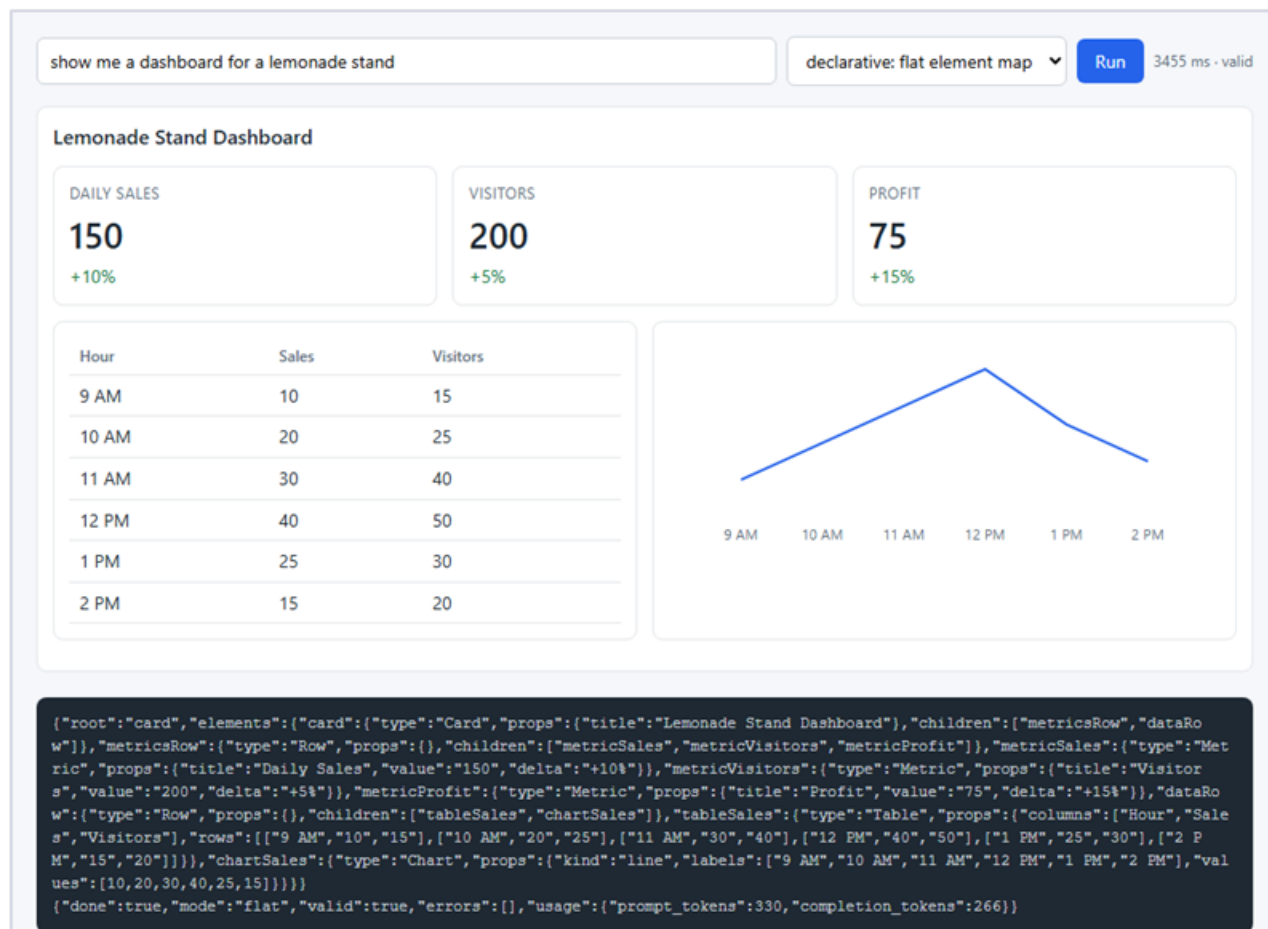
chunk 88

of 408 (tree) and of 266 (flat), replayed

- Left, the nested tree: a card with one metric inside, both still open, and no sign of what comes next.
- Right, the flat map: the title, the metric row with two pending slots, and the row below already in place.
- The hatched boxes are ids whose elements have not arrived; they hold the layout still.

The page's replay hook re-plays each recorded delta stream to a chosen chunk; this is what a user would see at that instant.

The finished flat map: a card, a row of three metrics, a table beside a chart, 266 tokens



266

completion tokens, flat map (330 prompt)

3.41s

to the last chunk; valid against the schema

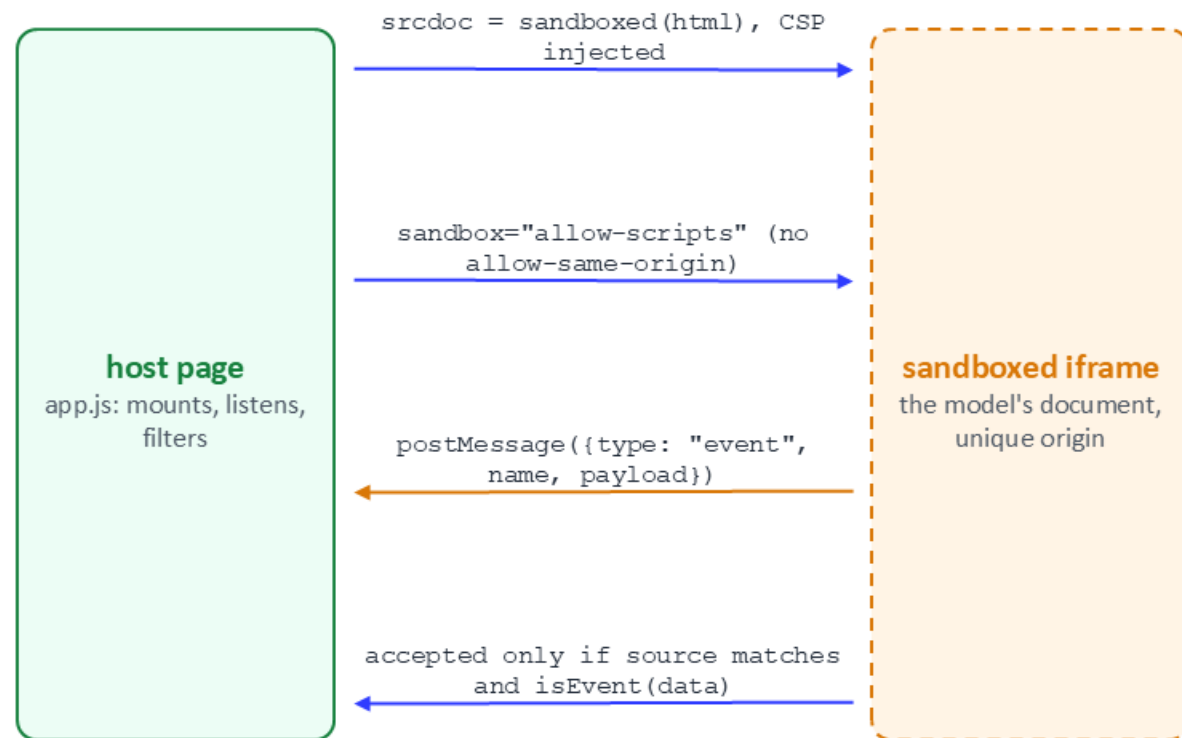
- The model chose the layout: metrics grouped in a **Row**, table and chart in a second **Row**, all inside one **Card**.
- The wire panel shows the element map: **root** first, then **card** with its children, then the children.
- The **done** frame carries **valid: true** and an empty **errors** list from the server's validator.

Same prompt as step 01.1; the difference is that the model, not the page, grouped the components.

The model writes the whole page; the host runs it in a box and accepts one message shape

Open-ended generation removes the catalog. The model writes HTML, CSS and JavaScript, and can draw any layout at a price on three fronts: tokens, trust and consistency.

- Tokens: a page is text, and the text of a page is long. The report says five to ten times a spec; this step measures it.
- Trust: the code came from a model and may do anything code does, so it must run in a box.
- The box has two layers. `sandbox="allow-scripts"` gives the document a unique origin and denies forms, popups and navigation.
- A Content Security Policy injected as the first tag in `<head>` denies every network request: inline code can compute but cannot phone home.
- The only way back out is `postMessage`, and the host keeps one shape from the one window it mounted.
- Consistency: nothing ties the page to the product's components, so every reply looks different.



Two arrows in, one shape out. The CSP inside the document blocks every request the code might make.

Takeaway Open-ended generation buys any layout at ten times the tokens, no streaming paint, and a page that must run in a box.

The box: a CSP that denies all network, injected first in the document, behind allow-scripts

```
[1] export const CSP = "default-src 'none'; style-src 'unsafe-inline'; script-src
'unsafe-inline'; img-src data:";
2
3 export const META = `<meta http-equiv="Content-Security-Policy" content="${CSP}">`;
4
5 export function sandboxed(html) {
6   // Put the CSP meta tag first in <head>, or first in the document if there is no
<head>.
[2]   const head = /<head[^>]*>/i.exec(html);
8   if (head) return html.slice(0, head.index + head[0].length) + META +
html.slice(head.index + head[0].length);
9   return META + html;
10 }
11
12 export function mount(iframe, html) {
[3]   iframe.setAttribute("sandbox", "allow-scripts");
14   iframe.srcdoc = sandboxed(html);
15 }
```

01_foundations/step_03_open_ended_html/page/sandbox.mjs

- 1 **default-src 'none'**: no fetch, no image, no font, no script from anywhere. Inline style and script are allowed because that is what the model writes.
- 2 The meta tag goes **first in `<head>`**, or first in the document, so it applies to every inline block after it.
- 3 **allow-scripts without allow-same-origin**. With both, the document would share the host's origin and could read its cookies and DOM.

`isEvent`, further down the same file, is the one shape the host accepts: `{type: "event", name, payload?}`.

The bridge back: only the mounted window, only the one shape

```
1 window.addEventListener("message", (event) => {  
2   // Only the mounted iframe, and only the one shape the host accepts.  
[1]   if (event.source !== frame.contentWindow || !isEvent(event.data)) return;  
4   inbox.push(event.data);  
[2]   inboxPanel.textContent += JSON.stringify(event.data) + "\n";  
6 });
```

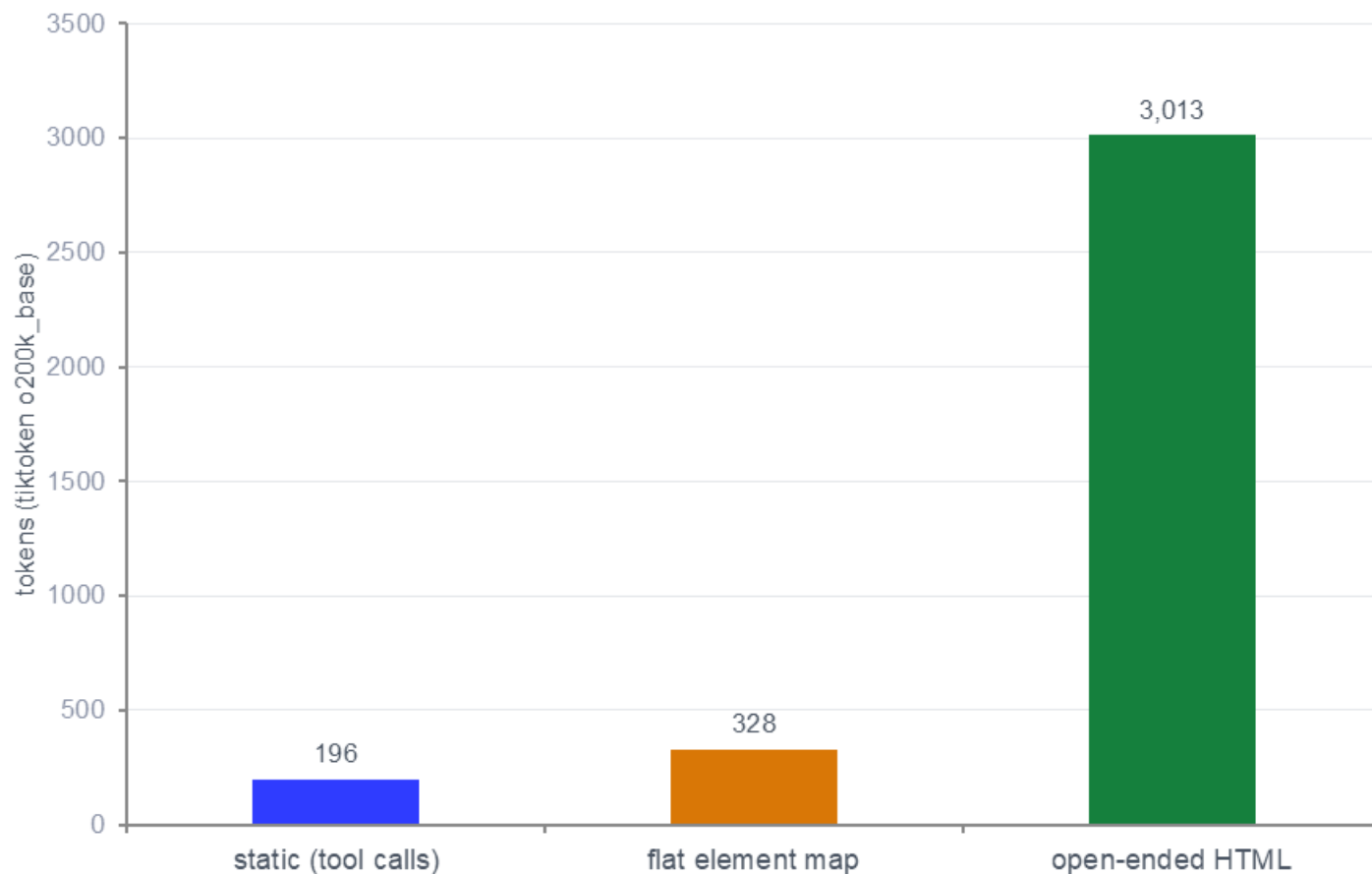
01_foundations/step_03_open_ended_html/page/app.js

1 Two checks before the host believes anything: the **source** must be the iframe the page mounted, and the data must pass **isEvent**.

2 An accepted event lands in the inbox panel. Step 01.4 turns it into the next model turn.

A sandboxed iframe has an opaque origin, so `event.origin` is the string `"null"` and cannot be checked; the source comparison does that job.

The same dashboard: 196 tokens as tool calls, 328 as a flat map, 3013 as HTML



- Same prompt, same model, three modes in one page. The HTML document is **9.2x** the flat map.
- Time to the last byte: static 4.7s, flat 4.0s, HTML 32.5s.
- Two earlier runs gave 9.8x and 10.4x. The report's range is five to ten.
- The HTML was 8718 characters and 208 tags with one button.
- The HTML mode has no partial paint: a half-written document cannot be mounted, so the user waits for all of it.

`tokens.py` counts what each mode made the model write; for the text modes the count matches the API's `completion_tokens`.

The model's own page in the sandbox: any layout, in yellow, 32 seconds later

show me a dashboard for a lemonade stand open-ended: html in a sandbox Run 31937 ms

Lemonade Stand Dashboard

\$1080
TOTAL SALES

540
LEMONADES SOLD

\$2.00
AVG PRICE / CUP

\$540
TOTAL PROFIT

Hour	Cups Sold	Sales (\$)	Profit (\$)
9 AM	30	60.00	30.00
10 AM	50	100.00	50.00
11 AM	70	140.00	70.00
12 PM	90	180.00	90.00
1 PM	100	200.00	100.00
2 PM	80	160.00	80.00
3 PM	60	120.00	60.00

events posted by the iframe appear here

```
<!doctype html>
<html lang="en">
<head>
<meta charset="UTF-8" />
```

3013

tokens for the HTML document, 9.2x the flat map

32.5s

to the last byte; 8718 chars, 208 tags, 1 button

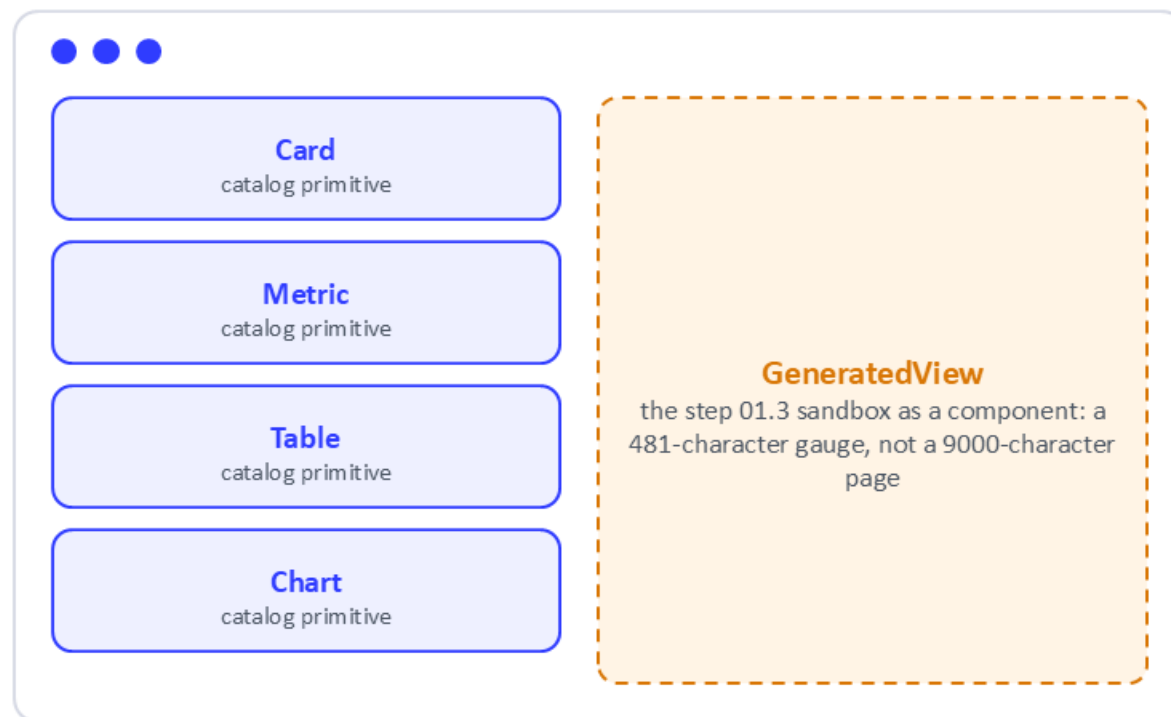
- Four headline numbers, a table and a chart, none of them from the catalog: the model drew its own components and chose the colour.
- The page is mounted whole in the iframe; the raw document is in the panel below, starting with `<!doctype html>`.
- The demo clicked the page's one button and the host received `{type: "event", name: "refreshData", payload: ...}` through the bridge.

The consistency cost is visible: this looks nothing like steps 01.1 and 01.2, and the next reply would look different again.

The report's default: declarative by default, one sandboxed escape hatch, and a click that becomes the next turn

The **hybrid** puts the step 01.3 container inside the step 01.2 catalog. **GeneratedView** is a component like any other, with one string prop; its renderer is the sandbox.

- The catalog gains one entry plus two prompt rules: what the hatch is for, and how small to keep it.
- Nothing in the server knows **GeneratedView** is special; it is a string prop that passes the schema. Only the renderer treats it differently.
- A **Button** carries an **action** name the model chose. The page does not know what it means; it posts it back.
- The server keeps a session transcript. A click becomes a user message on it and the model answers with the next layout.
- The first turn and every later turn share one code path and one stream.
- The catalog stays in charge; the sandbox is rented for one component.



Catalog primitives everywhere; the dashed box only where the catalog does not reach.

Takeaway The hybrid keeps the catalog in charge, rents the sandbox for one component, and makes a click the next user message.

The escape hatch is a renderer: the step 01.3 sandbox, as an entry in the table

```

1 export function GeneratedView({ html }) {
2   // The escape hatch: model-written HTML in the step 03 sandbox. The document
3   // goes through esc() because it is an attribute value here; the browser
4   // unescapes it when it reads srcdoc.
5   [1] return `<iframe class="card generated" sandbox="allow-scripts"
6     srcdoc="${esc(sandboxed(String(html ?? "")))}"></iframe>`;
7 }
8
9 [2] export const RENDERERS = { Card, Row, Column, Text, Metric, Table, Chart, Button,
10   GeneratedView };

```

01_foundations/step_04_hybrid_escape_hatch/page/render.mjs

1

Same box as step 01.3: `sandbox="allow-scripts"` and the CSP injected by `sandboxed`. The document is escaped because here it is an **attribute value**; the browser unescapes `srcdoc`.

2

`GeneratedView` is one more name in `RENDERERS`. The walker, the schema and the prompt pick it up from the catalog like any other.

In `catalog.py` the entry is one line, `"GeneratedView": ({ "html": STRING }, False)`, plus two prompt rules that say what it is for and to keep it short.

The loop closes: the click becomes a user message, and the same generator streams the reply

```

1 @app.post("/api/action")
2 def action(body: Action):
3     """The loop closes: the event becomes a user message, the model answers with the
next layout."""
4     session = SESSIONS.get(body.session)
5     if session is None:
6         raise HTTPException(404, "unknown session")
7     session["turn"] += 1
[1]     session["messages"].append({"role": "user", "content":
EVENT_PROMPT.format(action=body.action, payload=json.dumps(body.payload))})
[2]     return stream(declarative_turn(body.session))

```

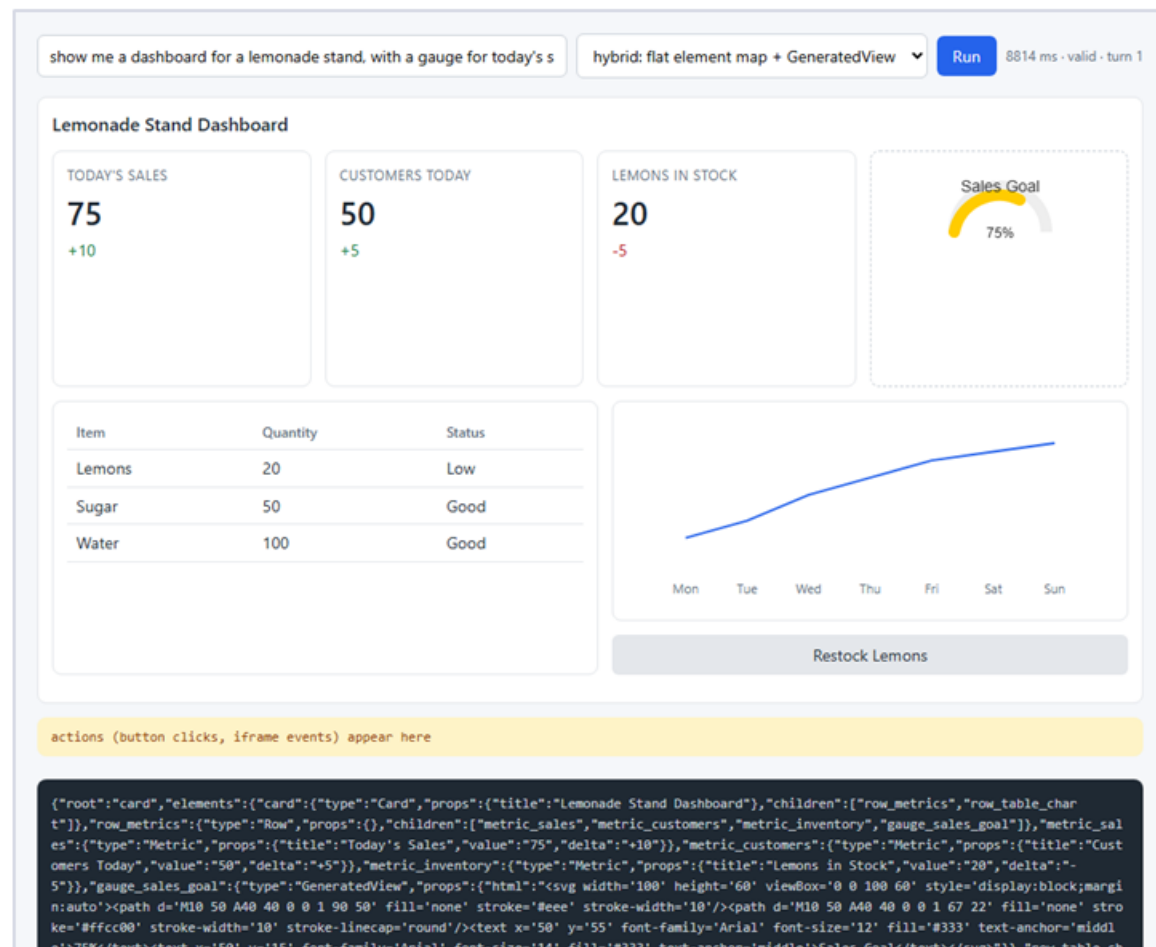
01_foundations/step_04_hybrid_escape_hatch/server.py

1 The event is appended to the session's transcript as **text**: `EVENT_PROMPT` names the action and its payload and asks for the whole updated dashboard, changing only what the event changes.

2 `declarative_turn` is the generator that streamed turn 1. It appends the model's reply to the transcript, so the next click continues from the layout on screen.

In `page/app.js`, a click on any catalog `Button` and a `postMessage` from any generated iframe both go through one `act` function that posts here and hands the reply to the same `consume` that rendered turn 1.

Turn 1: ten catalog components and one GeneratedView holding a 481-character gauge



11

elements, valid: 10 from the catalog, 1 GeneratedView

481

characters of HTML for the gauge; the whole page in 01.3 was about 9000

- The dashed card at the top right is the hatch: an inline SVG gauge for the sales goal, the one thing the catalog cannot show.
- Everything else is catalog: three **Metrics** in a **Row**, a **Table**, a **Chart**, and a **Button** labelled Restock Lemons with action `restock_lemons`.
- The wire panel shows the flat map; `gauge_sales_goal` is a **GeneratedView** whose `html` prop is the SVG.

Prompt: a dashboard for a lemonade stand, with a gauge for today's sales goal and a button to restock lemons.

After the click, two elements changed and nine are byte-for-byte the first layout

show me a dashboard for a lemonade stand, with a gauge for today's s hybrid: flat element map + GeneratedView Run 7540 ms · valid · turn 2

Lemonade Stand Dashboard

TODAY'S SALES

75

+10

CUSTOMERS TODAY

50

+5

LEMONS IN STOCK

100

+80

Sales Goal

75%

Item	Quantity	Status
Lemons	100	Good
Sugar	50	Good
Water	100	Good

Mon Tue Wed Thu Fri Sat Sun

Restock Lemons

```

{"source": "button", "action": "restock_lemons", "payload": {"label": "Restock Lemons"}}

{"root": "card", "elements": [{"card": {"type": "Card", "props": {"title": "Lemonade Stand Dashboard"}, "children": [{"row_metrics": {"type": "Row", "props": {"children": [{"metric_sales": {"type": "Metric", "props": {"title": "Today's Sales", "value": "75", "delta": "+10"}}, {"metric_customers": {"type": "Metric", "props": {"title": "Customers Today", "value": "50", "delta": "+5"}}, {"metric_inventory": {"type": "Metric", "props": {"title": "Lemons in Stock", "value": "100", "delta": "+80"}}, {"gauge_sales_goal": {"type": "GeneratedView", "props": {"html": "<svg width='100' height='60' viewBox='0 0 100 60' style='display: block; margin: auto;'><path d='M10 50 A40 40 0 0 1 90 50' fill='none' stroke='black' stroke-width='10'/><path d='M10 50 A40 40 0 0 1 67 22' fill='none' stroke='black' stroke-width='10' stroke-linecap='round'/><text x='50' y='55' font-family='Arial' font-size='12' fill='black' text-anchor='middle'>Sales Goal</text></svg>"}}]}]}]}]}]}]}

```

2 / 9

elements changed / unchanged; none added or removed

20 → 100

Lemons in Stock; delta -5 became +80, the table row followed

- The page sent `{"source": "button", "action": "restock_lemons", "payload": {"label": "Restock Lemons"}}`; it is in the yellow inbox panel.
- `metric_inventory` and `table_inventory` changed; the gauge, the chart and the other metrics came back identical.
- The whole layout was re-sent. A patch protocol would send only the two elements; sub-themes 03 and 05 do that.

The action name `restock_lemons` appears in no schema. The page treats it as opaque, the server as text, and only the model interprets it.

AG-UI, the transport

The report separates what the model generates from how it travels to the screen. Sub-theme 01 was about generation; this sub-theme is about transport. AG-UI is the transport for a product you own: one RunAgentInput request in, one stream of typed events out. Three steps build the server with ag-ui-protocol, write the client by hand so the wire stays visible, then replace it with @ag-ui/client and put the harness codelab's coding agent behind the protocol.

02.1 An AG-UI server: RUN_STARTED, TEXT_MESSAGE_*, RUN_FINISHED over SSE

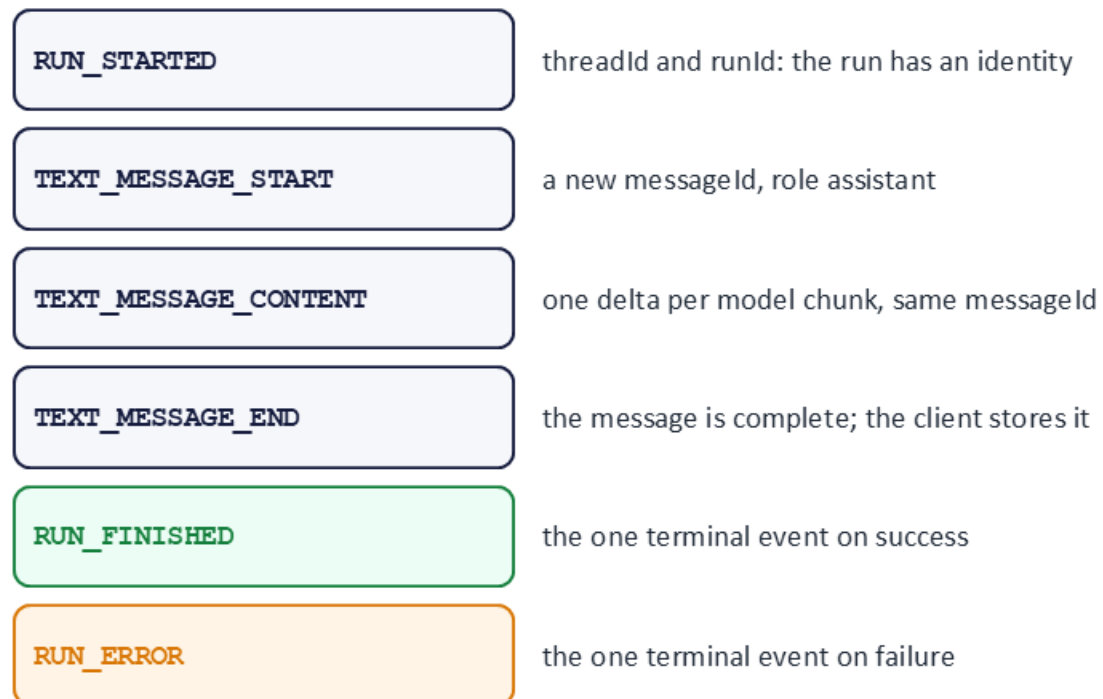
02.2 Tool calls and shared state: STATE_SNAPSHOT, STATE_DELTA, a client tool

02.3 The harness codelab's loop over AG-UI, with HttpAgent in the page

A run is a request; the reply is a stream of typed events

AG-UI fixes the **vocabulary** of what crosses the wire, so any AG-UI client can read any AG-UI agent.

- The request is one object, RunAgentInput: threadId, runId, messages, tools, context, state.
- The reply is text/event-stream: one JSON event per data: line, camelCase keys.
- A text message is three kinds of event joined by one messageId.
- Every run ends with exactly one terminal event: RUN_FINISHED or RUN_ERROR.
- The server is stateless between runs; the client owns the history and sends all of it.
- EventSource only does GET, so the page reads the body with fetch.



The whole vocabulary of step 02.1. Tool calls and state arrive in step 02.2.

Takeaway A transport is a fixed vocabulary of events with a join key; the messageId is that key.

The agent is a generator with two exits and one join key

```

1 def run(input: RunAgentInput):
2     """Yield the events of one run. A model failure becomes RUN_ERROR, not a broken
stream."""
3     yield RunStartedEvent(thread_id=input.thread_id, run_id=input.run_id)
4     message_id = str(uuid4())
5     try:
6         yield TextMessageStartEvent(message_id=message_id, role="assistant")
7         for delta in stream_text(to_openai(input.messages)):
8             yield TextMessageContentEvent(message_id=message_id, delta=delta)
9             yield TextMessageEndEvent(message_id=message_id)
10    except Exception as error: # noqa: BLE001 - the client must see a terminal
event
11        yield RunErrorEvent(message=str(error))
12        return
13    yield RunFinishedEvent(thread_id=input.thread_id, run_id=input.run_id)

```

02_ag_ui/step_01_ag_ui_server/agent.py

- 1 The run announces itself with the ids from the request. The generator yields pydantic events from **ag_ui.core** and never touches HTTP.
- 2 One **message_id** for the start, every delta and the end. The client joins them on it.
- 3 A model failure becomes **RUN_ERROR**, not a broken stream. Success ends with **RUN_FINISHED**. Exactly one terminal event either way.

server.py wraps this in EventEncoder and a StreamingResponse, so the first event leaves before the model has finished.

The page reads the stream with fetch, because the request has a body

```
1 export async function* readEvents(response) {  
[1]   const reader = response.body.getReader();  
3   const decoder = new TextDecoder();  
4   let buffer = "";  
5   while (true) {  
6     const { value, done } = await reader.read();  
7     if (done) break;  
[2]   buffer += decoder.decode(value, { stream: true });  
9     let cut;  
[3]   while ((cut = buffer.indexOf("\n\n")) >= 0) {  
11     const block = buffer.slice(0, cut);  
12     buffer = buffer.slice(cut + 2);  
13     const event = parseBlock(block);  
14     if (event) yield event;  
}
```

02_ag_ui/step_01_ag_ui_server/sse.mjs

- 1 **response.body.getReader()** gives raw bytes. EventSource would parse SSE for free, but it cannot POST a RunAgentInput.
- 2 A chunk can end in the middle of an event. The buffer holds the remainder until the next blank line.
- 3 One block is the data: lines up to a blank line; **parseBlock** turns it into one JSON event, and app.js switches on event.type.

app.js keeps the messages array; a finished assistant message joins it, so the next run sends the whole conversation back.

One request, 22 events, one terminal event

AG-UI step 01: one run, one text message, streamed

In one short sentence, what makes good lemonade?

Run

RENDERED

In one short sentence, what makes good lemonade?

Good lemonade is made with fresh lemons, the right balance of sweetness, and cold, clean water.

WIRE (ONE EVENT PER LINE)

```
{
  "type": "RUN_STARTED",
  "threadId": "54252b0d-ad2c-4bfa-9661-8c6c66f59c37",
  "runId": "5aabc12-e7fb-4bf2-8380-095a503ba79d"
}
{
  "type": "TEXT_MESSAGE_START",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "role": "assistant"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": "Good"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " lemonade"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " is"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " made"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " with"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " fresh"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " lemons"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " ,"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " the"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " right"
}
{
  "type": "TEXT_MESSAGE_CONTENT",
  "messageId": "84061b2d-557f-4c49-9f96-e9e11359a16d",
  "delta": " ."
}
```

18

TEXT_MESSAGE_CONTENT events in the recorded run

5

event kinds on the wire

- Left: the answer rendered word by word from the deltas.
- Right: every event as one line, camelCase keys, the same messageId on start, content and end.
- The page ran the prompt as a second request and got a different sentence; the wire looked the same.

Checkpoint: set API_KEY to a wrong value. The stream still ends with one terminal event, now RUN_ERROR, and the page still reaches a final status.

Generative UI as shared state: the model patches an object, the page renders it

Nothing on the wire says "metric card". A tool returns a **JSON Patch** to a dashboard object; the page re-renders the object.

- Tool calls stream in three parts joined by toolCallId; the result is a fourth event.
- STATE_SNAPSHOT opens the run with the whole object; STATE_DELTA carries a patch.
- Server tools draw nothing: show_metric returns an add on /dashboard/metrics/-.
- The client owns the object and sends it back, so the next run continues from the same dashboard.
- A client tool is declared in RunAgentInput.tools; the server streams the call and ends the run.
- The page answers as a tool message and starts the next run with no user text.

STATE_SNAPSHOT	the state this run starts from, sent or empty
TOOL_CALL_START	toolCallId and toolCallName
TOOL_CALL_ARGS	the JSON arguments, in pieces as the model streams
TOOL_CALL_END	the arguments are complete; the server parses them now
STATE_DELTA	a JSON Patch: add, replace or remove at a JSON Pointer
TOOL_CALL_RESULT	the tool's answer, role tool; absent for a client tool

The step 02.1 events are still there. These six are new.

Takeaway The component vocabulary lives in the state shape and the renderers, not in the protocol.

Apply the patch on the server first, then send it; stop the run for a client tool

```

1 waiting_on_client = False
2 for call_id, call in calls.items():
[1]     if call["name"] in client_tools:
4         waiting_on_client = True # the page runs it; its result opens the next run
5         continue
6         result, operations = execute(call["name"], json.loads(call["arguments"] or
("{}"))
[2]     if operations:
8         apply_patch(state, operations)
9         yield StateDeltaEvent(delta=operations)
10        yield ToolCallResultEvent(message_id=str(uuid4()), tool_call_id=call_id,
content=result, ro...
11        messages.append({"role": "tool", "tool_call_id": call_id, "content": result})
[3] if waiting_on_client:
13     break

```

02_ag_ui/step_02_ag_ui_tools_and_state/agent.py

1

A tool the page declared gets **no result on the server**. The flag is set and the loop moves on; the answer arrives on the next run.

2

apply_patch on the server copy, then the same operations go out as STATE_DELTA. Both copies stay equal.

3

The run ends with the call still open. The page draws the buttons; the click becomes a tool message.

The run opened with `yield StateSnapshotEvent(snapshot=state)`, where `state` is the client's copy or `initial_state()`.

Server tools return patches; not one of them draws anything

```

1 def show_metric(title: str, value: str, delta: str = "") -> list:
2     """Add one metric card to the dashboard."""
3     return [{"op": "add", "path": "/dashboard/metrics/-", "value": {"title": title,
4 "value": va...
5
6 def show_table(columns: list, rows: list) -> list:
7     """Set the dashboard table (one table, replaced each time)."""
8     return [{"op": "replace", "path": "/dashboard/table", "value": {"columns":
9 columns, "rows":...
10
11 def show_chart(kind: str, labels: list, values: list) -> list:
12     """Set the dashboard chart. kind is 'bar' or 'line'."""
13     return [{"op": "replace", "path": "/dashboard/chart", "value": {"kind": kind,
14 "labels": lab...

```

02_ag_ui/step_02_ag_ui_tools_and_state/tools.py

- 1 `/dashboard/metrics/-` is JSON Pointer for append. Three metrics in the recorded run became three add operations.
- 2 One table, **replaced** each time. The page's renderer decides what a table looks like; the protocol only moves the object.
- 3 Each tool is a pure function from arguments to operations. `json_patch.py` and `json-patch.mjs` apply them identically on both sides.

`render.mjs` rebuilds the dashboard from the state object on every `STATE_SNAPSHOT` and `STATE_DELTA`.

Three runs: build the dashboard, ask the page, record the answer

AG-UI step 02: tool calls and shared state

Buy a new cooler for \$40. Run

CHAT

Show me a dashboard for a lemonade stand.

```
show_metric({"title": "Total Revenue", "value": "$2,450", "delta": "+8%"} -> ok
```

```
show_metric({"title": "Lemonades Sold", "value": "1,225", "delta": "+5%"} -> ok
```

```
show_metric({"title": "Customer Satisfaction", "value": "92%", "delta": "+3%"} -> ok
```

```
show_table({"columns": ["Day", "Lemonades Sold", "Revenue"], "rows": [{"Monday", "180", "$360"}, {"Tuesday", "200", "$400"}, {"Wednesday", "195", "$390"}, {"Thursday", "210", "$420"}, {"Friday", "220", "$440"}, {"Saturday", "220", "$440"}]} -> ok
```

```
show_chart({"kind": "bar", "labels": ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"], "values": [180, 200, 195, 210, 220, 220]} -> ok
```

Here's the dashboard for your lemonade stand with key metrics, daily sales table, and a sales chart.

Buy a new cooler for \$40.

```
confirm_purchase({"item": "new cooler", "cost": 40})
```

Confirm purchase: new cooler for \$40? confirmed

```
record_purchase({"item": "new cooler", "cost": 40}) -> ok
```

The purchase of the new cooler for \$40 has been recorded.

DASHBOARD (RENDERED FROM STATE)

Day	Lemonades Sold	Revenue
Monday	180	\$360
Tuesday	200	\$400
Wednesday	195	\$390
Thursday	210	\$420
Friday	220	\$440
Saturday	220	\$440

new cooler: \$40

WIRE (ONE EVENT PER LINE)

```
{ "type": "RUN_STARTED", "threadId": "9381f38f-c80d-47ce-8ef8-004af601dbc5", "runId": "34a30c9e-ce4a-419c-9ae6-f308bc329504" }
```

64

TOOL_CALL_ARGS events to stream show_table's arguments

3

metrics in the second run's STATE_SNAPSHOT: the page sent the dashboard back

- Run 1 opens with STATE_SNAPSHOT at 0 metrics, then show_metric three times, show_table, show_chart, each followed by a STATE_DELTA.
- Run 2: the model calls confirm_purchase and the run finishes with no TOOL_CALL_RESULT; the confirm panel appears on the left.
- Run 3, after the click: record_purchase adds to /dashboard/purchases/-.
- Right: the dashboard rendered from state, never from a component message.

Checkpoint: remove confirm_purchase from CLIENT_TOOLS in app.js and ask to buy something. Why does the run no longer stop early?

A terminal and AG-UI are two transports for the same loop

The harness codelab's coding agent runs from a browser with **no change to harness/**. A bridge translates its side effects into events.

- `harness/` is a verbatim copy of the harness codelab's step 21: `call_llm`, `execute`, `todos`, `permissions`.
- A text delta from `on_delta` becomes `TEXT_MESSAGE_CONTENT`; an assembled tool call becomes three `TOOL_CALL_*` events.
- The plan is shared state: `STATE_SNAPSHOT {todos}` at the start, `STATE_DELTA` replace `/todos` after every `write_todos`.
- A permission question becomes a `CUSTOM` event named `permission`, because the browser has no prompt.
- Token usage the harness already collects rides on `RUN_FINISHED` in the `TokenUsage` shape.
- The page uses `HttpAgent` from `@ag-ui/client`: no SSE parsing, no message bookkeeping.

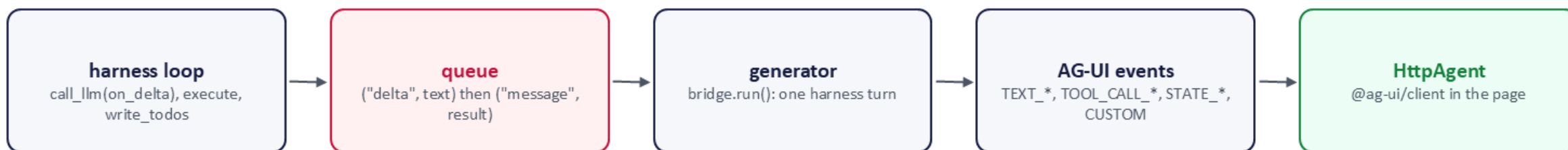
STATE_SNAPSHOT	{ "todos": [...] }: the harness's plan, module global
TEXT_MESSAGE_CONTENT	one per <code>on_delta</code> callback from <code>call_llm</code>
TOOL_CALL_ARGS	one delta: the harness assembles the call first
TOOL_CALL_RESULT	the return of <code>tools.execute</code> , permission layer included
CUSTOM permission	reason and decision; the terminal asked at the prompt
STATE_DELTA	replace <code>/todos</code> after <code>write_todos</code>
RUN_FINISHED	with usage: input and output tokens

The same vocabulary as steps 02.1 and 02.2, plus one CUSTOM event.

Takeaway A second transport costs a bridge and a page, not a new agent.

Callbacks push, an event stream pulls; a queue between two threads joins them

`call_llm` blocks and reports text through `on_delta`. A worker thread runs it and puts every callback on a queue; a generator drains the queue and yields events; `HttpAgent` reads them in the page.



server.py moves into the work directory before it imports the harness, which reads the current directory at import time.

Takeaway The bridge is 150 lines and the page is 90; harness/ has no diff against the harness codelab.

Every side effect of one tool call becomes an event; execute is the harness's own

```

1 for tool_call in message.tool_calls:
2     yield ToolCallStartEvent(tool_call_id=tool_call.id,
tool_call_name=tool_call.function.name,...
[1]     yield ToolCallArgsEvent(tool_call_id=tool_call.id,
delta=tool_call.function.arguments)
4     yield ToolCallEndEvent(tool_call_id=tool_call.id)
5
6     ASKED.clear()
[2]     args, result = execute(tool_call)
8     for reason in ASKED:
9         yield CustomEvent(name="permission", value={"reason": reason, "decision":
"allow"})
10    yield ToolCallResultEvent(message_id=str(uuid4()), tool_call_id=tool_call.id,
content=result...)
11    messages.append({"role": "tool", "tool_call_id": tool_call.id, "content":
result})
[3]    if tool_call.function.name == "write_todos":
13        yield StateDeltaEvent(delta=[{"op": "replace", "path": "/todos", "value":
copy.deepcopy...

```

02_ag_ui/step_03_ag_ui_from_the_harness/bridge.py

- 1 The arguments go out as **one TOOL_CALL_ARGS delta**, because the harness assembles the call before the loop sees it. Step 02.2 streamed them in pieces; both are legal.
- 2 **execute** is the harness's function, permission layer included. `ui.approve` is replaced by a stand-in that records the reason, allows, and each reason becomes a CUSTOM event.
- 3 After **write_todos**, the plan goes out as a replace on `/todos`. The page's copy of the state is refreshed from the module global.

The run closes with `RunFinishedEvent(..., usage=[token_usage(usage)])`: the same dict the terminal prints after every reply.

The page subscribes instead of parsing: HttpAgent keeps the buffer, the messages and the state

```
1 agent.subscribe({
[1]   onEvent({ event }) {
3     wire.textContent += JSON.stringify(event) + "\n";
4   },
5   onTextMessageStartEvent() {
6     textElement = addMessage("assistant", "");
7   },
[2]   onTextMessageContentEvent({ textMessageBuffer }) {
9     textElement.textContent = textMessageBuffer; // the client keeps the buffer
10  },
11  onToolCallStartEvent({ event }) {
[3]    toolElements[event.toolCallId] = addMessage("tool", `${event.toolCallName}(`);
13  },

```

02_ag_ui/step_03_ag_ui_from_the_harness/app.js

- 1 **onEvent** sees every event, which is how the wire panel is still drawn. The subscriber gets one method per event type.
- 2 **textMessageBuffer** is the text so far, kept by the client. sse.mjs and the messages array from steps 02.1 and 02.2 are gone.
- 3 Later handlers draw tool results, the permission line from CUSTOM, and the todo list from **onStateChanged**. After the run, agent.messages and agent.state hold everything the next run sends.

agent.addMessage(...) then await agent.runAgent() is the whole run. The bundle exists because @ag-ui/client imports rxjs and zod by bare name.

A coding task from the browser: files written, a command run, a plan as state

AG-UI step 03: the harness codelab's loop, driven from the browser

Plan this with write_todos first: add a function greet(name) to hello.py, call it with 'AG-UI', run the file again. Run

CHAT

Create hello.py that prints 'hello from AG-UI', then run it with python and report the output.

```
write_file({"path":"hello.py","content":"print('hello from AG-UI')\n"})
Wrote hello.py
```

```
bash({"command":"python hello.py"})
hello from AG-UI
```

permission: run: python hello.py -> allow

I have created hello.py that prints 'hello from AG-UI' and ran it. The output is:

hello from AG-

Plan this with write_todos first: add a function greet(name) to hello.py, call it with 'AG-UI', run the file again.

```
write_todos({"todos":[{"content":"Add a function greet(name) to hello.py","activeForm":"Adding a function greet(name) to hello.py","status":"pending"},{"content":"Call greet('AG-UI') in hello.py","activeForm":"Calling greet('AG-UI') in hello.py","status":"pending"},{"content":"Run hello.py again and report output","activeForm":"Running hello.py again and reporting output","status":"pending"}]})
```

```
[ ] Add a function greet(name) to hello.py
[ ] Call greet('AG-UI') in hello.py
[ ] Run hello.py again and report output
```

```
write_todos({"todos":[{"content":"Add a function greet(name) to hello.py","activeForm":"Adding a function greet(name) to hello.py","status":"in_progress"},{"content":"Call greet('AG-UI') in hello.py","activeForm":"Calling greet('AG-UI') in hello.py","status":"pending"},{"content":"Run hello.py again and report output","activeForm":"Running hello.py again and reporting output","status":"pending"}]})
```

```
[~] Add a function greet(name) to hello.py
[ ] Call greet('AG-UI') in hello.py
[ ] Run hello.py again and report output
```

```
read_file({"path":"hello.py"})
print('hello from AG-UI')
```

TODOS (SHARED STATE)

```
[x] Add a function greet(name) to hello.py
[x] Call greet('AG-UI') in hello.py
[~] Run hello.py again and report output
```

gpt-4.1-mini: 2053 in, 50 out

1,148 / 28

tokens in / out on the first RUN_FINISHED

2,053 / 50

tokens in / out on the second, 46 event lines

- Task 1: write_file hello.py, bash python hello.py, CUSTOM permission: run: python hello.py -> allow, then 27 text deltas.
- Task 2 plans with write_todos first; the page ends with two todos completed and one in progress, drawn from agent.state.
- Right: the todo panel is shared state, not chat.

Checkpoint: set HARNESS_WORKDIR to a project of your own and ask for a change from the browser. Then diff harness/ against the harness codelab's step 21: it is empty.

A2UI, Google's declarative protocol

Sub-theme 01 found that a flat element map streams best. A2UI v0.9.1 is that shape as a published protocol: four envelope messages, a flat component list joined by ids, a data model addressed by JSON Pointer, and a swappable catalog. The report places it in the declarative middle of the generation axis. This sub-theme builds the messages by hand, has the model write them through the official SDK, then renders them with Google's Lit renderer over the AG-UI transport from sub-theme 02.

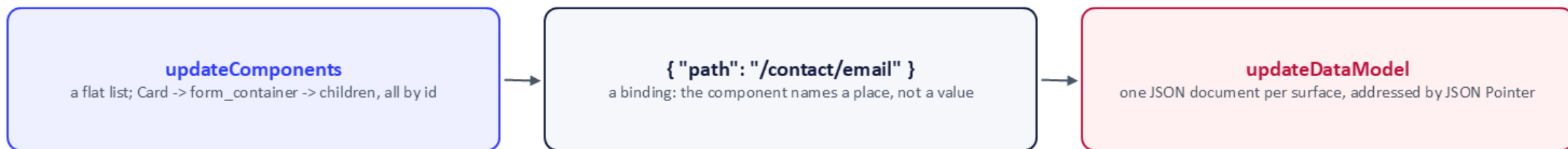
03.1 A2UI messages by hand

03.2 A2UI from a model

03.3 The official renderer, over AG-UI

Structure and data are two messages; a binding is the only join

`updateComponents` says there is a text field bound to `/contact/email`. `updateDataModel` says what is at `/contact/email`. Neither message resends the other.



The client keeps a component map and a data model per surface. A tree is never on the wire; it is rebuilt at paint time from the ids.

Takeaway A client that keeps a map of components and a pointer-addressed document can render any catalog.

The whole server-to-client protocol is four messages; everything else is the catalog

A2UI v0.9.1 is the **flat element map** from sub-theme 01, published as a protocol with a schema.

- Each message is a JSON object with a version and exactly one of four keys.
- `createSurface` opens a surface and names the catalog it will use.
- `updateComponents` sends a flat list; one component must have the id root.
- `updateDataModel` writes a value at a JSON Pointer path, or deletes it.
- `deleteSurface` drops the surface and every piece of state under it.
- Validation runs against three official schemas; a registry plugs the catalog in.

<code>createSurface</code>	surfaceId, catalogId, optional theme and sendDataModel
<code>updateComponents</code>	a flat list of components joined by ids
<code>updateDataModel</code>	path (JSON Pointer) and value, or path alone to remove
<code>deleteSurface</code>	the client forgets the surface and its state

The envelope schema. Which components may appear inside updateComponents is the catalog's business, not the envelope's.

Takeaway A2UI is a flat map plus a data model; a client that keeps those two things can render any catalog.

Three of the four builders fit on one slide; the shape is the lesson

```

1 def update_components(surface_id, components):
2     """An updateComponents message: a flat list; one component must have id
   'root'."""
3     return {"version": VERSION, "updateComponents": {"surfaceId": surface_id,
4 "components": list(components)}}
5
6 def update_data_model(surface_id, path="/", value=None, remove=False):
7     """An updateDataModel message. Omit value (remove=True) to delete the key at
   path."""
8     body = {"surfaceId": surface_id, "path": path}
9     if not remove:
10         body["value"] = value
11     return {"version": VERSION, "updateDataModel": body}
12
13
14 def delete_surface(surface_id):
15     """A deleteSurface message: the client drops the surface and its state."""
16     return {"version": VERSION, "deleteSurface": {"surfaceId": surface_id}}

```

03_a2ui/step_01_a2ui_messages_by_hand/a2ui.py

- 1 A flat list under `components`. The nested tree never crosses the wire; the client rebuilds it from ids.
- 2 `path` is a JSON Pointer into the surface's data model. Omitting `value` turns the write into a delete.
- 3 `deleteSurface` carries only the id. The page keeps nothing else; the server owns the conversation.

Each builder returns the envelope: a version and exactly one of the four keys. `validate()` checks the result against the official schemas.

The client state has no DOM in it: a map for structure, a pointer write for data

```

1 apply(message) {
2   const kind = messageType(message);
3   const body = message[kind];
4   if (kind === 'updateComponents') {
5     [1]   for (const component of body.components) this.components.set(component.id,
6         component);
7   } else if (kind === 'updateDataModel') {
8     [2]   if ('value' in body) pointerSet(this.data, body.path ?? '/', body.value);
9     else pointerDelete(this.data, body.path ?? '/');
10  } else {
11    [3]   throw new Error(`${kind} is not a surface update`);
12  }
13 }

```

03_a2ui/step_01_a2ui_messages_by_hand/static/surface.mjs

- 1 Components go into a **Map** keyed by id. Any order, any time; a parent may name a child that has not arrived.
- 2 Data goes through **pointerSet**. A bound **Text** reads the same document at paint time, so typing shows at once.
- 3 Anything else is not a surface update. **createSurface** and **deleteSurface** are the store's business, not the surface's.

tree() rebuilds the nested view from the map when the page paints, marking ids that have not arrived as **{missing: true}**.

The spec's contact form: 4 messages, 25 components, 0 missing refs, one repaired rule set

A2UI messages by hand [replay with deleteSurface](#)

[mail] **Contact Us**

First Name Last Name

First Name Last Name

Jane Doe

Email Address

Email

john.doe@example.com

Phone Number

Phone

1234567890

Preferred Contact Method

☒ Email

☐ Phone

☐ SMS

☒ Subscribe to our newsletter

Send Message

25

components in the map after updateComponents, missing refs 0

3

check rules repair_checks rewrote so the spec's own stream validates

- The second message fails the spec's own schema at /updateComponents/components/14/checks/0: condition is required.
- After updateDataModel the page shows 8 inputs, first name John, newsletter box checked.
- Clicking Send Message produces a submitContactForm action with its context resolved at click time.
- deleteSurface leaves 0 surfaces on the page; nothing else was ever stored there.

Checkpoint: reorder contact_form.jsonl so a child precedes its parent. The page renders the same form; the map ignores order.

Prompt-first: the catalog rides in the prompt, so a loop has to check the output

Nothing constrains the model at generation time; the protocol pairs the prompt with a **generate, validate, correct** loop.

- DirectJsonFormat renders the Basic Catalog schema into the system prompt.
- DirectJsonStreamParser yields updateComponents as soon as a component is reachable from root.
- Children not seen yet become empty loading_<id> rows, so the page never has a missing ref.
- The full parser repairs small faults such as trailing commas and validates each block.
- On failure the partial surfaces are deleted and the error becomes the next user turn, once.
- The report calls this the cost of the declarative middle: someone has to check.



Streaming is an optimisation on top of the validated result, never a replacement for it.

Takeaway Prompt-first generation is cheap to build and costs a full catalog schema per request, plus a second request whenever validation fails.

The SDK gives three things: a prompt, a stream parser and a full parser

```
1 def system_prompt():
2     """Role text, the SDK's workflow rules, then the full JSON schema of the
   catalog."""
3     [1] return inference_format().prompt_generator.generate(role_description=ROLE,
4     include_schema=True)
5
6 def stream_parser():
7     """Feed model text chunks in; get A2UI messages out as soon as they are
   usable."""
8     [2] return DirectJsonStreamParser(inference_format()._supported_catalogs[0])
```

03_a2ui/step_02_a2ui_from_a_model/prompt.py

1 `include_schema=True` puts the full Basic Catalog JSON schema in the prompt: 41,083 characters before the user types.

2 The stream parser is fed model chunks and yields messages as soon as they are usable. It does not repair.

`parse_reply()` is the third piece: it runs the full parser on the finished text and raises with the validator's message when the reply cannot be made valid.

The correction loop: withdraw what streamed, hand the error back, try once more

```
1 text = "".join(reply)
2 try:
3     messages, prose = prompt.parse_reply(text)
4 except ValueError as error:
5     yield "note", {"attempt": attempt, "error": str(error)[:300]}
6     for surface_id in created:
7         yield mirror(envelope.delete_surface(surface_id))
8     conversation += [{"role": "assistant", "content": text},
9                     {"role": "user", "content": f"The reply failed validation. Fix
10    it and send the complete reply again.\n{error}"}]
11    continue
```

03_a2ui/step_02_a2ui_from_a_model/server.py

1 The full parser runs on the finished reply. This is the validated result the page will keep.

2 Every surface the stream created is deleted first. The page must not keep a half-rendered, invalid form.

3 The validator's message becomes the next user turn, and the loop goes round once more; ATTEMPTS is 2.

Before this block, the same loop fed each chunk to the stream parser and yielded its `updateComponents` messages straight to the page.

createSurface at 4.86s, 38 progressive updates, the final form at 10.81s, one attempt

A2UI from a model

Make a contact form with first name, email, a newsletter checkbox and a send button. Generate

First Name

Email

☒ Subscribe to newsletter

Send

Server got 'submit' at 14:58:54 with firstName='Ada', email='ada@example.com', newsletter=True

```

0.40s POST /generate "Make a contact form with first name, email, a newsletter checkbox and a send button."
4.86s createSurface: main
5.34s updateComponents: 2 components, 1 loading placeholders, missing refs 0
5.54s updateComponents: 3 components, 1 loading placeholders, missing refs 0
5.62s updateComponents: 3 components, 1 loading placeholders, missing refs 0
5.65s updateComponents: 3 components, 1 loading placeholders, missing refs 0
5.65s updateComponents: 3 components, 1 loading placeholders, missing refs 0
5.65s updateComponents: 3 components, 1 loading placeholders, missing refs 0
5.67s updateComponents: 4 components, 2 loading placeholders, missing refs 0
5.68s updateComponents: 4 components, 2 loading placeholders, missing refs 0
5.73s updateComponents: 4 components, 2 loading placeholders, missing refs 0
5.75s updateComponents: 5 components, 3 loading placeholders, missing refs 0
  
```

9,929

prompt tokens on gpt-4.1-mini; 600 completion tokens

38

progressive updateComponents messages before the final one

- The POST went out at 0.40s; createSurface arrived at 4.86s; the final updateComponents at 10.81s.
- The click at 11.00s posted {firstName: Ada, email: ada@example.com, newsletter: true}; the status Text showed the server's reply.
- Many of the 38 messages are the same components again: the stream parser re-sends a component each time a cut string heals.

Checkpoint: a custom catalog with fewer components is the usual answer to the 41K-character prompt.

A2UI is a generation format and AG-UI is a transport; one CUSTOM event joins them

Each layer stops where its vocabulary ends: **A2A** moves messages between agents, **AG-UI** moves runs to a browser, **A2UI** names components.

- AG-UI does not know what a surface is; A2UI does not know what a run is.
- Every A2UI envelope becomes the value of one CUSTOM event named a2ui.
- Correction attempts become STEP_STARTED and STEP_FINISHED; usage rides in RUN_FINISHED.
- A click is a second run; the action and the client data model travel in forwardedProps.
- The official @a2ui/lit renderer replaces render.mjs: markdown Text and catalog functions run.
- This step has no second agent, so A2A is not on the wire; its slot is where run() reads the last user message.



The report's stack. Nothing in A2UI changes to ride AG-UI, and nothing in AG-UI changes to carry it.

Takeaway Transport and generation format are separate choices; one AG-UI event type carries the whole A2UI protocol.

On the page, the join is one case in the event switch

```

1 function onEvent(event) {
2   switch (event.type) {
3     case 'CUSTOM':
[1]       if (event.name === 'a2ui') {
[2]         processor.processMessages([event.value]);
6         const kind = Object.keys(event.value).find((k) => k !== 'version');
7         const count = event.value.updateComponents?.components.length;
8         note(`CUSTOM a2ui ${kind}${count ? ` (${count} components)` : ''}`);
9       } else {
[3]         note(`CUSTOM ${event.name} ${JSON.stringify(event.value).slice(0, 120)}`);
11      }
12      break;

```

03_a2ui/step_03_a2ui_lit_and_ag_ui/src/page.mjs

1 A CUSTOM event named `a2ui` is the whole join. Its value is one A2UI envelope, handed to the official `MessageProcessor`.

2 `processMessages` is `@a2ui/web_core`'s state machine; `A2uiSurface` elements re-render from signals, nothing is torn down.

3 Other CUSTOM names, such as `a2ui.note`, are logged. AG-UI carries them without knowing what they mean.

The processor was built once over the Lit Basic Catalog, with an alias so the spec's `v0_9_1` catalog id resolves to the `v0_9` registration.

On the server, one function maps the loop's (kind, payload) pairs onto AG-UI event classes

```

1 def to_events(agent_input, pairs):
2     """Map the loop's (kind, payload) pairs onto AG-UI event objects."""
3     ids = dict(thread_id=agent_input.thread_id, run_id=agent_input.run_id)
4     yield RunStartedEvent(**ids)
5     for kind, payload in pairs:
6         [1] if kind is None:
7             yield CustomEvent(name="a2ui", value=payload)
8         elif kind == "note":
9             yield CustomEvent(name="a2ui.note", value=payload)
10        [2] elif kind == "step":
11            step = StepStartedEvent if payload["started"] else StepFinishedEvent
12            yield step(step_name=f"attempt {payload['attempt']}")

```

03_a2ui/step_03_a2ui_lit_and_ag_ui/server.py

1

`kind is None` is an A2UI envelope: it becomes the value of a `CustomEvent` named `a2ui`.

2

Attempts of the correction loop become AG-UI steps, so a client can show 'attempt 2' without reading A2UI.

Below this, `text` becomes a TEXT_MESSAGE triple, `error` a RUN_ERROR, and `done` a RUN_FINISHED whose result carries the usage. `EventEncoder` writes the frames.

The same form drawn by @a2ui/lit 0.11.0, fed by 54 CUSTOM a2ui events between RUN_STARTED and RUN_FINISHED

A2UI over AG-UI, rendered by @a2ui/lit

Make a contact form with first name, email, a newsletter checkbox and a send button Run agent

First Name

First Name

Email

Email

☒ **Subscribe to newsletter**

Server got 'submit' at 15:04:04 with firstName='Ada', email='ada@example.com', newsletter=True; client data model had 92 bytes

```

1.63s POST /agent run with 1 message(s)
1.67s RUN_STARTED ""
1.69s STEP_STARTED "attempt 1"
7.19s CUSTOM a2ui createSurface
7.95s CUSTOM a2ui updateComponents (2 components)
8.44s CUSTOM a2ui updateComponents (3 components)
8.48s CUSTOM a2ui updateComponents (3 components)
8.48s CUSTOM a2ui updateComponents (3 components)
8.55s CUSTOM a2ui updateComponents (3 components)
8.61s CUSTOM a2ui updateComponents (3 components)

```

54

CUSTOM a2ui updateComponents events before RUN_FINISHED at 15.64s

9,929

prompt tokens; 686 completion tokens, reported in RUN_FINISHED.result

- The run started at 1.63s; createSurface arrived at 7.19s; STEP_STARTED and STEP_FINISHED bracket attempt 1.
- The click at 15.78s sent a second run; RUN_FINISHED for it came at 15.81s with {answered: submit}.
- The Text reading # Contact Us is a heading here; in step 03.1 it was raw text. The official renderer renders markdown.

Checkpoint: compare a text reading # Contact Us here and in step 03.1. The official renderer renders markdown and runs catalog functions; the hand renderer did neither.

A click's whole answer is three lines: RUN_STARTED, one CUSTOM a2ui event, RUN_FINISHED

```
click -> 15.78s action submit from sendButton
        {"firstName":"Ada","email":"ada@example.com","newsletter":true}
        15.81s RUN_FINISHED {"answered":"submit"}

wire of an action run (POST /agent), one AG-UI event per data: line:

data: {"type":"RUN_STARTED","threadId":"t1","runId":"r2"}

data: {"type":"CUSTOM","name":"a2ui","value":{"version":"v0.9.1",
        "updateDataModel":{"surfaceId":"main","path":"/form/status",
        "value":"Server got 'submit' at 15:04:04 with firstName='Ada', ...

data: {"type":"RUN_FINISHED","threadId":"t1","runId":"r2",
        "result":{"answered":"submit"}}
```

3

AG-UI events in the action run, answered in 0.03s

- The action and the client data model arrived in forwardedProps; AG-UI has no message type for them.
- The answer is one updateDataModel into /form/status, the path the server's mirror found the status Text bound to.
- One threadId for the page, a new runId per generation or click. A surface lives across runs until deleteSurface.

The status Text read: Server got 'submit' at 15:04:04 with firstName='Ada', email='ada@example.com', newsletter=True; client data model had 92 bytes.

OpenUI Lang, the line-oriented format

This sub-theme is about generation, not transport. OpenUI Lang is the language the report places in the declarative middle of its spectrum and measures as the most token-efficient open format: one statement per line, id = Component(args), positional arguments, references allowed before their definition. Sub-theme 01 streamed a JSON element map and paid for every key and bracket; this sub-theme replaces the JSON with the language, builds the parser by hand, swaps in the real packages, recomputes the report's token table cell by cell, and builds the report's hybrid on OpenUI's own reference implementation.

04.1 OpenUI Lang, parsed by hand

04.2 lang-core and react-lang

04.3 The token benchmark, reproduced

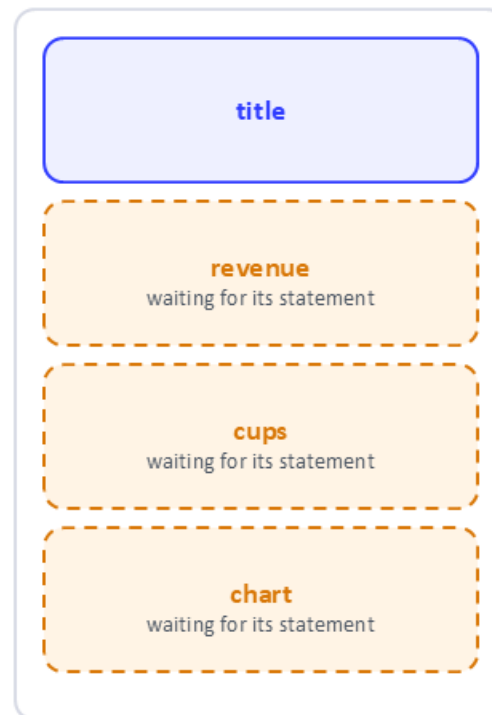
04.4 OpenUI's html artifact

One statement per line, positional arguments, forward references: the layout paints first

A JSON tree cannot render until its closing bracket arrives. An OpenUI Lang program is **renderable after its first line**.

- A newline outside brackets and strings ends a statement; the parser acts on every complete line.
- `Text("...", "heading")` has no key names; the catalog's property order names the arguments.
- `root` names `chart` five lines before `chart` is defined; the resolver turns a missing name into a placeholder.
- The renderer draws a placeholder as a grey skeleton box, so the layout appears and fills in top-down.
- The resolver is re-run on every pushed line, never patched; a half-written line is held back, not guessed at.
- Two parsers, Python for the tests and JavaScript for the page, produce byte-identical trees.

```
root = Stack([title, kpis, chart])
title = Text("Week 37", "heading")
kpis = Stack([revenue, cups],
"row")
revenue = Metric("Revenue", "$482")
cups = Metric("Cups sold", 241)
chart = BarChart("Cups", days,
[.])
days = ["Mon", "Tue", "Wed"]
```



A shortened program after three lines: the Stack and the title are drawn, three names wait for their statements.

Takeaway One statement per line, positional arguments and forward references are the whole design; the catalog, not the parser, names the arguments.

A name with no statement yet becomes a placeholder, and the renderer draws that as a skeleton

```
1 def reference(name: str) -> object:
[1]     if name not in program.statements or name in visiting:
3         unresolved.append(name)
[2]         return {"type": "placeholder", "name": name}
5     visiting.add(name)
6     try:
7         result = value(program.statements[name])
8     finally:
9         visiting.discard(name)
[3]     if isinstance(result, dict) and result.get("type") == "element":
11         result["statementId"] = name
12     return result
```

04_openui_lang/step_01_openui_lang_parser/openui_parse.py

- 1 Two reasons to give up: the statement has not arrived, or the name is already on the stack, which is a cycle.
- 2 The placeholder is a real node in the tree. The page shows it as a grey box; **unresolved** lists its name.
- 3 An element that came from a statement remembers its id, so the page can find the DOM node it produced.

The resolver walks from `root`. It runs again on every pushed line, so a placeholder is replaced the moment its statement arrives.

The parser never knows what "row" means; the catalog zips the positional arguments with names

```

1 def element(node: dict) -> object:
2     name = node["name"]
3     if name not in catalog:
4         errors.append(f"unknown component {name}")
5         return None
6     params = catalog[name]
7     if len(node["args"]) > len(params):
8         errors.append(f"{name} takes {len(params)} arguments, got
{len(node['args'])}")
9     props = {param: value(arg) for param, arg in zip(params, node["args"])}
10    return {"type": "element", "typeName": name, "props": props}

```

04_openui_lang/step_01_openui_lang_parser/openui_parse.py

- 1 The catalog is a map from component name to its parameter names, read from the `properties` order in `catalog.json`.
- 2 `zip` pairs each positional argument with a parameter name. Change the order in the catalog and the same text renders differently.
- 3 The element node carries a `typeName` and named `props`; the renderer keys one DOM function on the name.

Checkpoint: swap `direction` and `gap` under `Stack` in `catalog.json` and render `program.oui` again. The catalog, not the parser, names the arguments.

After three lines the title is drawn and six forward references are skeletons

OpenUI Lang, parsed by hand
Render the whole program
Stream it line by line
3 statements, 6 unresolved

Lemonade stand, week 37

Program so far

```

root = Stack([title, kpis, chart, table, actions], "column", "1")
title = Text("Lemonade stand, week 37", "heading")
kpis = Stack([revenue, cups, margin], "row", "m")

```

Unresolved references

```

actions
chart
cups
margin
revenue
table

```

5

names unresolved after line 1, and the layout already exists

6

skeleton boxes on the page after three lines

- Line 1 is `root = Stack([title, kpis, chart, table, actions], ...)`: five placeholders, one Stack.
- Line 3 adds `kpis` with three more names; the count goes up before it comes down.
- Each grey box is a placeholder node the resolver returned for a name with no statement yet.
- Nothing in a JSON tree format is renderable at this point; the root object has not closed.

The recorded run streams `program.oui` one line per SSE message and re-renders after every push.

After the last line every skeleton has been replaced; both parsers agree byte for byte

OpenUI Lang, parsed by hand Render the whole program Stream it line by line 12 statements, 0 unresolved

Lemonade stand, week 37

REVENUE
\$482
 +12%

CUPS SOLD
241
 +8%

MARGIN
63%
 -2%

Cups per day

Flavour	Cups	Price
Classic	140	\$2.00
Pink	61	\$2.50
Ginger	40	\$3.00

Next step
Lemons run out on Saturday. Order before Thursday.

Order lemons

Program so far

```

root = Stack([title, kpis, chart, table, actions], "column", "1")
title = Text("Lemonade stand, week 37", "heading")
kpis = Stack([revenue, cups, margin], "row", "m")
revenue = Metric("Revenue", "$482", "+12%")
cups = Metric("Cups sold", 241, "+8%")
margin = Metric("Margin", "63%", "-2%")
chart = BarChart("Cups per day", days, [28, 35, 31, 44, 52, 51])
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
table = Table(["Flavour", "Cups", "Price"], [{"Classic", 140, "$2.00"}, {"Pink", 61, "$2.50"}, {"Ginger", 40, "$3.00"}])
actions = Card("Next step", [note, restock])
note = Text("Lemons run out on Saturday. Order before Thursday.", "small")
restock = Button("Order lemons", "restock", "primary")
  
```

Unresolved references

(none)

12

statements parsed from program.oui

0

unresolved references and 0 errors after line 12

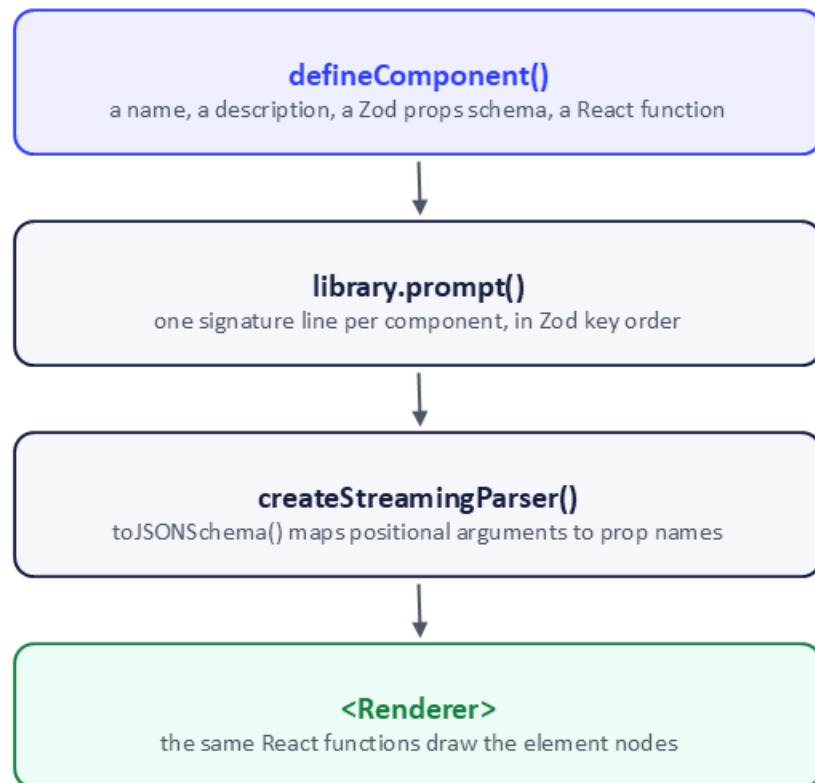
- Line 7 references days one line before line 8 defines it; the chart is a skeleton for one line, then replaced.
- `kpis = Stack([revenue, cups, margin], "row", "m")` has no key names at all; the catalog supplied them.
- The Python and JavaScript parsers produce byte-identical trees for `program.oui`; `test_step.py` checks that.
- The reference parser goes one step further and auto-closes the pending line so a half-written statement renders too.

Checkpoint: swap the order of direction and gap under Stack in catalog.json. The same text renders differently.

One Zod catalog generates the prompt, the parser's parameter map and the renderers

Step 04.1 kept the catalog, the parser and the renderer in three files. The official packages derive **all three from one source**.

- `defineComponent()` takes a name, a description, a Zod props schema and a React function.
- `library.prompt()` prints one signature line per component, in Zod key order, plus the language rules.
- `library.toJSONSchema()` is the step 04.1 catalog shape; the streaming parser reads it to name the arguments.
- `<Renderer>` from `@openui/react-lang` re-parses the accumulated text on every chunk and draws the catalog's components.
- The server forwards every model delta as one SSE message and never parses the program itself.
- React 19 ships CommonJS only, so esbuild bundles the page once; no config file, no dev server.



One definition, three uses. Add a component and the prompt, the parser and the renderer change together.

Takeaway In the declarative mode the catalog is code, not prose, and the model can only produce what the catalog allows.

Three Zod keys become the signature line, the parameter map and the props the renderer receives

```

1 const Metric = defineComponent({
2   name: "Metric",
3   [1] description: "One number with a label and an optional change",
4   [2] props: z.object({
5     label: z.string(),
6     value: z.union([z.string(), z.number()]),
7     delta: z.string().optional().describe('such as "+12%"),
8   }),
9   [3] component: ({ props }) =>
10     h("div", { className: "metric" },
11       h("div", { className: "label" }, props.label),
12       h("div", { className: "value" }, String(props.value)),
13       h("div", { className: `delta ${String(props.delta ?? "").startsWith("-") ?
14         "down" : "up"}` }, props.delta ?? "")),
15   });

```

04_openui_lang/step_02_openui_react_lang/library.mjs

- 1 The description goes into the prompt after the signature. Text on a prop's `.describe()` does not; put what the model must know here.
- 2 Key order is argument order: `Metric (label, value, delta?)`. The prompt, the parser and the renderer cannot disagree.
- 3 The React function receives `{ props }` with names already mapped. It is React without JSX, so `h` is `createElement`.

The same object produces `Metric(label: string, value: string | number, delta?: string)` — One number with a label and an optional change in the prompt.

The system prompt is generated, not written: eight signature lines in Zod key order

```

1 ## Component Signatures
2
3 [1] Arguments marked with ? are optional. Sub-components can be inline or referenced;
4 prefer refere...
5 Stack(children: any[], direction?: "column" | "row", gap?: "s" | "m" | "l") - Lays
children out...
6 Text(text: string, size?: "small" | "body" | "heading") - A run of text
7 [2] Metric(label: string, value: string | number, delta?: string) - One number with a
label and an ...
8 Table(columns: string[], rows: (string | number)[][])) - A table with column headers
and rows of...
9 BarChart(title: string, labels: string[], values: number[]) - A bar chart with one
series
10 Card(title: string, children: any[]) - A bordered box with a title and children
11 Button(label: string, action: string, variant?: "primary" | "secondary") - A button
that fires ...
12 [3] Input(name: string, placeholder?: string, type?: "text" | "email" | "number") - A
single-line t...

```

04_openui_lang/step_02_openui_react_lang/prompt.txt

1

The library's own streaming advice: prefer references over inline calls, so the layout paints before its contents.

2

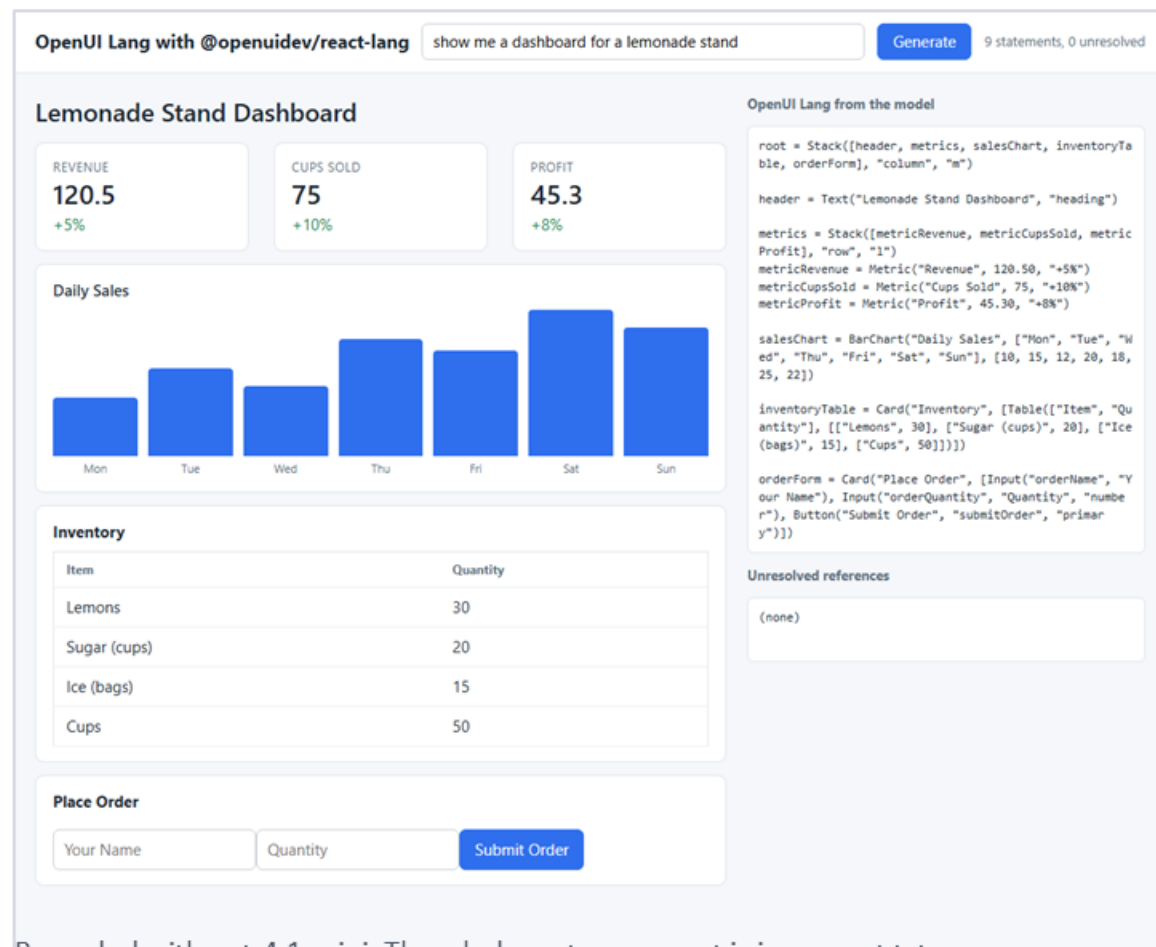
The `Metric` line from the previous slide. A `?` marks an optional argument; optional arguments can be omitted from the end.

3

The whole prompt is 48 lines and 3163 characters; `server.py` caches it in `prompt.txt` and sends it as the system message.

`npm run prompt` regenerates the file. Add a ninth `defineComponent()` and a ninth signature line appears, with no other file touched.

The model wrote 9 statements with 0 unresolved, and the catalog's React components drew them



Recorded with gpt-4.1-mini. The whole system prompt is in prompt.txt.

48

lines, 3163 characters, in the generated system prompt

9

statements from gpt-4.1-mini for the lemonade-stand prompt, 0 unresolved

- A heading, three metrics in a row, a seven-day bar chart, an inventory table inside a card, and an order form.
- The model wrote an inline Table(...) inside a Card and left blank lines between groups; the parser accepts both.
- parse.meta.errors carries codes such as unknown-component and missing-required; a production app would send them back as a repair turn.
- Every delta is one SSE message; the page accumulates them in one string and hands it to <Renderer>.

The report's token table, recomputed cell by cell: every one of the 28 numbers reproduces

Steps 04.1 and 04.2 asserted that the line-oriented syntax is cheaper than JSON. This step **measures the claim** on the report's own seven scenarios.

- samples/ holds the seven .oui programs the report's benchmark recorded and the 53-component schema.json.
- convert.py projects each parsed tree into Thesys C1 JSON, json-render's element map and patch stream, and YAML.
- All four texts are projections of one parsed tree, so syntax is the only variable.
- Every text is counted with tiktoken's o200k_base; seconds are the token count at 60 tokens per second.
- The projections are byte-identical to the OpenUI repository's committed artifacts; test_step.py checks that.
- Not reproduced: the model run. The .oui samples are the ones the report generated; this step does not regenerate them.

4800

tokens, OpenUI Lang,
seven scenarios

9122

tokens, YAML, the
cheapest JSON-shaped
format

10180

tokens, json-render
patches, the most
expensive

28 of 28

per-scenario cells that
match the report's table

3.04x

contact form, complete
render, patches over
OpenUI Lang

23%

cheaper: the element
map without its patch
envelope

Takeaway The ratio is stable near 2x because JSON pays per field; first paint is a different quantity and favours OpenUI Lang only when the program is written root-first.

One parsed tree, four texts: the projection is the whole experimental design

```

1 def projections(oui: str) -> dict[str, str]:
2     """The same UI in every format. The .oui text is the model's own output."""
3     result = parse(oui if oui.endswith("\n") else oui + "\n", CATALOG)
4     if result.root is None or result.unresolved or result.errors:
5         raise ValueError(f"sample did not parse cleanly: {result.unresolved}
6         {result.errors}")
7     spec = to_spec(result.root)
8     return {
9         "oui": oui,
10        "yaml": to_yaml(spec),
11        "cl": cl_text(result.root),
12        "jsonl": to_jsonl(spec),
13        "map": compact(spec),
14    }

```

04_openui_lang/step_03_format_benchmark/benchmark.py

- 1 The step 04.1 parser, copied unchanged, with the benchmark's 53-component schema as the catalog.
- 2 A sample that does not parse cleanly stops the run. Every scenario must be one complete tree before it is projected.
- 3 The .oui text is the model's own output; the other four are projections of its tree, which is also how the report produced them.

map is the element map as one compact document, a column the report does not have; it sits between OpenUI Lang and YAML.

First paint is a different quantity: only OpenUI Lang has a renderable prefix

```

1 def first_paint_tokens(texts: dict[str, str]) -> dict[str, int]:
2     """Tokens that must arrive before a renderer can draw anything.
3
4     [1] OpenUI Lang: the first line (`root = ...`) is a complete statement.
5     [2] json-render patches: the `/root` patch names an element that arrives in
6         the last line (children are added before parents), so the whole stream.
7         C1 and YAML are one nested document: the whole text.
8     """
9     [3] first_line = texts["oui"].split("\n", 1)[0] + "\n"
10    return {"oui": count(first_line), "yaml": count(texts["yaml"]), "c1":
11           count(texts["c1"]),
12           "jsonl": count(texts["jsonl"]), "map": count(texts["map"])}

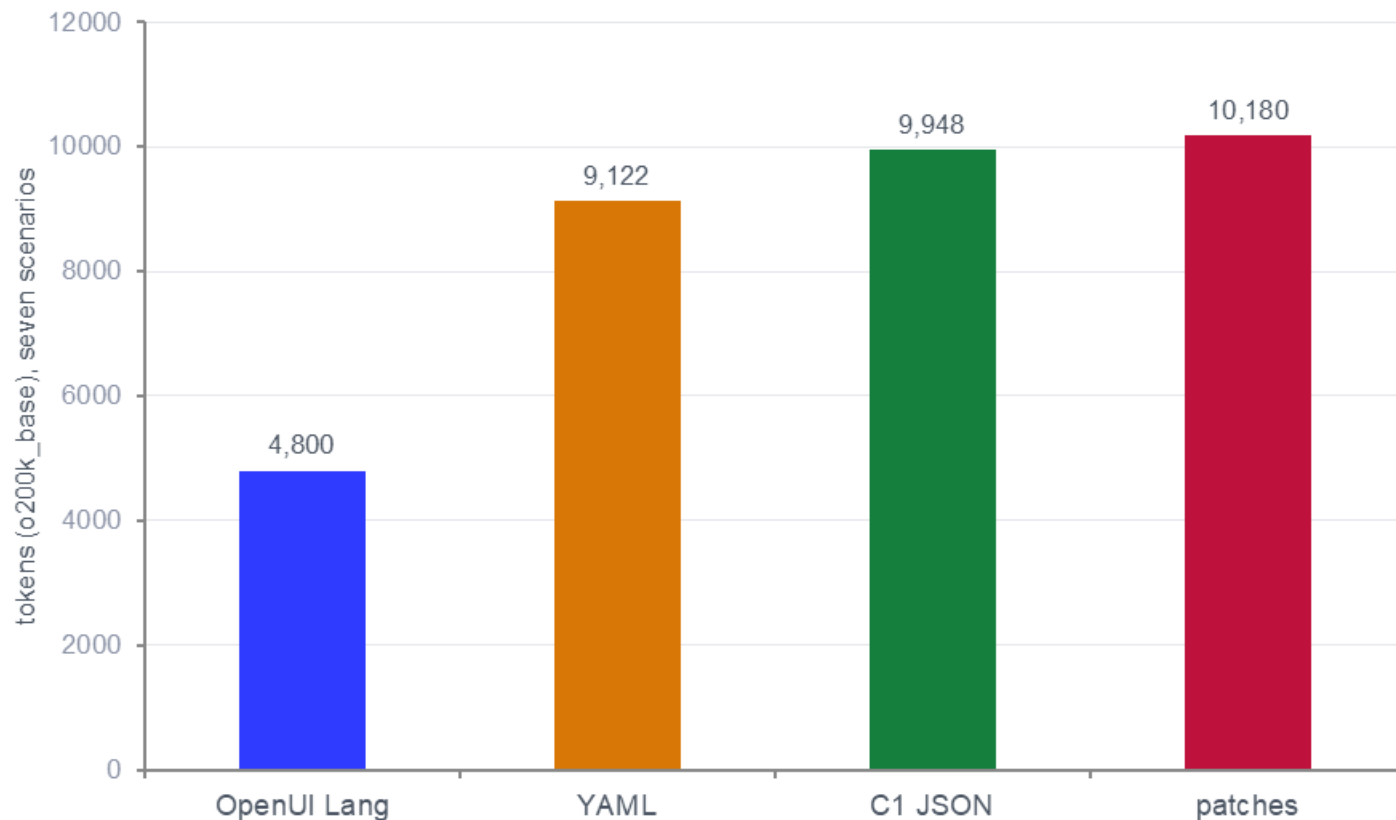
```

04_openui_lang/step_03_format_benchmark/benchmark.py

- 1 `root = Stack(...)` is a complete statement on line 1. A renderer can draw the shell after those tokens.
- 2 The report's patch projection inserts children before parents, so the `/root` patch arrives last and first paint is the whole stream.
- 3 For C1 and YAML the count is the whole text: a nested document renders nothing before its last bracket.

The report's 3.04x is a ratio of complete-render times at a constant token rate. First paint favours OpenUI Lang far more, but only root-first.

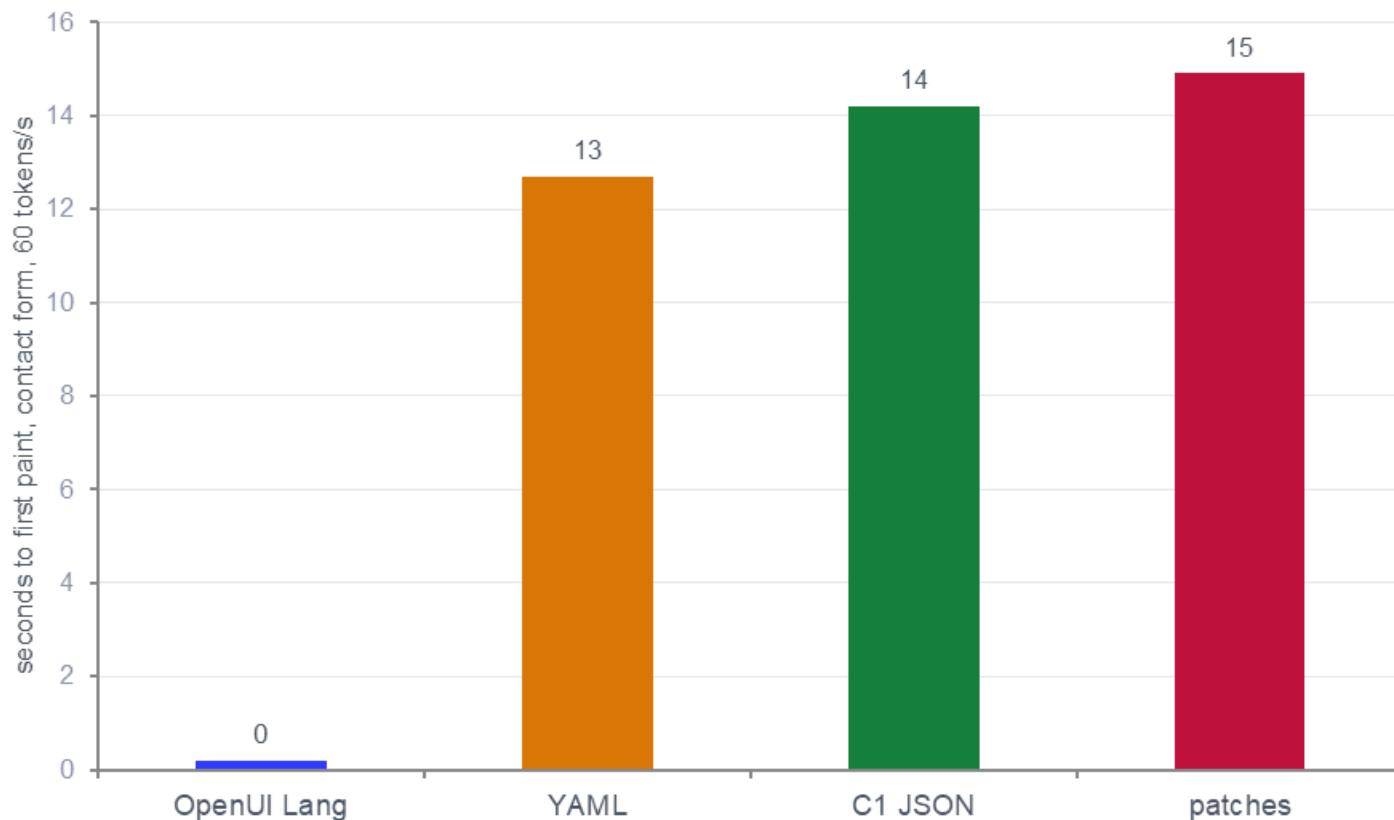
OpenUI Lang totals 4800 tokens across seven scenarios; every JSON-shaped format is near 2x



- Every cell of the report's table reproduces: the four totals and all 28 per-scenario numbers.
- The savings column is nearly flat, 42.4% to 61.4% against YAML, because JSON pays per field, not per string.
- YAML is the cheapest JSON-shaped format: it drops quotes and most brackets but still pays for every key name.
- The patch stream is the most expensive: about 11 tokens of envelope per element, over 212 elements.

The committed .c1.json files were reformatted by prettier after the count and come out 10 to 94 tokens lighter than the table; the .oui and patch files count exactly as tabled.

At 60 tokens per second the contact form paints at 0.2 s in OpenUI Lang and after the whole document elsewhere



- Only OpenUI Lang has a renderable prefix: `root = Stack(...)` is a complete statement on line 1.
- C1 JSON and YAML are one nested document; nothing renders before the last bracket.
- The report's patch projection inserts children before parents, so its `/root` patch is the last line.
- Complete render: 4.9 s against 14.9 s for patches, the report's 3.04x.
- First paint favours OpenUI Lang only when the program is written root-first; the generated prompt insists on it.

Seconds are token counts at a constant 60 tokens per second, the report's own convention; no model was timed.

The table as printed: four totals, 28 cells, and the report's own numbers underneath

```
tokens (o200k_base)
scenario      OpenUI  YAML  C1  JSON  patches  vs YAML
simple-table   148    316    357    340   -53.2%
chart-with-data 231    464    516    520   -50.2%
contact-form   294    762    849    893   -61.4%
dashboard     1226   2128   2261   2247   -42.4%
pricing-page   1195   2230   2379   2487   -46.4%
settings-panel  540   1077   1205   1244   -49.9%
e-commerce-product 1166   2145   2381   2449   -45.6%
TOTAL         4800   9122   9948   10180  -47.4%
report TOTAL   4800   9122   9948   10180

seconds at 60 tokens/s: complete (first paint)
contact-form    4.9 (0.2)  12.7 (12.7)  14.2 (14.2)  14.9 (14.9)
e-commerce-product 19.4 (0.2)  35.8 (35.8)  39.7 (39.7)  40.8 (40.8)
```

28 of 28

per-scenario cells equal to the report's table

-47.4%

OpenUI Lang against YAML in total; -51.7% against C1, -52.8% against patches

7873

tokens for the element map alone, 23% under the patch stream

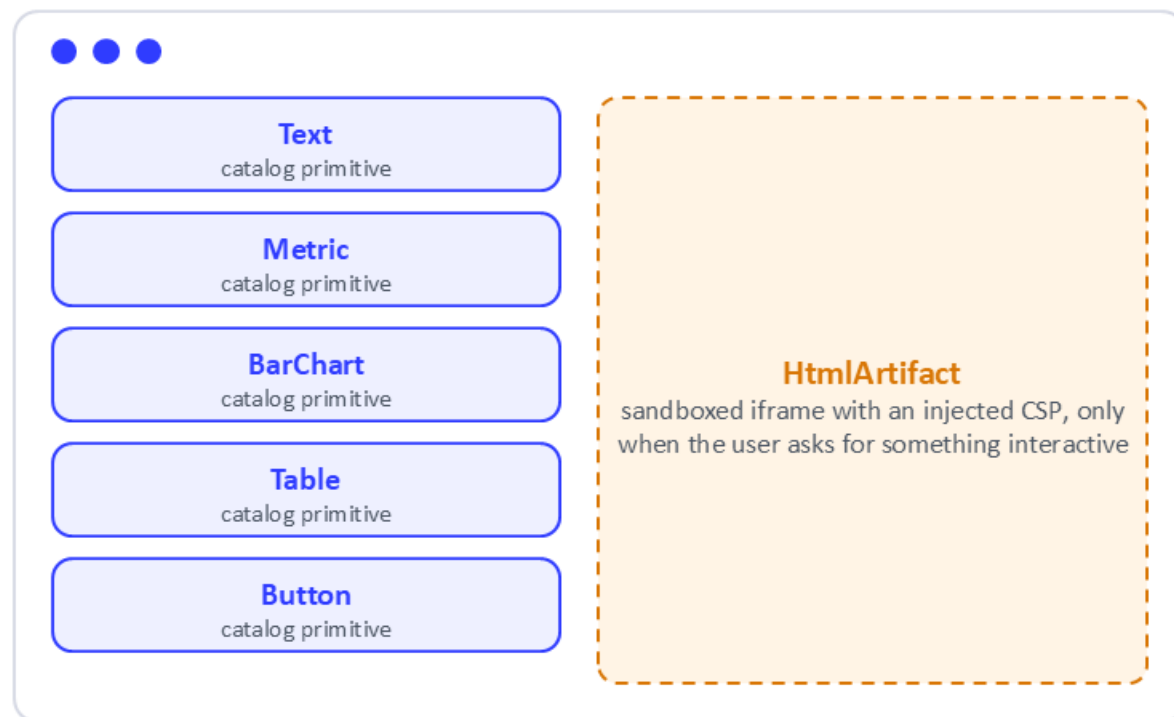
- The savings column runs from -42.4% (dashboard) to -61.4% (contact form): flat, because the overhead is per field.
- Checkpoint: the map column is not in the report. The difference to patches is the {"op": "add", "path": "..."} envelope, about 11 tokens per element over 212 elements.

python demo.py; the third table, not shown, counts the repository's committed C1 and YAML files as checked in and marks the cells that differ.

The hybrid is one prompt rule and one component: catalog by default, an HtmlArtifact where the catalog does not reach

Step 04.2 gave the model a catalog and nothing else, so an interactive request had nowhere to go. This step builds the escape hatch on **OpenUI's examples/miscellaneous/html-artifact** reference implementation.

- The catalog keeps the eight step 04.2 components and adds Markdown and HtmlArtifact(title, document).
- Two prompt rules from the example: never put a dashboard into an HtmlArtifact; only use one when the user asks for something interactive.
- The document is the second string argument of one statement; it streams like any other string, with no second endpoint and no tool call.
- `usesStreaming()` shows the raw source while the document grows, then Raw and Rendered tabs.
- `sandbox.mjs` injects a Content Security Policy first in `<head>`, lists external resources, and accepts one message shape from the iframe.
- The example defers the CSP, the document check and the message check to "before production"; this step does them.



The report's diagram on OpenUI's reference implementation: catalog primitives by default

Takeaway Open-ended generation took 86% of one reply, so the catalog is the default and the sandbox the exception.

useIsStreaming() decides between the raw source and the iframe; sandboxed() adds the CSP before srcDoc

```

1 function HtmlArtifactRenderer({ props }) {
[1]   const isStreaming = useIsStreaming();
3   const [view, setView] = useState(isStreaming ? "raw" : "rendered");
4   useEffect(() => {
5     if (!isStreaming) setView("rendered");
6   }, [isStreaming]);
7
[2]   isStreaming || view === "raw"
9     ? h("pre", { className: "artifact-raw" }, props.document)
10    : h("iframe", {
11      title: props.title,
12      className: "artifact-frame",
[3]      sandbox: "allow-scripts",
14      referrerPolicy: "no-referrer",
15      srcDoc: sandboxed(props.document),

```

04_openui_lang/step_04_openui_html_artifact/html-artifact.mjs

- 1 The library's own streaming flag, read through a hook. The renderer needs no prop from the page to know the document is still arriving.
- 2 While streaming, or on the Raw tab, a `pre` shows the document as it stands: the partial value grows on every chunk, not a spinner.
- 3 `sandbox="allow-scripts"` gives the document a unique origin and no access to the host; `sandboxed()` takes the network away too.

The full component is 31 lines; the head, the tabs and the problems list are elided here. The Zod schema underneath is two strings: `title` and `document`.

The CSP goes first in <head>, and a policy the model wrote itself is removed before it can loosen ours

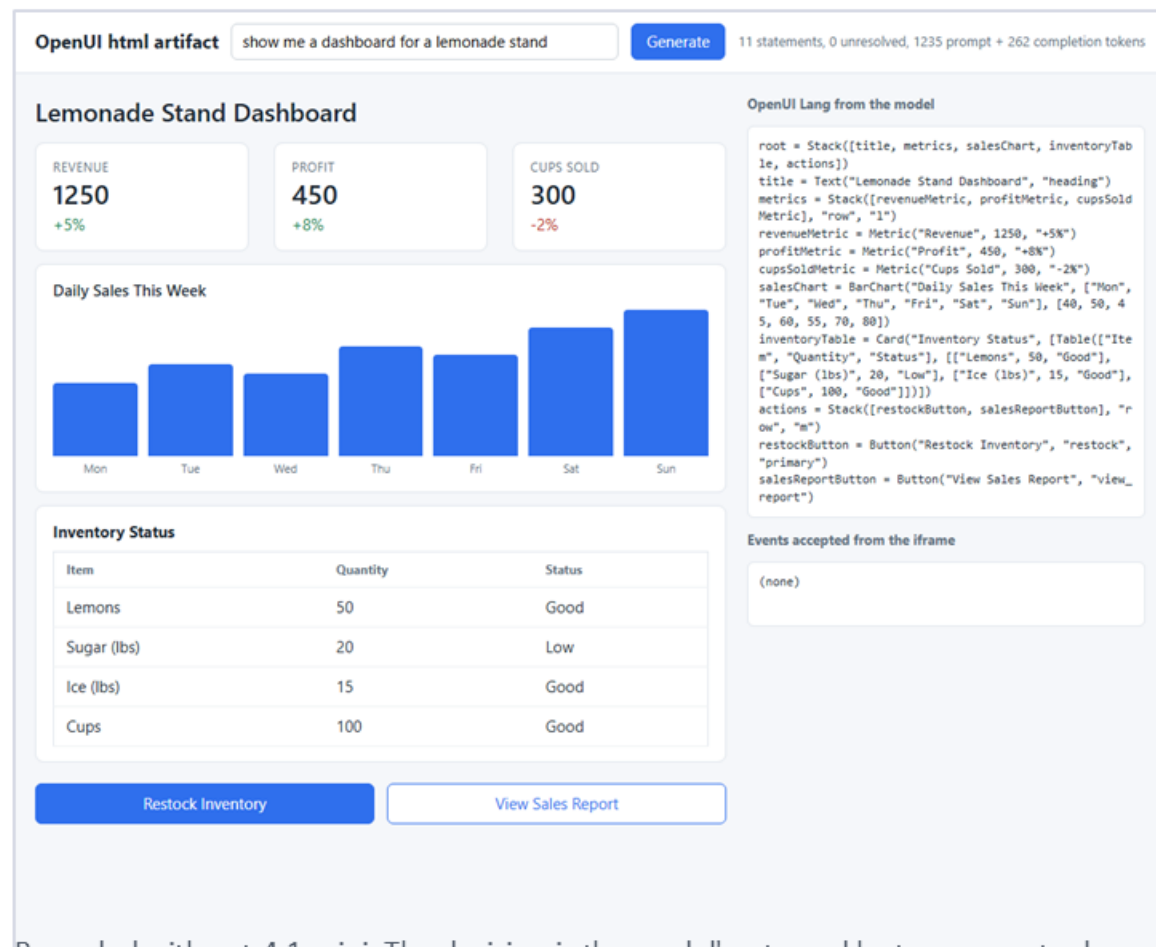
```
[1] export const CSP = "default-src 'none'; style-src 'unsafe-inline'; script-src
'unsafe-inline'; img-src data:";
2
3 export const META = `<meta http-equiv="Content-Security-Policy" content="\${CSP}">`;
4
5 const CSP_META_RE = /<meta[^>]+http-equiv\s*=\s*"'"?content-security-
policy["']?[^>]*>/gi;
6
7 export function sandboxed(html) {
[2]   const cleaned = html.replace(CSP_META_RE, "");
9   const head = /<head[^>]*>/i.exec(cleaned);
[3]   if (head) return cleaned.slice(0, head.index + head[0].length) + META +
cleaned.slice(head.index + head[0].length);
11   return META + cleaned;
12 }
```

04_openui_lang/step_04_openui_html_artifact/sandbox.mjs

- 1 `default-src 'none'`: no fetch, no WebSocket, no external script or stylesheet. Inline style and script stay allowed because the prompt asks for them.
- 2 Any Content-Security-Policy meta the model wrote is stripped first, so the document cannot loosen the policy it is about to receive.
- 3 A CSP meta tag applies only to what comes after it: insert right after the opening `<head>`, or prepend when there is none.

`checkDocument()` lists external resources, network calls and meta refresh next to the artifact; `isEvent()` keeps one message shape from the iframe and drops the rest.

The dashboard prompt stays in the catalog: 11 statements, 262 completion tokens, no artifact



11

catalog statements for "show me a dashboard for a lemonade stand"

262

completion tokens, 1235 prompt tokens

- Metric, BarChart, a Table inside a Card, and two Buttons in a row: the same components as step 04.2.
- The rule that kept it here: "Never put a dashboard into an HtmlArtifact."
- The completion token count from the API equals the o200k_base count of the program, so the token shares are exact.
- Checkpoint: remove that rule from PROMPT_OPTIONS, ask again, and compare the completion tokens with 262.

Recorded with gpt-4.1-mini. The decision is the model's, steered by two prompt rules.

The calculator reply mid-stream: the Markdown intro rendered, the raw document growing

OpenUI html artifact
Streaming
3 statements, 0 unresolved

Here's an interactive lemonade price calculator. Adjust the price per cup and the number of cups sold to see your total revenue.

Lemonade Price Calculator
Generating artifact ... 429 characters so far

```
<!doctype html><html><head><style>body{font-family:system-ui;padding:1rem;max-width:400px;margin:auto}label{display:block;margin-top:1rem}input[type=number]{width:100%;padding:.5rem;font-size:1rem}output{display:block;margin-top:1rem;font-weight:bold;font-size:1.25rem}</style></head><body><h1>Lemonade Price Calculator</h1><label>Price per cup ($):<input type="number" id="price" min="0" step="0.01" value="1.00"></label><label>
```

OpenUI Lang from the model

```
root = Stack([Intro, artifact])
Intro = Markdown("Here's an interactive lemonade price calculator. Adjust the price per cup and the number of cups sold to see your total revenue.")
artifact = HtmlArtifact("Lemonade Price Calculator", "
<!doctype html><html><head><style>body{font-family:system-ui;padding:1rem;max-width:400px;margin:auto}label{display:block;margin-top:1rem}input[type=number]{width:100%;padding:.5rem;font-size:1rem}output{display:block;margin-top:1rem;font-weight:bold;font-size:1.25rem}</style></head><body><h1>Lemonade Price Calculator</h1><label>Price per cup ($):<input type="number" id="price" min="0" step="0.01" value="1.00"></label><label>
```

Events accepted from the iframe

(none)

460

characters of the document at the moment of the screenshot

2

children of root: a Markdown, then one HtmlArtifact

- The status line reads "Generating artifact ... 460 characters so far"; the pre below it is the partial value the streaming parser exposes.
- `parse.meta.incomplete` is true for most of the stream, because the document is one string that has not closed.
- The Markdown child arrived first, as the prompt rule asks, and is already rendered above the artifact.
- No iframe exists yet: `uselsStreaming()` is true, so the renderer shows source, not a sandbox.

The prompt: "build me an interactive lemonade price calculator I can play with".

The calculator runs in its sandboxed iframe: typing 9 into the price field changed the total to \$90.00

OpenUI html artifact build me an interactive lemonade price calculator I can use to see my total revenue. Generate 3 statements, 0 unresolved, 1238 prompt + 341 completion tokens

Here's an interactive lemonade price calculator. Adjust the price per cup and the number of cups sold to see your total revenue.

Lemonade Price Calculator
Raw Rendered

Lemonade Price Calculator

Price per cup (\$):

Number of cups sold:

Total Revenue: \$90.00

OpenUI Lang from the model

```

root = Stack([intro, artifact])
intro = Markdown("Here's an interactive lemonade price calculator. Adjust the price per cup and the number of cups sold to see your total revenue.")
artifact = HtmlArtifact("Lemonade Price Calculator", "
<doctype html><html><head><style>body{font-family:system-ui;padding:1rem;max-width:400px;margin:auto}label{display:block;margin-top:1rem}input[type=number]{width:100%;padding:.5rem;font-size:1rem}output{display:block;margin-top:1rem;font-weight:bold;font-size:1.25rem}</style></head><body><h1>Lemonade Price Calculator</h1><label>Price per cup ($)</label><input type='number' id='price' min='0' step='0.01' value='1.00'></label><label>Number of cups sold:</label><input type='number' id='cups' min='0' step='1' value='10'></label><output id='total'>Total Revenue: $10.00</output><script>const priceInput=document.getElementById('price');const cupsInput=document.getElementById('cups');const totalOutput=document.getElementById('total');function updateTotal(){const price=parseFloat(priceInput.value)||0;const cups=parseFloat(cupsInput.value)||0;const total=(price*cups).toFixed(2);totalOutput.textContent='Total Revenue: $'+total;}priceInput.addEventListener('input',updateTotal);cupsInput.addEventListener('input',updateTotal);updateTotal();</script></body></html>")

```

Events accepted from the iframe

(none)

341

completion tokens for the calculator reply, 1238 prompt tokens

292

of those 341 tokens are the document: 86% of the reply

- `sandbox='allow-scripts'`, and the CSP meta is first in `<head>`: the document's script ran under the policy.
- Events accepted from the iframe: none. The document posted nothing, and `isEvent()` would have dropped anything else.
- The dashboard reply cost 262 tokens; open-ended generation is paid only for the one component that needs it.

The report's cost of open-ended generation, paid once: the catalog is the default and the sandbox the exception.

json-render, Vercel's declarative renderer

json-render is one of the open formats in the declarative middle of the report: the model emits a JSON document that names components from a catalog you wrote, and a renderer draws it. Sub-theme 01 built that loop by hand. This sub-theme hands the prompt, the parser and the renderer to the library: one `defineCatalog` object produces the system prompt, the typed component map, the validation and the JSON Schema. The spec is a flat element map that streams as RFC 6902 patches, and the same spec renders in a browser and in a terminal.

05.1 A catalog and the React Renderer

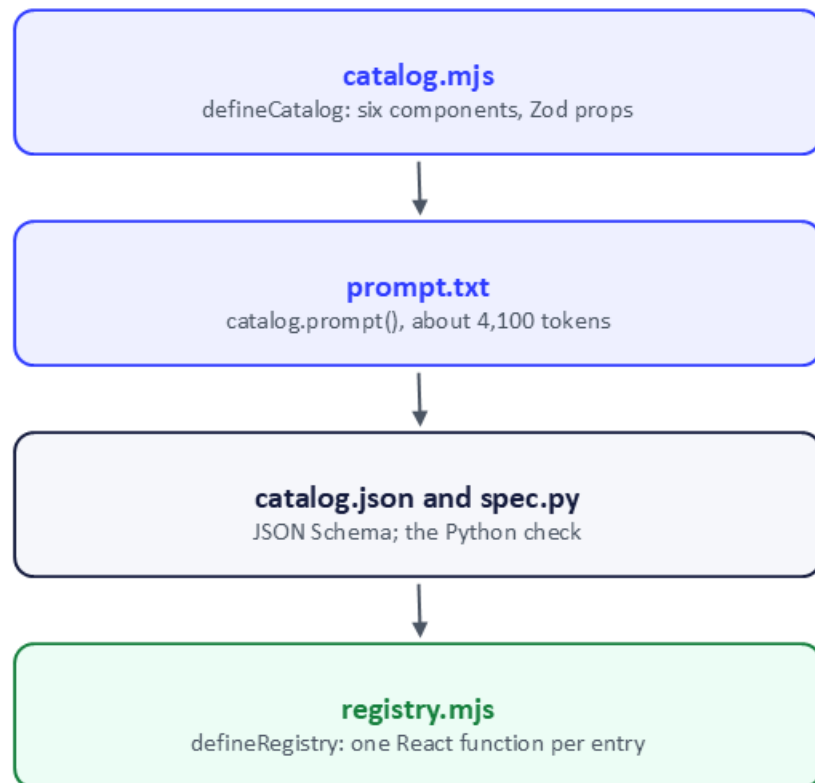
05.2 Streaming JSON Patch into the element map

05.3 Actions, and a second target

The catalog is the contract: one object, four outputs

`defineCatalog` with Zod props produces the **prompt**, the **JSON Schema**, the **typed registry** and the **validator**, and the page paints once when the whole spec arrives.

- Six components with Zod props; `.nullable()` marks a prop the model may skip and the prompt shows it as `subtitle?`.
- `catalog.prompt()` writes the system prompt; four extra rules ask for one JSON object instead of the library's native JSONL.
- Python never imports the catalog: `prompt.mjs` and `catalog_json.mjs` run once and the text is cached in `prompt.txt` and `catalog.json`.
- `spec.py` checks the reply against that JSON Schema: unknown types, dangling child ids, wrong prop types.
- `defineRegistry` maps each entry to a React function written with `htm`; no JSX, no bundler, an import map of pinned esm.sh builds.
- The spec is a flat element map `{root, elements, state}`, not a nested tree; elements name each other by id.



Change a prop in the catalog and three of the four outputs regenerate; only the registry needs a hand edit.

Takeaway One annotation in the catalog reaches three layers: the prompt, the JSON Schema and the Python check.

Six components, Zod props, one description each

```
[1] export const catalog = defineCatalog(schema, {
  2   components: {
  3     Card: {
  4       props: z.object({
  5         title: z.string(),
  6         subtitle: z.string().nullable(),
  7       }),
  8       description: "A titled container. Put related elements inside it.",
  9     },
 10     Row: {
 11       props: z.object({
 12         gap: z.enum(["small", "large"]).nullable(),
 13       }),
 14       description: "Lays its children out side by side.",
 15     },
 16   },
 17 });
```

05_json_render/step_01_json_render_catalog/catalog.mjs

- 1 `schema` from `@json-render/react/schema` fixes the spec shape: `{root, elements: {id: {type, props, children}}}`.
- 2 `.nullable()` is the one annotation: the prompt shows `subtitle?`, the JSON Schema allows null, `spec.py` accepts it.
- 3 The description goes into the generated prompt word for word; it is the model's only guide to when to use the component.

The same file serves the prompt export, the JSON Schema export, the page and the Node tests.

The server checks the spec for the three faults a renderer fails on quietly

```

1 for element_id, element in elements.items():
2     kind = element.get("type")
[1]     if kind not in components:
4         problems.append(f"{element_id}: unknown type {kind!r}")
5         continue
6     for child in element.get("children", []):
[2]         if child not in elements:
8             problems.append(f"{element_id}: child {child!r} is not in elements")
[3]     props = {k: v for k, v in element.get("props", {}).items() if not
is_expression(v)}
10     expressions = set(element.get("props", {})) - set(props)
11     validator = jsonschema.Draft202012Validator(props_schema(components[kind],
expressions))
12     for error in validator.iter_errors(props):
13         where = "/".join(str(p) for p in error.path) or "props"
14         problems.append(f"{element_id}: {kind}.{where}: {error.message}")

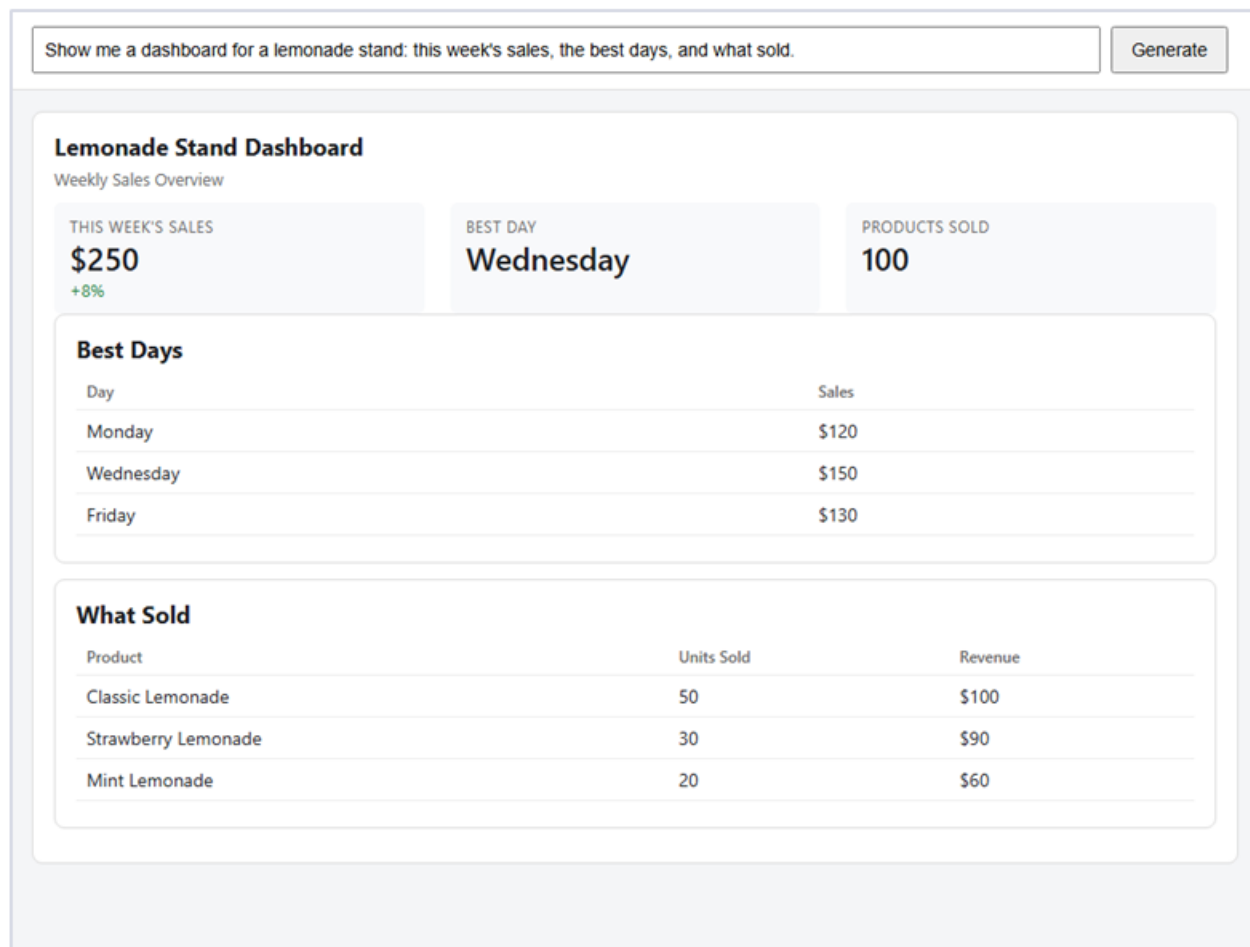
```

05_json_render/step_01_json_render_catalog/spec.py

- 1 An unknown type: the catalog has no such component, so the renderer would draw nothing for that element.
- 2 A dangling child id: a parent names an element that is not in the map.
- 3 Expression props such as `{"$state": ...}` are set aside; the renderer resolves them before the component sees them.

Python cannot run the Zod schema, so it validates against the JSON Schema `catalog_json.mjs` exported: two runtimes, one catalog.

The whole dashboard arrived at 5.8 s and painted once



5.8 s

first paint, after the complete spec

9

elements in the spec

- Three metrics in a row, then two cards with tables: the model's own layout from six components.
- The three numbers live in `state`; the metrics read them with `$state` and `$template` props.
- Nothing rendered before 5.8 s; the reply was 351 tokens, the prompt 4,109.

Most of the 4,100-token prompt is the library's text about state, repeats, events and visibility: the price of a general renderer.

A flat element map paints on its second line

The library's native format is **one RFC 6902 patch per line**; each element is a line complete on its own, so the page renders after every patch.

- The four "one object" rules are gone; `llm.py` streams with no JSON mode, because JSONL is many objects.
- The first line sets `/root`; the second adds the root element, and the page can paint.
- A parent lists children that have not arrived; the renderer skips them until their line comes.
- `json_patch.py` mirrors the library's compiler with its two tolerances: `add` creates a missing parent, `replace` a missing target.
- `server.py` relays every chunk unchanged and compiles a copy, so it ends with the spec, the patches and the timings.
- `app.mjs` pushes each chunk into `createSpecStreamCompiler`; `Renderer` gets an empty element map until `/elements` exists.

<code>add /root</code>	the root id, "dashboard"; nothing to draw yet
<code>add /elements/dashboard</code>	a Card with a title: the page paints, first paint
<code>add /elements/row-metrics</code>	a Row the card already named as a child
<code>add /elements/metric-sales</code>	a Metric whose value is a \$state reference
<code>add /state/sales/total</code>	the number the metric reads; it fills in on arrival
<code>... 13 patches in all</code>	9 elements; the last line may arrive without a newline

The element map fills one line at a time; every line is a complete JSON object with an op, a path and a value.

Takeaway Streaming changes when the user sees something, not the prompt cost: 4,043 prompt tokens, 487 completion tokens.

Push each chunk into the compiler; render only when a patch applied

```

1 for await (const chunk of chunks(response)) {
[1]   const { result, newPatches } = compiler.push(chunk);
3   if (newPatches.length === 0) continue;
4   // The first patch sets /root alone; Renderer reads spec.elements[spec.root]
5   // with no guard, so give it an empty map until /elements arrives.
[2]   setSpec(result.elements ? result : { ...result, elements: {} });
7   if (firstPaint === null && result.root && result.elements?.[result.root]) {
8     firstPaint = seconds();
9     setStatus(`first paint at ${firstPaint} s`);
10  }
11 }
[3] setSpec(compiler.getResult()); // applies a last line that had no newline

```

05_json_render/step_02_json_render_streaming_patches/app.mjs

1 `push` returns the spec so far and the patches this chunk completed; `newPatches` is empty while a line is still partial.

2 The first patch sets `/root` alone and `Renderer` has no guard for that, so the page hands it an empty element map.

3 A stream may end without a newline; `getResult()` applies that last line.

`result` is a new object whenever a patch applied, so `setSpec` re-renders only on a real change.

The server forwards every chunk and compiles a copy on the way

```

1 def relay(prompt):
2     """Forward each chunk as it arrives, and compile a copy on the way."""
3     started = time.perf_counter()
4     stream = llm.stream_text(system_prompt(), prompt)
5     [1] compiler = SpecStream()
6     timings = {"first_chunk": None, "first_paint": None, "complete": None}
7     for chunk in stream:
8         if timings["first_chunk"] is None:
9             timings["first_chunk"] = time.perf_counter() - started
10        compiler.push(chunk)
11    [2] if timings["first_paint"] is None and compiler.has_root():
12        timings["first_paint"] = time.perf_counter() - started
13    [3] yield chunk
14    compiler.finish()
15    timings["complete"] = time.perf_counter() - started

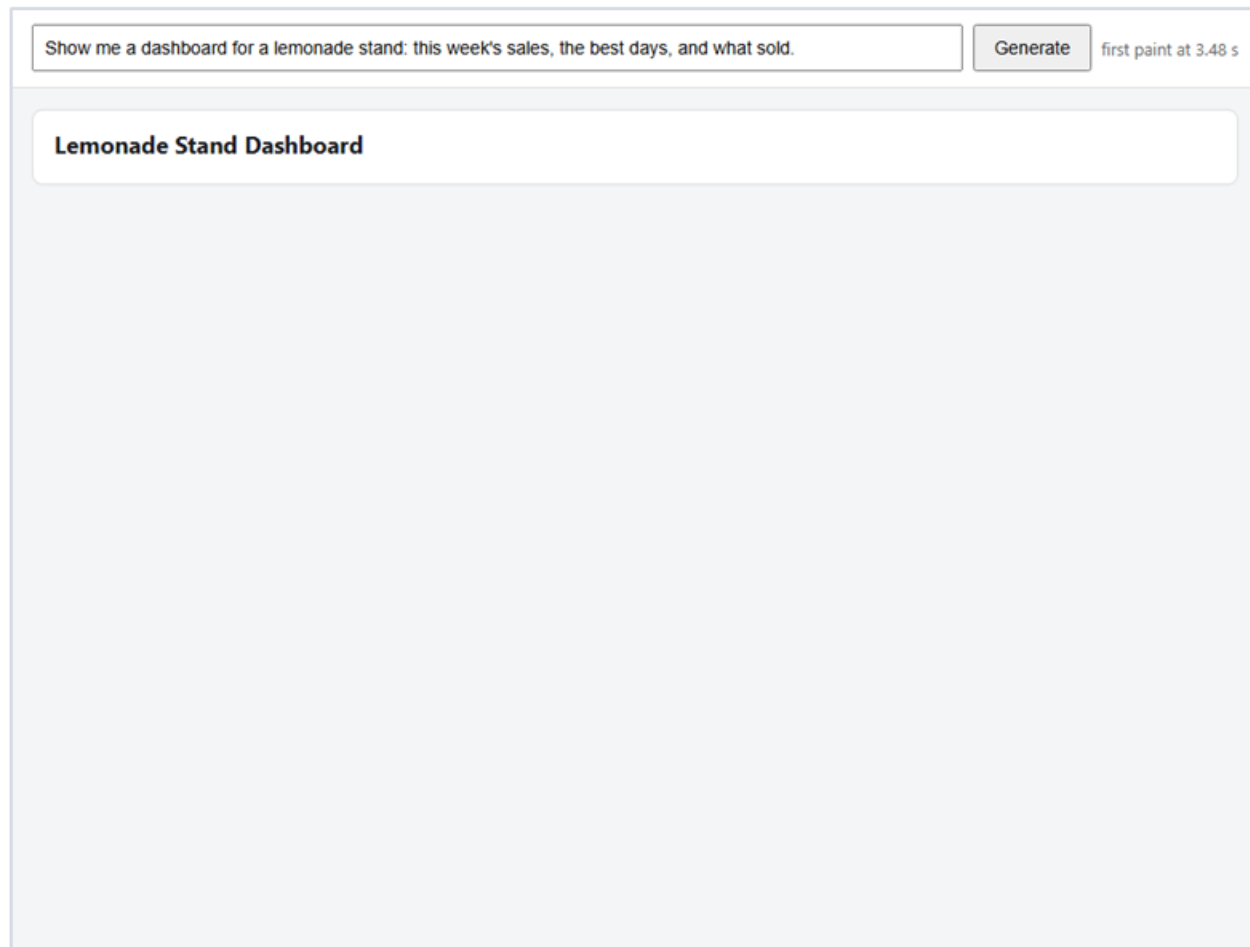
```

05_json_render/step_02_json_render_streaming_patches/server.py

- 1 `SpecStream` is the Python mirror of `createSpecStreamCompiler`: the same lines, the same two tolerances.
- 2 `has_root()` is the moment a first paint became possible; the demo compares it with the page's own timestamp.
- 3 The chunk goes to the page unchanged; the server never re-encodes the model's text.

The server ends with the compiled spec, the patch list, the usage and three timings, all at `GET /last`.

At 3.48 s the page shows one card with a title and no children yet



3.48 s

first paint on the page

2.53 s

first chunk at the server

- The status line reads "first paint at 3.48 s": the root Card has painted with its title alone.
- The Card already names `row-metrics` as a child; the renderer skips it until its line arrives.
- The 2.5 s before the first chunk is the model reading a 4,000-token prompt; no format removes that.

Step 05.1 showed nothing until 5.8 s; this page shows something 2.3 s earlier.

The same page when the stream ends at 10.14 s, after 13 patches

first paint at 3.48 s, complete at 10.14 s, 13 patches

Lemonade Stand Dashboard

THIS WEEK'S SALES
\$500
+5%

BEST DAY
Wednesday
+20%

PRODUCTS SOLD
250
+10%

Best Days

Day	Sales
Monday	50
Wednesday	75
Friday	65

What Sold

Product	Units Sold	Revenue
Classic Lemonade	120	\$240
Strawberry Lemonade	80	\$160
Mint Lemonade	50	\$100

10.14 s

complete on the page

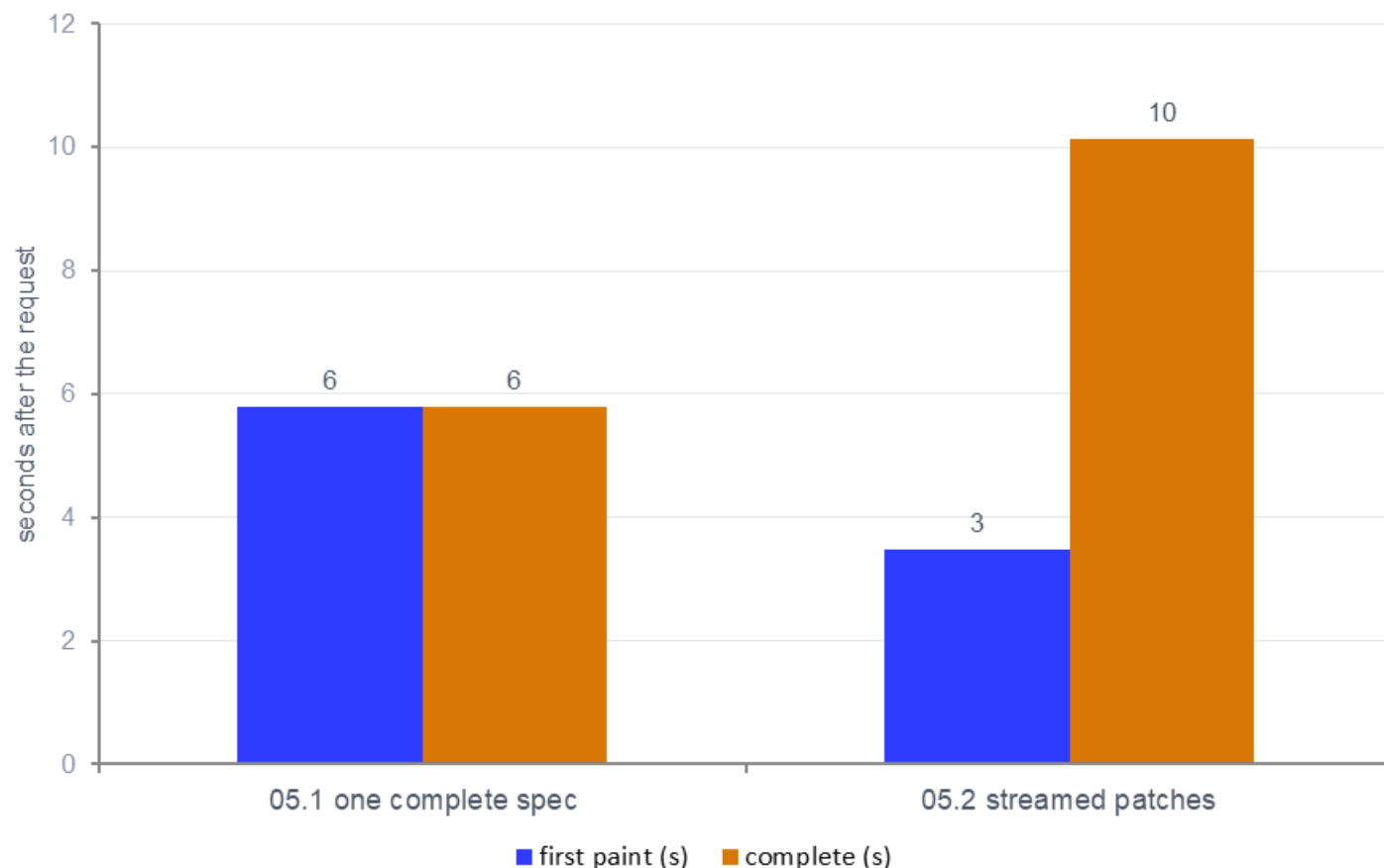
13

patches, 9 elements

- The status line now reads first paint 3.48 s, complete 10.14 s, 13 patches.
- Three metrics filled in from `/state/...` patches; a fresh copy of `spec.state` per version is what lets them reach the store.
- The catalog check passed on the compiled spec, the same check as step 05.1.

The whole reply took longer than step 05.1's 5.8 s; what changed is when the first element appeared.

One paint at 5.8 s, or a first element at 3.48 s and the last at 10.14 s



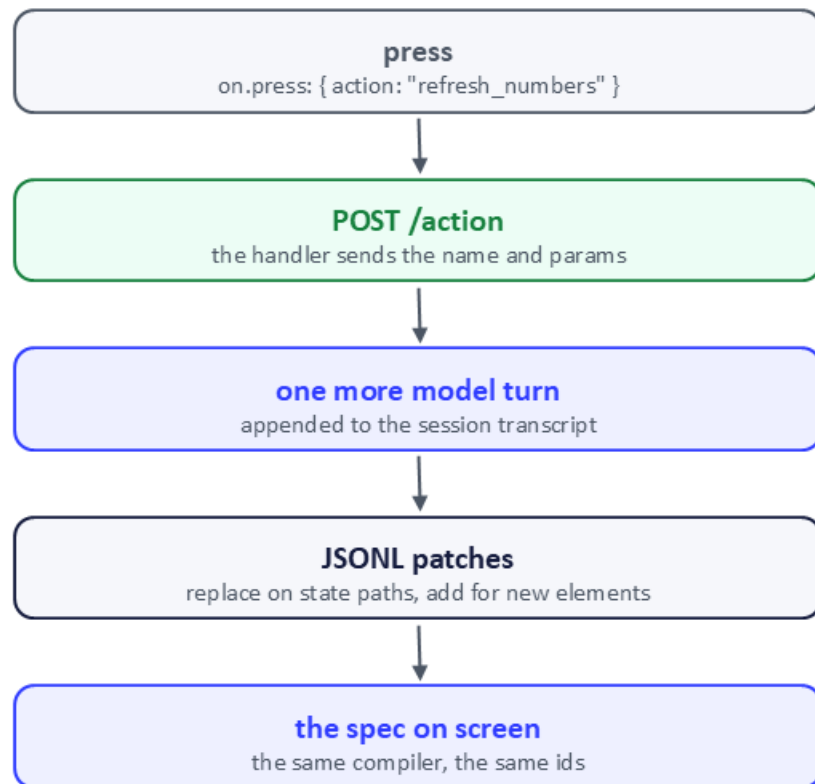
- In step 05.1 first paint and complete are the same moment: nothing renders before the last bracket closes.
- Streaming moved the first paint 2.3 s earlier and the last element 4.3 s later in this pair of runs.
- Both runs paid about 4,000 prompt tokens; the reply was 351 and 487 tokens.
- The 2.5 s to the first chunk is prompt reading and belongs to neither format.

Numbers from the recorded demos of steps 05.1 and 05.2; runs vary with the model's speed.

A press is data the model answers with more data

A `Button` binds its press with an ``on`` field on the element; catalog actions go to the server for the next turn, `setState` stays in the page.

- Two catalog actions, `refresh_numbers` and `show_details`, declared with Zod `params`; the model's press carries a typed `metric`.
- The page maps each action to one handler: `POST /action`, then push the reply into the same compiler, kept in a ref.
- `server.py` keeps one transcript per session; a press becomes a user message that asks for patches against the spec already produced.
- Built-in actions (`setState`, `pushState`, `removeState`) change the state model and never leave the page.
- `spec.py` now checks every `on` binding against the declared actions, before the page drops one with a console warning.
- The loop the report describes: the model edits a document, and the document is the UI.



setState takes the short path: the page changes its state model and the server never hears about it.

Takeaway The compiler lives in a ref so the second turn's patches land on the spec the first turn built.

Every catalog action is the same handler: post it, then compile the reply

```

1 async function serverAction(action, params) {
2   const started = performance.now();
3   note(`${action} ${JSON.stringify(params)} -> server`);
4   const response = await fetch("/action", {
5     method: "POST",
6     headers: { "content-type": "application/json" },
7     body: JSON.stringify({ action, params }),
8   });
9   [1] const patches = await consume(response);
10  [2] note(`${action}: ${patches} patches applied in $((performance.now() - started) /
11    1000).toFixed...
12  }
13  const handlers = {
14    [3] refresh_numbers: (params) => serverAction("refresh_numbers", params ?? {}),
15    show_details: (params) => serverAction("show_details", params ?? {}),

```

05_json_render/step_03_json_render_actions_and_targets/app.mjs

- 1 **consume** is shared with the first turn: it pushes the reply into the compiler in the ref and returns the patch count.
- 2 The action log under the dashboard in the demo is these **note** calls: name, params, patches, seconds.
- 3 Two catalog actions, two handlers, one function; **params** comes from the spec's **on** binding.

setState never reaches a handler; the page's **onStateChange** is the only trace it keeps of a built-in action.

The second target: the same catalog, a component map of ink Box and Text

```

1 export const components = {
[1]   Card: ({ element, children }) =>
3     h(Box, { flexDirection: "column", borderStyle: "round", paddingX: 1,
marginBottom: 0 },
4       h(Text, { bold: true }, element.props.title),
5       element.props.subtitle ? h(Text, { dimColor: true }, element.props.subtitle) :
null,
6       children),
[2]   Row: ({ children }) => h(Box, { flexDirection: "row", gap: 2 }, children),
8   Metric: ({ element }) =>
9     h(Box, { flexDirection: "column", marginRight: 2 },
10      h(Text, { dimColor: true }, element.props.label.toUpperCase()),
11      h(Text, { bold: true }, String(element.props.value)),
12      element.props.delta ? h(Text, { color: "green" }, element.props.delta) :
null),
13   Button: ({ element }) => h(Text, { color: "cyan" }, `[ ${element.props.label} ]`),
14 };
15
[3] export const InkRenderer = createRenderer(catalog, components);

```

05_json_render/step_03_json_render_actions_and_targets/ink_render.mjs

- 1 The ink package hands components the whole **element**; the React one hands them **props**. The catalog and the spec are shared.
- 2 A Row is a horizontal Box; a Card is a bordered column. Layout words change, the spec does not.
- 3 **createRenderer(catalog, components)** wraps the providers; a **state** prop seeds the **\$state** expressions.

tests/targets.test.mjs renders one spec through both targets and checks that every text the page shows, the terminal shows.

One catalog, one spec, two component maps

The element map is data about **components and state**, not about a medium; each target supplies its own component map and nothing else changes.

catalog.mjs

seven components and two actions with Zod props; the prompt, the schema and the validator come from it

demo_spec.json

the spec after generate, refresh_numbers and show_details: root, elements, state, on bindings

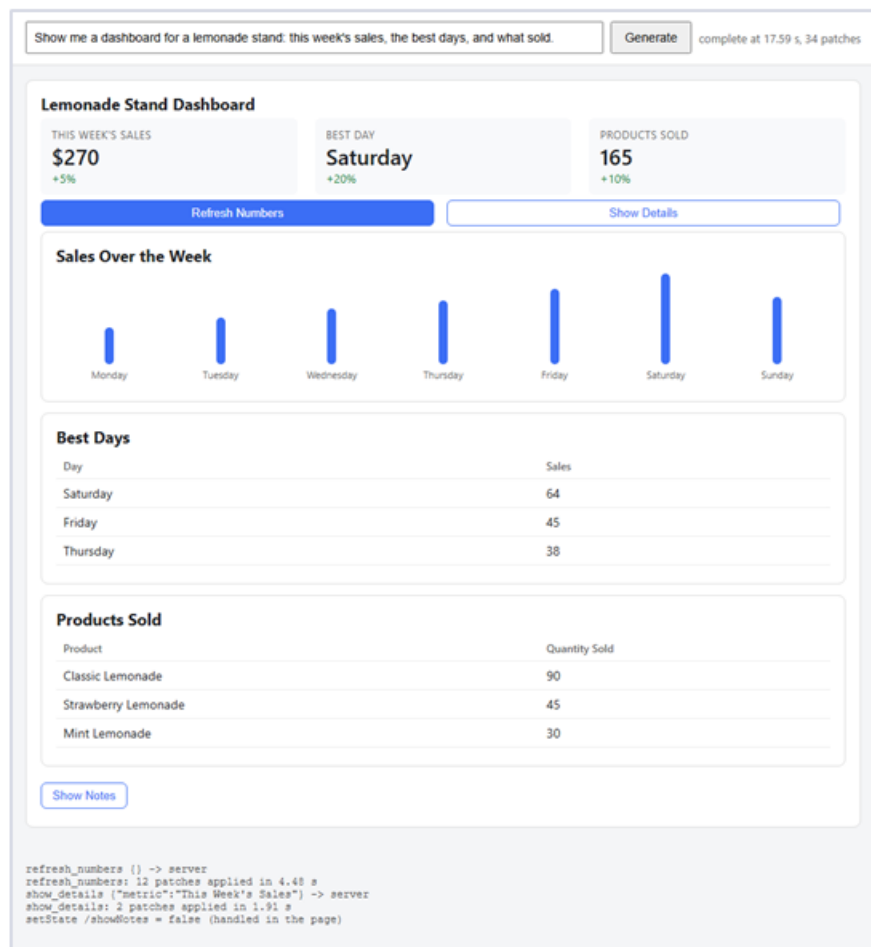
registry.mjs in the browser · ink_render.mjs in the terminal

React functions with htm, or ink Box and Text; the json-render family also has React Native, Vue, Svelte, PDF and email

The same demo_spec.json produced the page screenshot and the terminal render on the next slides.

Takeaway Write the catalog once; add a target by writing one component map for it.

Three presses: two went to the model as turns, one stayed in the page



34

patches in the first turn, 17.59 s, 1,228 completion tokens

12

patches for refresh_numbers, 4.48 s, 282 tokens

- The action log at the bottom: `refresh_numbers {} -> server`, then `show_details {"metric": "This Week's Sales"} -> server`, then `setState /showNotes = false` (handled in the page).
- The refresh turn was all `replace` on state paths, one of them a single array cell, `/state/sales/dailySales/0`; the metrics and the chart changed without a new spec.
- The toggle reads "Show Notes" because the last press hid the card locally; the server's spec still has `/showNotes` at true.

Twelve patches refreshed a whole dashboard for 282 output tokens, because the first turn put the numbers in state.

The same spec in the terminal, and it shows the notes card the page hid

```
Lemonade Stand Dashboard
THIS WEEK'S SALES   BEST DAY   PRODUCTS SOLD
$270                Saturday   165
+5%                 +20%       +10%
[ Refresh Numbers ] [ Show Details ]
```

```
Sales Over the Week
Monday      ██████████ 22
Saturday    ████████████████████ 64
```

```
[ Hide Notes ]
```

```
Notes
Detailed sales analysis:
```

1

spec file, demo_spec.json, through both targets

0

lines skipped by the catalog check

- `node ink_render.mjs demo_spec.json`: the same values, \$270, Saturday, 165, drawn with Box and Text.
- The button reads "[Hide Notes]" and the Notes card is present: the server's spec has the model's `/showNotes = true` patch.
- The page hid that card with `setState`, which never reached the server. Built-in actions are local state; if the agent must know, send them.

Cut to its top; the full render in the step README has the whole chart, both tables and the notes text.

MCP Apps, UI in a host you do not control

The report's second transport. Sub-theme 02 put the agent inside a product you own. Here the agent lives in someone else's chat client, and you ship the interface to it as an MCP resource: a `ui://` URI, an HTML document, a tool that points at it, and a JSON-RPC bridge over `postMessage` between a sandboxed iframe and the host. The spec is SEP-1865. Both steps use FastMCP on the server and no SDK anywhere else. At the end, one server renders in a browser host, in a terminal host, and in Claude Desktop, unchanged.

06.1 A tool result that carries its interface

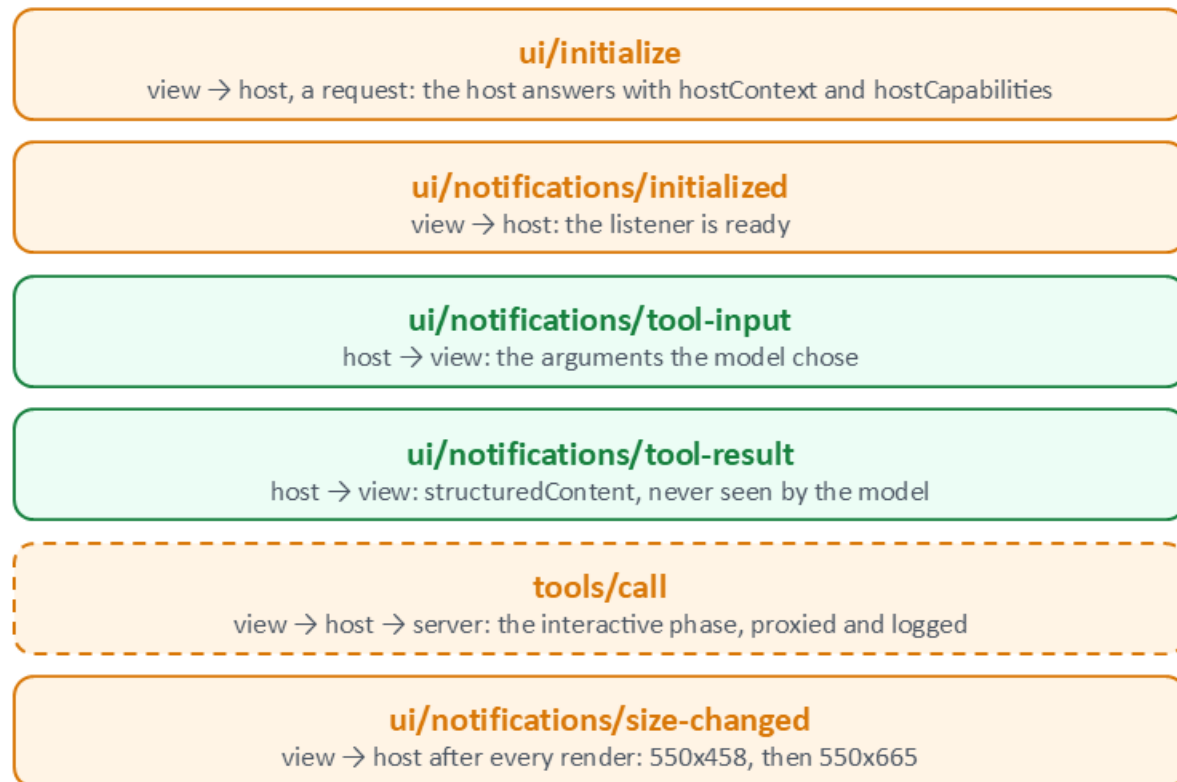
06.2 The same server in a real host

06.3 Hybrid inside an MCP App

The interface travels as a resource; the data arrives later, as a notification

A tool result has **two audiences**: **content** for the model and for text-only hosts, **structuredContent** for the view in the iframe.

- The server declares one resource at `ui://lemonade/dashboard.html`, MIME type `text/html;profile=mcp-app`.
- The tool's `_meta.ui.resourceUri` links the tool to that resource; a host without the extension ignores it.
- The host reads the resource, builds a CSP from its metadata, mounts it with `sandbox="allow-scripts"`.
- The view speaks JSON-RPC over `postMessage` in three functions; no SDK on either side.
- The order is fixed: `ui/initialize`, `initialized`, then `tool-input` and `tool-result`, then a size report.
- The view never talks to the server. Every `tools/call` goes through the host, which can log or refuse it.

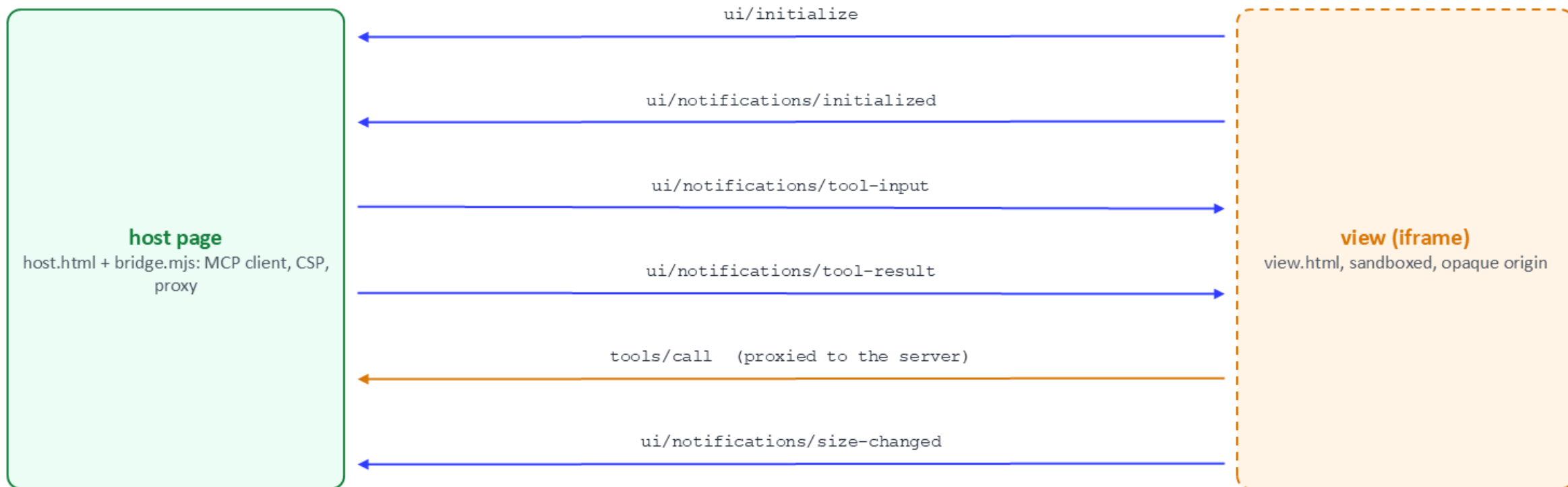


The six messages of the demo, in the order the bridge log printed them.

Takeaway The resource is static and declared up front; the data comes as a tool result; every message between them passes through a host that can refuse it.

Three parties, two channels: the view never talks to the server

MCP over HTTP or stdio between the server and the host; **JSON-RPC over `postMessage`** between the host and the sandboxed iframe. Every request from the view crosses the host.



Blue: the lifecycle and the two data notifications. Amber: a request the host proxies to the server.

Takeaway The host holds tool-input and tool-result until the view says initialized; send them earlier and the listener is not there yet.

One tool, one resource, and the `_meta` link between them

```
[1] @server.tool(meta={"ui": {"resourceUri": VIEW_URI}})
2 def lemonade_dashboard(days: int = 7) -> types.CallToolResult:
3     """Sales dashboard for the lemonade stand over the last `days` days (1 to 28)."""
4     days = max(1, min(int(days), 28))
5     rows = data.sales(days)
6     return types.CallToolResult(
7         content=[types.TextContent(type="text", text=data.as_text(days, rows))],
8         structuredContent={"days": days, "rows": rows, "summary": data.summary(rows...
9     )
10
11
12 [3] @server.resource(VIEW_URI, name="lemonade_dashboard_view", mime_type=MIME_TYPE, met...
13 def dashboard_view() -> str:
14     """The dashboard's HTML document. Static: the data arrives later, over postMess...
15     return VIEW_FILE.read_text(encoding="utf-8")
```

06_mcp_apps/step_01_mcp_app_resource/server.py

- 1 `meta` becomes `_meta` on the wire.
`ui.resourceUri` is the spec's field: it tells a host which resource to mount after this tool runs.
- 2 Two halves of one result. `content` is prose the model reads. `structuredContent` is the same numbers as data; the view renders it and the model never sees it.
- 3 The resource: a `ui://` URI, the MIME type `text/html;profile=mcp-app`, and a `meta` with an empty CSP, so the host allows no external connection or script.

A normal FastMCP server plus three things: a `ui://` resource, a tool that points at it, and a result with text and data.

The view's lifecycle: initialize, say so, then report the size

```

1 // --- lifecycle: initialize, then say so, then report the size -----
2 (async () => {
3   [1] const init = await request("ui/initialize", {
4     appInfo: { name: "Lemonade Dashboard", version: "0.1.0" },
5     appCapabilities: { availableDisplayModes: ["inline"] },
6     protocolVersion: "2025-06-18",
7   });
8   applyContext(init.hostContext);
9   [2] notify("ui/notifications/initialized", {});
10   new ResizeObserver(() => {
11     [3] notify("ui/notifications/size-changed", { width: document.body.scrollWidth,...
12   }).observe(document.body);
13 }) ();

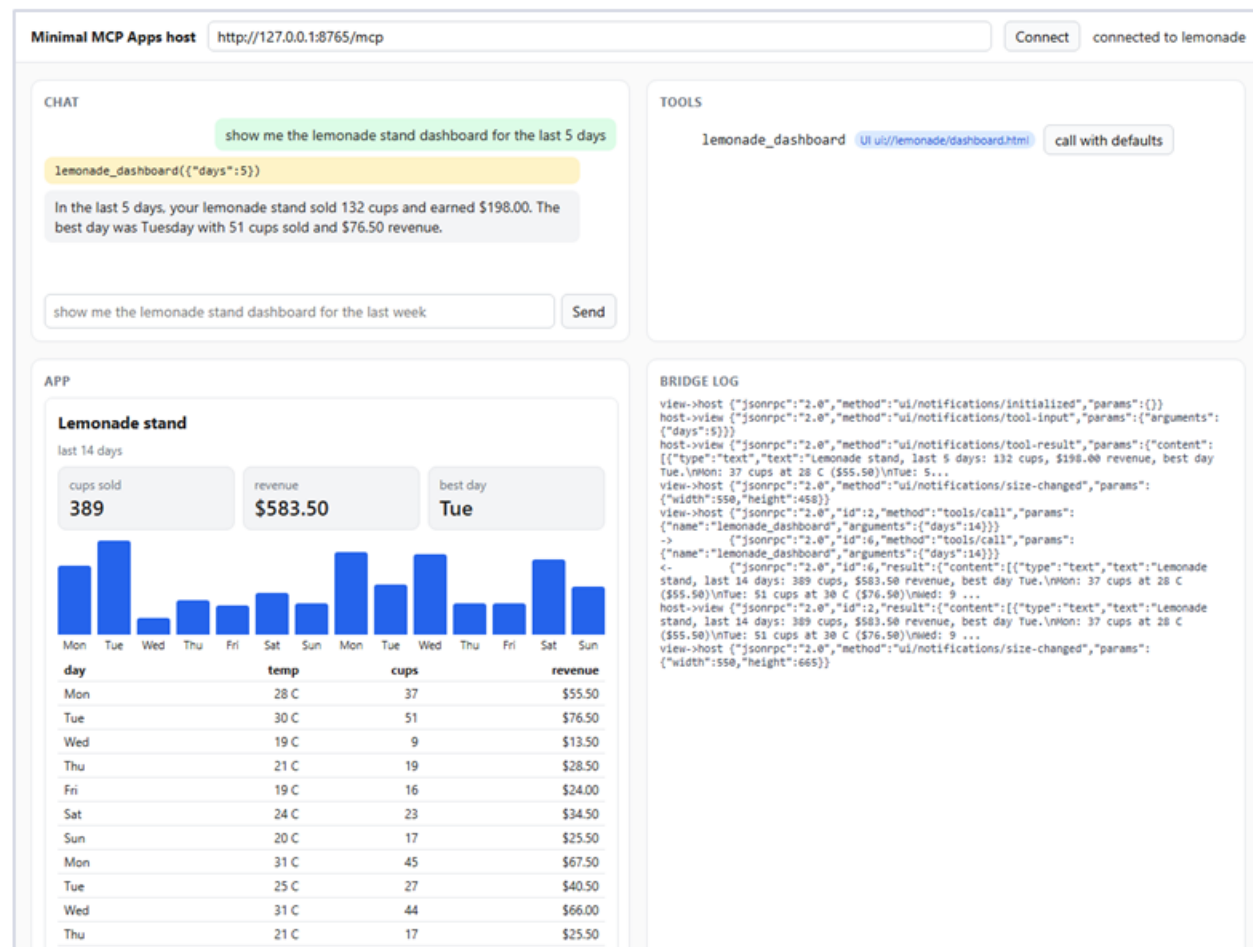
```

06_mcp_apps/step_01_mcp_app_resource/view.html

- 1 `ui/initialize` is a request with an id; the host answers with `hostContext` (theme, CSS variables, container size) and `hostCapabilities`.
- 2 `ui/notifications/initialized` has no id. Only after it does the host send `tool-input` and `tool-result`; the bridge holds them until then.
- 3 A `ResizeObserver` sends `size-changed` after every render. In the demo: 550x458 first, 550x665 after the view called the tool for 14 days.

`request()` and `notify()` are two `postMessage` wrappers; the spec needs no SDK, and this is the proof.

One sentence for the model, a dashboard for the iframe, and a frame that grew



132 cups · \$198.00

the model's answer for `lemonade_dashboard({'days':5})`; best day Tuesday

550x458 → 550x665

size-changed reports: after mount, then after the view called `tools/call` for 14 days

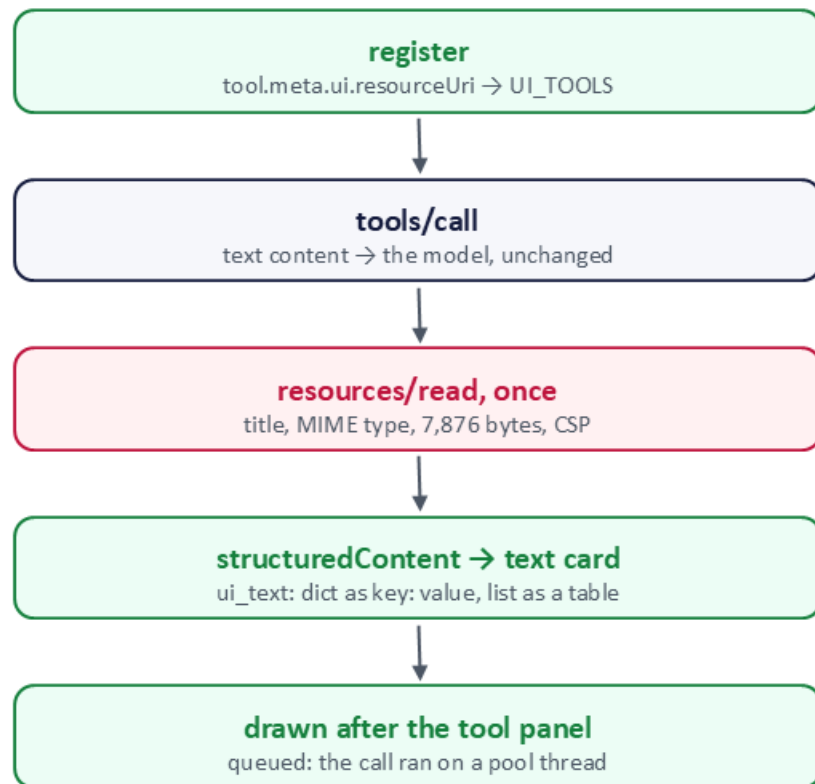
- Top left, the chat: the tool call and a one-sentence answer; the model read the text content only.
- Bottom left, the app: the view in a sandboxed iframe, mounted from `ui://lemonade/dashboard.html`, shown after the 14-day click.
- Right, the bridge log: `ui/initialize`, `initialized`, `tool-input`, `tool-result`, `size-changed`, in the fixed order.
- The demo clicked "14 days" inside the view; the host proxied `tools/call` to the server and the frame grew.

Recorded by demo.py: both processes, a headless browser, the bridge log and this screenshot.

Ship both: text that stands alone, and a `ui://` view for hosts that can mount it

Real hosts are built by other people. The report's rule for this side of the map is to **ship both**: text for any host, a view for hosts that can mount it.

- The server and the browser host are unchanged from step 06.1; only the host changes.
- `harness/` is the harness codelab's step 26 MCP client, extended into a text host over stdio.
- Registration remembers which tools carry a `ui://` resource, from `_meta.ui.resourceUri`.
- The resource is read once per session: title, MIME type, size, CSP; the template is static.
- The result's `structuredContent` is queued as a card and drawn after the tool panel, not inside the call.
- Claude Desktop and Goose mount the same resource in a real `iframe`; the configuration is one `stdio` command.



The harness as an MCP Apps host in text. The model's string is the same as in step 26.

Takeaway The server did not change between three hosts; the host decides what it can show.

The call queues a card; the string the model reads does not change

```
[1] uri = UI_TOOLS.get(tool_name(server, tool))
2   if uri and not getattr(result, "isError", False):
3       PENDING_APPS.append((tool_name(server, tool), server, uri, result))
[2] return history.cap(text)
5
6
[3] def show_pending(name):
8     """Draw the card queued for one tool call, if any. The loop calls this afte...
9     for entry in PENDING_APPS:
10         if entry[0] == name:
11             PENDING_APPS.remove(entry)
12             show_app(*entry[1:])
13             return True
14     return False
```

06_mcp_apps/step_02_mcp_app_in_a_real_host/harness/mcp_client.py

- 1 `UI_TOOLS` was filled at registration from `_meta.ui.resourceUri`. A tool with a view and a non-error result gets its tuple queued in `PENDING_APPS`.
- 2 The return value is the same as in step 26: capped text. The model never learns that a card was drawn, and the prompt grows by the text lines only.
- 3 `show_pending` runs from the loop, right after `ui.tool()` drew the panel. The call itself ran on a pool thread and must not write the terminal.

One added line in `harness/agent.py` calls `show_pending(name)` after the tool panel.

What a host learns from resources/read, kept for the session

```

1 def read_ui_resource(server, uri):
2     """What a host learns from resources/read, kept per session: title, MIME ty...
3     from . import ui_text
4
5     if (server, uri) not in UI_RESOURCES:
6         [1] contents = client().read(server, uri).contents[0]
7         html = getattr(contents, "text", None) or ""
8         meta = getattr(contents, "meta", None) or {}
9         [2] UI_RESOURCES[(server, uri)] = {
10             "title": ui_text.title_of(html) or uri,
11             [3] "mimeType": contents.mimeType,
12             "size": len(html.encode("utf-8")),
13             "csp": (meta.get("ui") or {}).get("csp") or {},
14         }
15     return UI_RESOURCES[(server, uri)]

```

06_mcp_apps/step_02_mcp_app_in_a_real_host/harness/mcp_client.py

- 1 **resources/read**, once per (server, uri). The resource is a static template; a host may prefetch and cache it at connection time. This one reads it lazily.
- 2 Four facts go into the card's subtitle: the title from the HTML, the MIME type, the size in bytes and the declared CSP. In the demo: 7,876 bytes, restrictive default.
- 3 **show_app** checks the MIME type against **text/html;profile=mcp-app** and refuses anything else, which is what the spec asks of every host.

harness/ui_text.py then turns any structuredContent into lines: a dict as key: value, a list of flat dicts as a table.

Two panels: the text the model saw, then the app as this host can show it

```
$ MCP_ALLOW=mcp_lemonade_* python -m harness.agent -p "Show me the
lemonade stand dashboard for the last 5 days. Answer in one sentence."
2,093 prompt · 23 completion · 1,792 cached
┌ mcp_lemonade_lemonade_dashboard 5 ────────────────────
│ Lemonade stand, last 5 days: 132 cups, $198.00 revenue, best day Tue.
│ Mon: 37 cups at 28 C ($55.50)
│ Tue: 51 cups at 30 C ($76.50)
│ ...
└ app · Lemonade dashboard ────────────────────
  ui://lemonade/dashboard.html | text/html;profile=mcp-app
  | 7,876 bytes | csp: restrictive default | rendered as text
  days: 5
  rows:
    day  temperature  cups  revenue
    Tue           30    51    76.5
  summary:
    total_cups: 132
    total_revenue: 198
    best_day: Tue
2,227 prompt · 46 completion · 2,048 cached
```

2,093 → 2,227

prompt tokens across the tool call: the text lines only; the table never reached the model

7,876 bytes

the ui:// resource, read once over stdio, restrictive default CSP

- The first panel is the tool result as the model saw it: text.
- The second is the MCP App as a terminal can show it: the resource's identity, then the data as a table.
- Claude Desktop and Goose mount the same resource in a real iframe with one stdio command; that run was not recorded.

Recorded by demo.py: the harness in print mode with server.py --stdio as a child process (condensed).

One interface, two transports: same numbers, same picture, different contract

The lemonade dashboard could also be an AG-UI state object rendered by a page you own. The report's rule is **not to choose**: keep the interface as data and ship it on both roads.

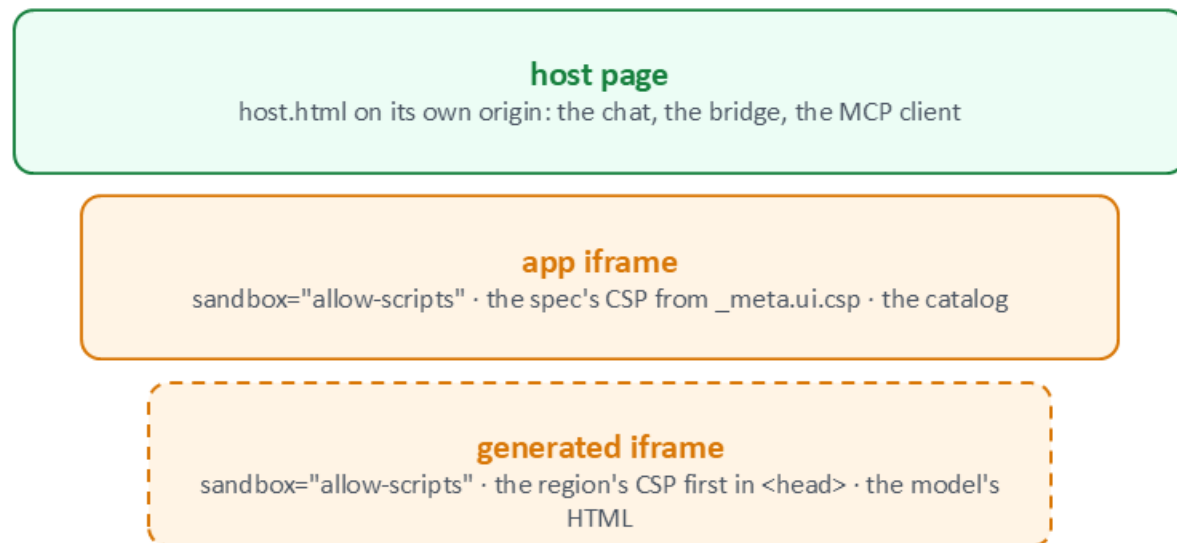
	AG-UI (sub-theme 02)	MCP Apps (sub-theme 06)
who owns the page	you your page, your event stream	the host's vendor Claude, ChatGPT, VS Code, Goose
the wire	SSE events to your page text, tool calls, state deltas	MCP to your server a tool result: text plus structured content; the view speaks JSON-RPC to the host
who renders	your renderer, from state a catalog you drew	your HTML, in the host's sandbox an iframe with a CSP the host built
who runs the loop	you your server drives the model	the host the model picks the tool; the view calls through the host
where it appears	your product one surface you control	any compliant host the same server, unchanged, in three hosts

Takeaway Sub-theme 07 takes the first road inside the harness; this sub-theme took the second. The server is the same either way.

The hybrid ships as a tool result, and its open-ended part runs one frame deeper

One tool result carries a **catalog spec** the view draws itself and one **generated document** the view mounts in a second sandbox.

- Step 01.4 built the report's hybrid on a page you own. This step puts it inside a host you do not control.
- `lemonade_report(days, focus)` returns `components`, six items in the shape of step 01.1, and `generated`, one HTML document.
- The MCP server makes the model call for the region. MCP Apps keeps credentials on the server side; the host never sees the key.
- The view renders the spec with four escaped renderers and mounts the document in a nested `sandbox="allow-scripts"` iframe.
- The app's own iframe and policy from step 06.1 are unchanged. A `srcdoc` frame has no URL for `frame-src` to match, so it inherits the app's policy and adds its own.
- The region can say one thing to the app: `{type: "event", name, payload}`. The app, not the region, speaks to the host.

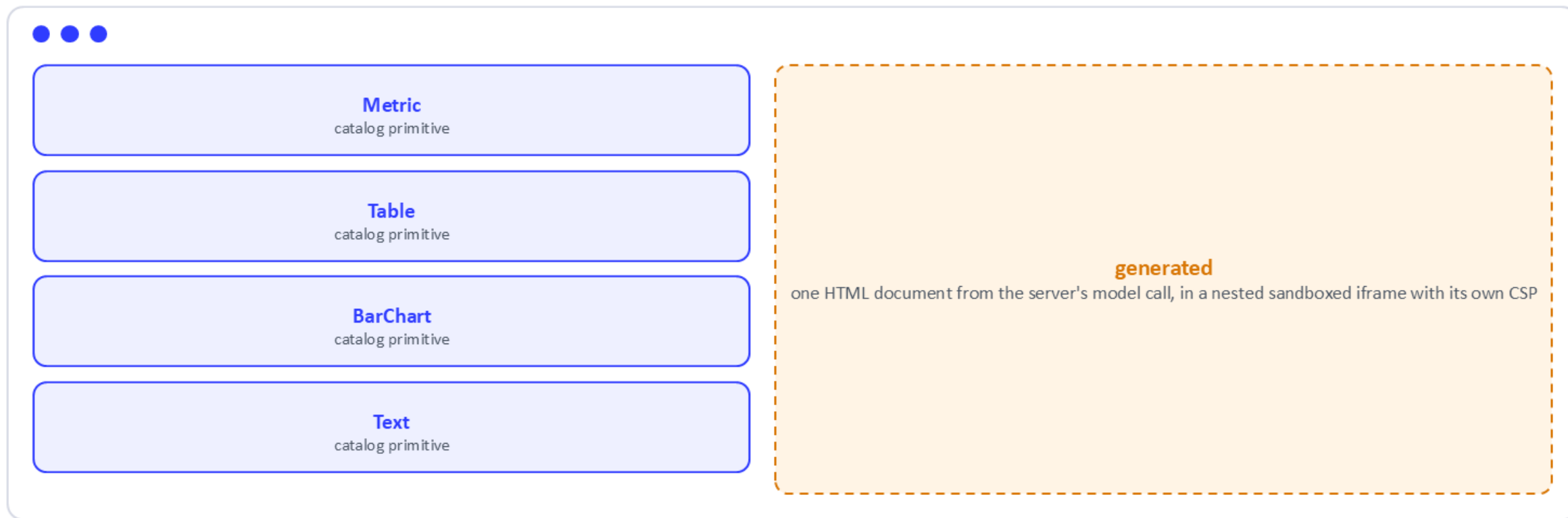


Two boundaries, two policies: the host trusts the app as far as the resource declared; the app trusts the region not at all.

Takeaway Catalog components stay cheap and trusted; the open-ended part pays its way inside a boundary the app owns.

Four catalog components the view owns, one generated region the server's model wrote

The same picture as step 01.4, now inside a **tool result**: *components* on the left, *generated* on the right.



components: 829 bytes of JSON, no model call • generated: 1,565 bytes of HTML, 370 + 473 tokens

Takeaway A `GeneratedView` in the components list renders as a stub: markup runs only through `generated`, which has its own frame.

One call, two halves: the server's own model call sits inside the tool

```

1 @server.tool(meta={"ui": {"resourceUri": VIEW_URI}})
2 def lemonade_report(days: int = 7, focus: str = report.DEFAULT_FOCUS) ->
types.CallToolResult:
3     """Sales report for the lemonade stand over the last `days` days (1 to 28), with
one interactiv...
4     days = max(1, min(int(days), 28))
5     rows = data.sales(days)
[1]     generated = report.generate_region(days, rows, focus) # the server's own model
call
[2]     text = data.as_text(days, rows) + f"\nThe interface also shows {focus}
({generated['source']})."
8     return types.CallToolResult(
9         content=[types.TextContent(type="text", text=text)],
[3]         structuredContent={"days": days, "focus": focus, "components":
report.components_for(days, ...

```

06_mcp_apps/step_03_mcp_app_hybrid/server.py

- 1 `generate_region` calls the model from the server. The host and the view never see this key.
- 2 The text for the model says a region exists and where it came from, and nothing of its HTML.
- 3 `components` is the catalog spec; `generated` is the document plus its source, usage and size.

Step 06.1's resource, link and policy are unchanged; only what the tool returns is new.

Two sources, one listener: the host above, the region below, one accepted shape

```

1 window.addEventListener("message", (event) => {
[1]   if (event.source === window.parent) return hostMessage(event.data);
[2]   if (event.source === generated.contentWindow && isEvent(event.data)) return
regionEvent(event.data);
4   // anything else, or any other shape, is dropped
5 });
6
7 function regionEvent({ name, payload }) {
8   app.events.push({ name, payload });
9   document.getElementById("events").textContent = `event from the generated region:
${name} ${JSON.stringify(payload ...
10  // The region cannot reach the host. The app can: it hands the event on as model
context, coalesced.
11   clearTimeout(contextTimer);
12   contextTimer = setTimeout(() => {
[3]     const text = `In the report's interactive region the user set ${name}:
${JSON.stringify(payload ?? {})}.`;
14     request("ui/update-model-context", { content: [{ type: "text", text }] });
15   }, 250);

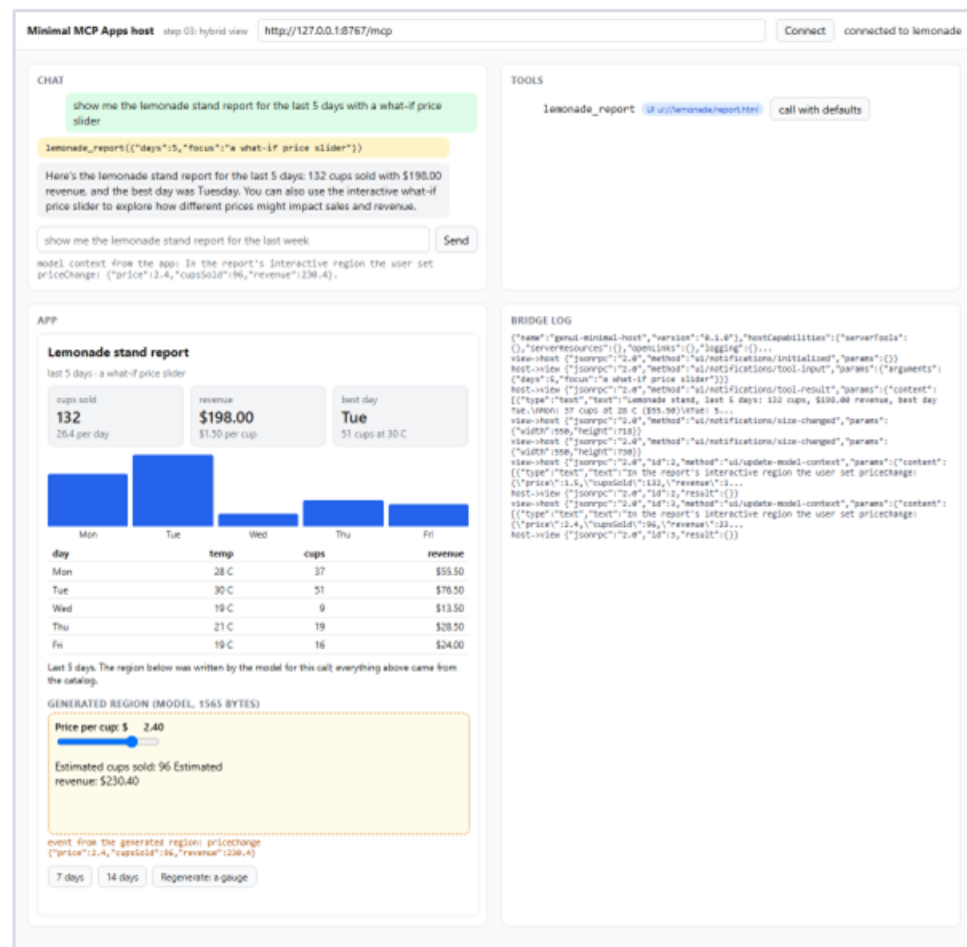
```

06_mcp_apps/step_03_mcp_app_hybrid/view.html

- 1 Messages from the parent are the host's JSON-RPC, as in step 06.1.
- 2 Messages from the inner frame must pass `isEvent: {type: "event", name, payload}`. A `tools/call` from the region is dropped.
- 3 The app forwards one sentence to the host as `ui/update-model-context`, debounced by 250 ms; the region never reaches the host.

Two source checks, one accepted shape: the region speaks to the app, and the app speaks to the host.

Catalog above, generated region below, and the region's event reached the host



370 + 473

tokens for the server's model call

1,565

bytes of generated HTML

- The model called `lemonade_report({"days":5,"focus":"a what-if price slider"})` and answered in two sentences.
- Three metrics, a bar chart, a table and a line of text came from the catalog: 829 bytes of JSON, no model call.
- The amber dashed frame is the generated region, one sandbox inside the other.
- Under the chat: the model context the app forwarded after the slider moved, `priceChange` at price 2.4.

The inner region on its own: the model's script ran two frames down

Price per cup: \$ 2.40



Estimated cups sold: 96 Estimated revenue: \$230.40

2.4

price the demo set on the slider

230.4

revenue in the event payload

- The document is the model's: a range input, a recompute on every input event (96 cups at 2.4), a `parent.postMessage` with the new values.
- Its CSP is the region's own, injected first in the head: no network, inline style and script only.
- The app accepted the event, drew it, and forwarded a sentence as `ui/update-model-context`; the host put it in the next turn.

Generative UI in the harness

The two codelabs meet here. The harness codelab ends with a coding agent that talks in prose and tool panels, then moves the loop onto TrueForge. This sub-theme gives both harnesses a rendering surface. Both steps are declarative generation in the report's terms; the transports are a terminal, a browser page fed over SSE, and the TrueForge SDK. Sub-theme 06 put an interface into a host the reader does not control; this one lets the reader's own agent answer with one.

07.1 A render_ui tool for the harness

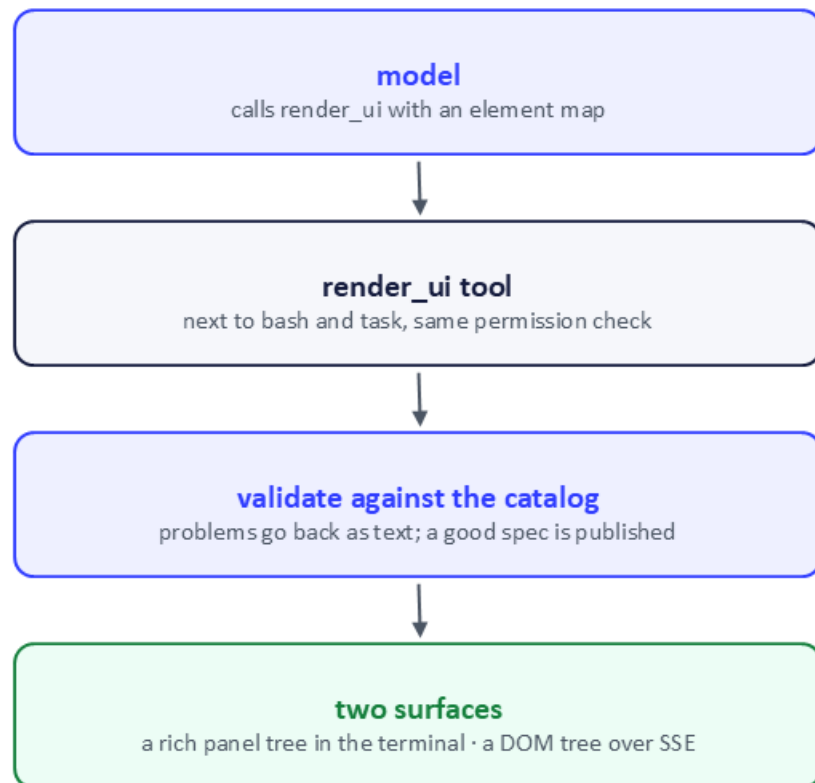
07.2 TrueForge's generative UI, rendered in a page of ours

07.3 Hybrid on TrueForge

The spec is the interface: validate once, draw on any surface

Generative UI as **one tool**: the model emits a json-render element map, the harness validates it against a catalog, and every surface draws from that data.

- `harness/` is the stage 15 loop copied whole; nothing in the loop changes.
- `genui.py` holds a six-component catalog with typed props, a required list and a children flag.
- `validate()` returns every problem as a plain sentence; the model reads the list as the tool result.
- `llm.py` writes the catalog into the system prompt from the same dictionary the validator reads.
- `ui.py` draws one `rich` renderable per element; `web.py` pushes the spec to every open tab over SSE.
- The model sees one sentence back. The picture is drawn by the surface, from data.



Add a surface and nothing in the agent changes.

Takeaway One agent, one spec, two surfaces: the model composes from a catalog it saw once; a new surface changes nothing.

Validate, publish, report: a bad spec comes back as corrections, not a crash

```

1 def render_ui(spec: dict) -> str:
2     """Validate a json-render element map and hand it to every surface.
3
4     The terminal draws it in ui.tool(); the web page gets it over SSE. The
5     model only sees this short string, never the picture: the spec is data
6     that the surfaces interpret, not something the model has to describe.
7     """
8
9     [1] problems = validate(spec)
10    if problems:
11        [2] return "Invalid spec, nothing rendered:\n- " + "\n- ".join(problems)
12    web.publish(spec)
13    [3] return f"Rendered {len(spec['elements'])} elements from root '{spec['root']}'."

```

07_harness_genui/step_01_render_ui_tool/harness/tools.py

- 1 `validate()` returns a list of sentences. An empty list means the spec is valid.
- 2 The problems go back to the model as the tool result. The loop continues; the model corrects and calls again.
- 3 The model only ever sees this line. The picture is drawn by the surfaces, not described to the model.

The tool sits behind the same `execute()` permission check as `bash`; the subagent does not get it.

The prompt is generated from the catalog, so prompt and validator cannot drift

```
1 def catalog_prompt():
2     """The catalog as text for the system prompt: one line per component."""
3     lines = []
4     [1] for name, entry in CATALOG.items():
5         props = ", ".join(f"{p}: {t}" for p, t in entry["props"].items())
6         kids = " Takes children." if entry["children"] else ""
7         [2] lines.append(f"- {name} ({props}): {entry['description']}{kids}")
8     return "\n".join(lines)
```

07_harness_genui/step_01_render_ui_tool/harness/genui.py

1

The same **CATALOG** dictionary that **validate()** checks against. One definition, three uses.

2

One line per component: name, typed props, description, and whether it takes children.

llm.py places this text in the system prompt under the instruction to call **render_ui** for dashboards, tables and charts.

One prompt, a ten-element map, drawn by the terminal and the browser alike



1,365 · 308

prompt · completion tokens, first call

10

elements in the map

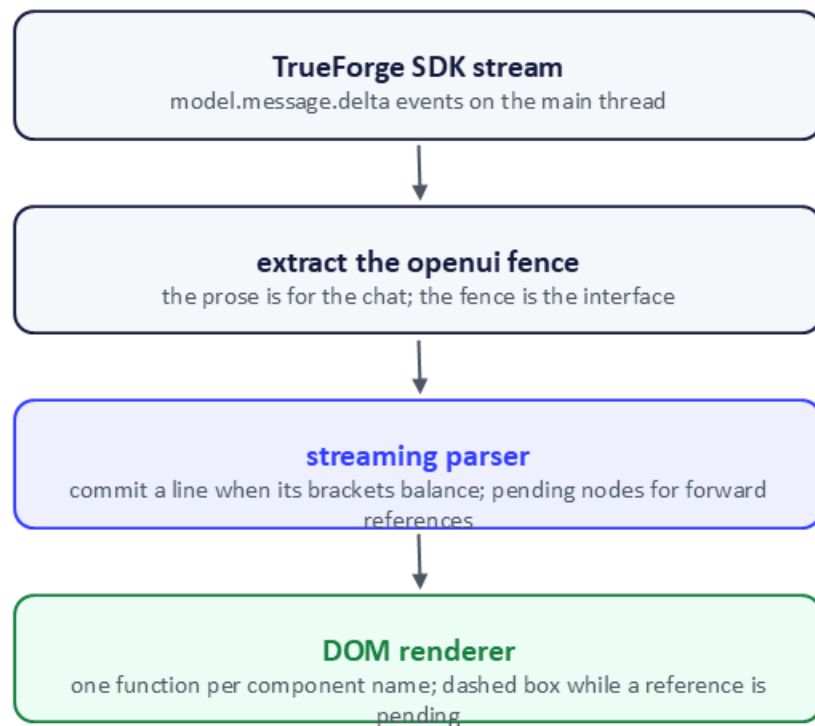
- Metrics, a seven-bar chart, an inventory Table inside a nested Card, and a weather card, from one call.
- The terminal drew the same spec inside the `render_ui · 10 elements` tool panel.
- The demo script captured this picture headlessly from the web surface.

The model saw the catalog once, in the system prompt, and produced nested Cards, a Chart and a Table on the first call.

A hosted harness already speaks a UI language; any parser that knows the grammar can draw it

With `generative_ui` enabled, TrueForge's agent answers with an **OpenUI Lang program inside a fence**. This step renders that program in a page of its own.

- `client/genui.py` opens a session with generative UI on and streams one turn over the SDK.
- `extract_program()` pulls the text out of the first ``openui fence, even one that never closed.
- `openui_parse.py` is streaming-first: statements are `name = expr`; references may point forward.
- A statement commits at the first newline where its brackets balance; a multi-line `Card([` waits.
- `server.py` sends one line per SSE event; `app.js` feeds each line to the parser and redraws from `root`.
- Unknown components become a labelled box, pending references a dashed placeholder.



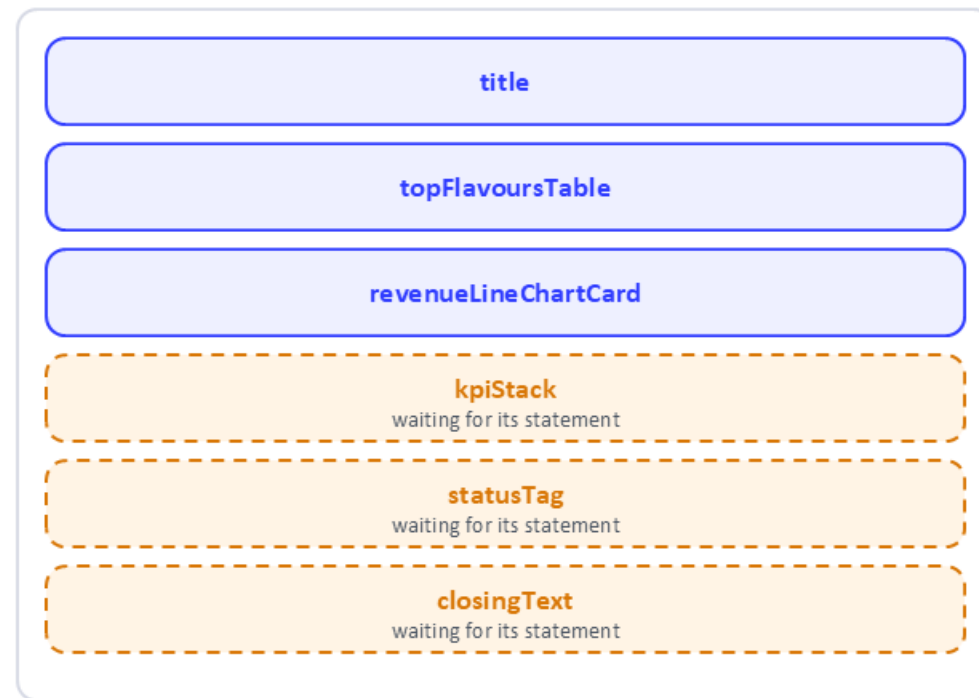
The same parser exists in Python and JavaScript, tested against the same program.

Takeaway A UI language is a string with a grammar: any parser that knows the grammar can draw it, and streaming follows from statements that commit line by line.

The root line names six children on line 1; the page draws the skeleton before their lines arrive

The recorded program, 25 lines in: four statements have landed, `kpiStack` is still open, and three children are dashed placeholders.

```
root = Stack([title, topFlavoursTable, revenueLineChartCard, kpiStack,
statusTag, closingText], "column", "1", "center")
title = TextContent("Lemonade Stand Report", "large-heavy")
topFlavoursTable = Card([
    CardHeader("Top 3 Flavours - Cups Sold & Price"),
])
revenueLineChartCard = Card([
    CardHeader("Weekly Revenue Over Four Weeks"),
])
kpiStack = Stack([
statusTag = Tag("Performance", null, "md", "success")
closingText = TextContent("Sales are strong and trending upward, keep
focusing on the top flavours to maximize revenue.", "default")
```



Bright lines have committed; dim lines are still to come. `kpiStack = Stack()` is held back until its closing bracket balances.

Takeaway Structure before content: the layout is known from the first line, and each box fills in when its statement lands.

A statement commits at the first newline where its brackets balance

```

1 def feed(self, text):
2     """Add a chunk. Every complete statement it finishes is parsed now."""
3     self.buffer += text
4     while True:
5         # the first newline at which the text before it is balanced ends a statement
6         start, cut = 0, None
7         while cut is None:
8             nl = self.buffer.find("\n", start)
9             if nl == -1:
10                 return # the rest is an unfinished line: hold it back
11             if complete(self.buffer[:nl]):
12                 cut = nl
13                 start = nl + 1
14             line, self.buffer = self.buffer[:cut], self.buffer[cut + 1:]
15         self.add_line(line.replace("\n", " "))

```

07_harness_genui/step_02_trueforge_generative_ui/openui_parse.py

1

No newline yet: the rest of the buffer is an unfinished line and is held back for the next chunk.

2

`complete()` is a bracket-balance check. The loop walks newline by newline until the text before one balances.

3

A multi-line `Card([` becomes one statement; its inner newlines are folded to spaces before parsing.

`feed()` takes any chunk; complete statements are parsed at once, an open list waits for its closing bracket.

Enabling generative_ui is the whole configuration; the agent was told nothing about OpenUI Lang

```

1 def open_session():
2     """A session on an inline agent with generative UI switched on."""
3     spec = AgentSpec(
4         model=Model(name=MODEL),
5         instructions=INSTRUCTIONS,
6         [1] config=RuntimeConfig(generative_ui=GenerativeUiConfig(enabled=True)),
7     )
8     [2] session = client().sessions.create(agent=SessionAgentSpecBody(spec=spec))
9     return session.data.id

```

07_harness_genui/step_02_trueforge_generative_ui/client/genui.py

1 The only difference from Part 7's client. The catalog, the grammar and the fence come from TrueForge's own harness prompt.

2 An inline agent spec on a session. `ask()` then streams one turn and collects `model.message.delta` events on the main thread.

`extract_program()` then returns what is inside the first ``openui fence, and copes with a fence a cut stream never closed.

Twenty-five lines in: the root named six children, three are still dashed placeholders

TrueForge generative UI, rendered here 25 lines, waiting for kpiStack, statusTag, closingText

Lemonade Stand Report

Top 3 Flavours - Cups Sold & Price

Flavour	Cups Sold	Price per Cup (\$)
Classic Lemon	150	1.5
Strawberry	120	1.75
Mint	100	1.6

Weekly Revenue Over Four Weeks

Week 1 Week 2 Week 3 Week 4

Revenue (\$) by Week - Revenue (\$)

waiting for kpiStack

waiting for statusTag

```

root = Stack([title, topFlavoursTable, revenueLineChartCard, kpiStack,
statusTag, closingText], "column", "1", "center")

title = TextContent("Lemonade Stand Report", "large-heavy")

topFlavoursTable = Card([
  CardHeader("Top 3 Flavours - Cups Sold & Price"),
  Table([
    Col("Flavour", ["Classic Lemon", "Strawberry", "Mint"]),
    Col("Cups Sold", [150, 120, 100], "number"),
    Col("Price per Cup ($)", [1.50, 1.75, 1.60], "number")
  ])
])

revenueLineChartCard = Card([
  CardHeader("Weekly Revenue Over Four Weeks"),
  LineChart(
    ["Week 1", "Week 2", "Week 3", "Week 4"],
    [Series("Revenue ($)", [225, 260, 240, 280])],
    "linear",
    "Week",
    "Revenue ($)"
  )
])

kpiStack = Stack([

```

25

lines received of 32

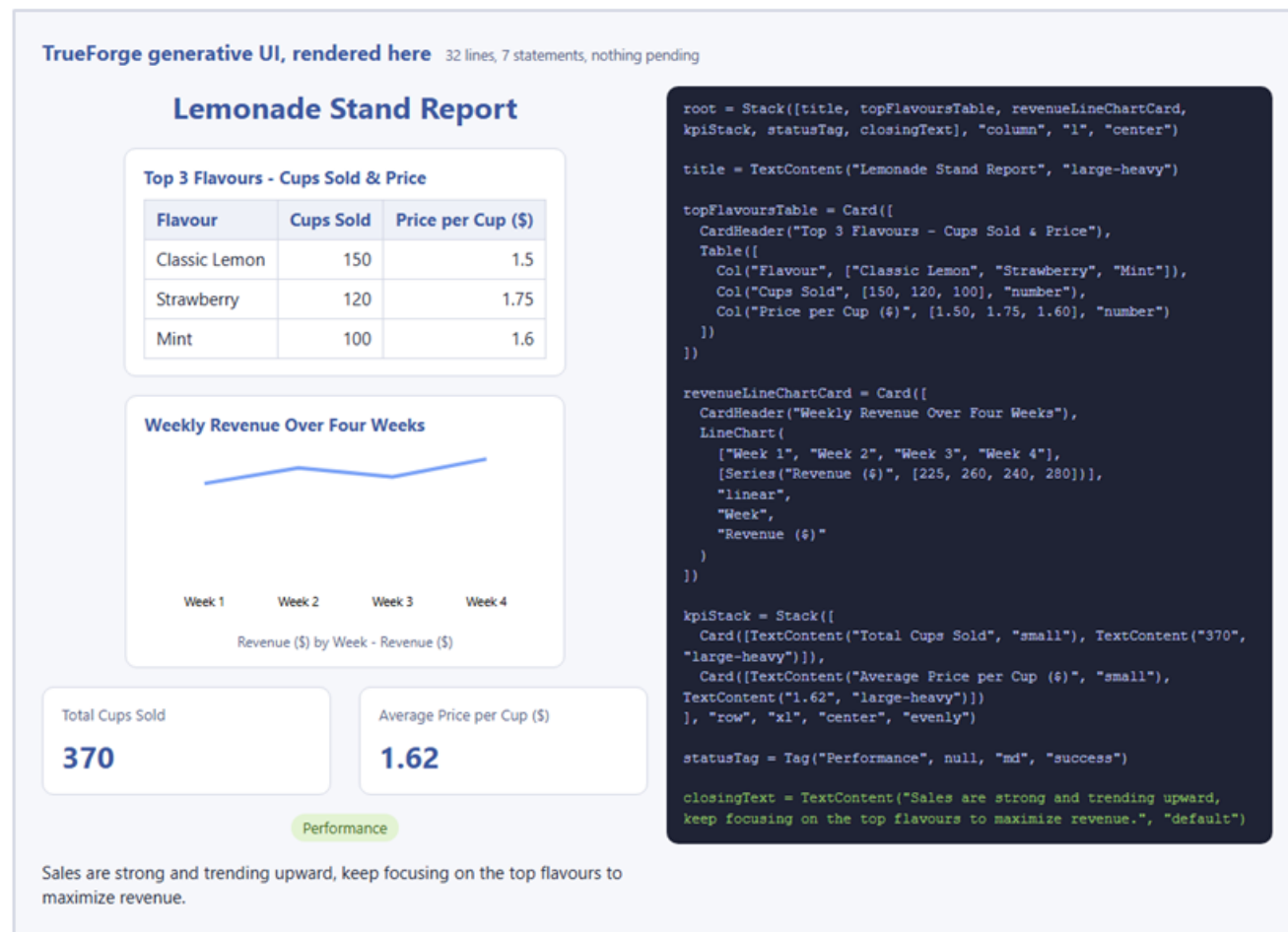
3 of 6

children still pending

- The title, the flavours table and the revenue chart card have their statements and are drawn.
- `kpiStack`, `statusTag` and `closingText` are named on line 1 and wait as dashed boxes.
- The program text beside the tree highlights the line that just arrived.

One line per SSE event; the tree is redrawn from `root` after every line.

A 32-line program of 7 statements, drawn by a page that never saw TrueForge's UI



6,954 · 352

input · output tokens for the turn

1,288

input tokens labelled harness: the catalog and grammar

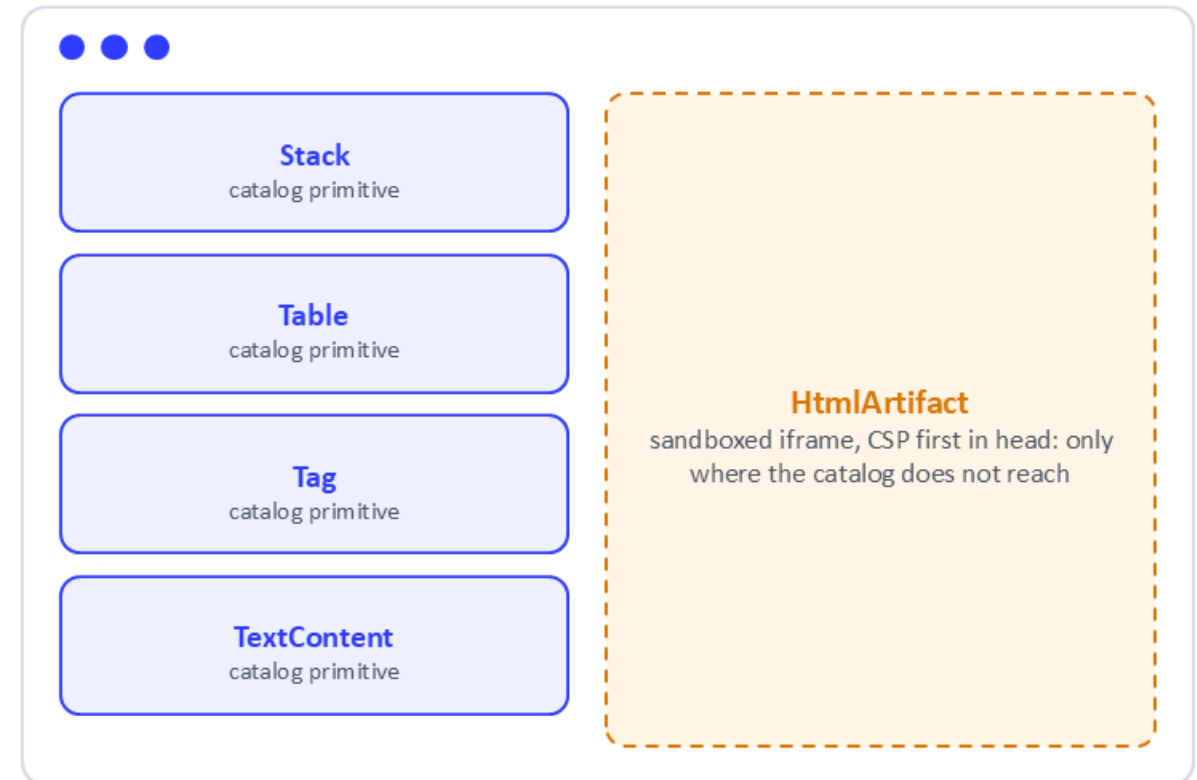
- A title, a table, a line chart card, two KPI cards, a tag and a sentence: every placeholder filled.
- The renderer's positional args are a reading of what the agent emitted, not a copy of TrueForge's library.
- Anything outside that reading lands in the **unknown** box with its args visible.

The step 01 model wrote 308 tokens of JSON for a comparable dashboard; the argument for OpenUI Lang is streaming, and larger UIs.

The hosted harness sets the catalog; the surface adds the one component it lacks

TrueForge's generative UI is **catalog-only**. The report's hybrid needs one **open-ended** component, and it can live on the client side of the SDK.

- The agent's instructions allow one `HtmlArtifact(title, document)` statement inside the `openui` program.
- Only when the user asks for something interactive; dashboards and reports stay in the catalog.
- TrueForge hands the agent its catalog through a system tool, `get_openui_instructions`, before every reply.
- The page renders the artifact in a sandboxed iframe with a Content Security Policy injected first in head.
- Every other component goes through step 07.2's catalog renderer, unchanged.
- The TrueForge chat UI does not know the component; this page is where the hybrid exists.



Catalog primitives from TrueForge's own guide by default; one boxed open-ended component, added by the surface.

Takeaway A harness you do not own can still get an escape hatch, as long as the surface that renders its replies owns the box.

One renderer, two states: raw source while the line is open, an iframe once it closes

```
[1] HtmlArtifact([title = "Artifact", document = ""], partial) {
  2   const box = el("section", "artifact");
  3   box.dataset.state = partial ? "streaming" : "ready";
  4   const head = el("div", "artifact-head");
  5   head.append(el("strong", "", String(title)));
[2]   if (partial) {
  7     head.append(el("span", "artifact-status", `Generating artifact ...
    ${document.length} characters so far`)...
  8     box.append(head, el("pre", "artifact-raw", document));
  9     return box;
 10   }
 11   head.append(el("span", "artifact-status", `${document.length} characters,
    sandboxed iframe`));
 12   const frame = el("iframe", "artifact-frame");
 13   frame.title = String(title);
 14   frame.setAttribute("sandbox", "allow-scripts");
 15   frame.setAttribute("referrerpolicy", "no-referrer");
[3]   frame.srcdoc = sandboxed(String(document));
```

07_harness_genui/step_03_trueforge_hybrid/web/render.mjs

- 1 The parser marks a statement whose line is still in the buffer as **partial**; the renderer gets that flag.
- 2 Streaming: a status line and the raw document so far, the way OpenUI's example shows an artifact generating.
- 3 Complete: srcdoc is the document with the **CSP meta first in head**; sandbox=allow-scripts, no referrer.

sandboxed() in web/sandbox.mjs removes any CSP the model wrote and injects default-src 'none' with inline style and script allowed.

The instructions frame the surface's catalog as the guide's, minus the interactive parts, plus one

```
1 The surface that renders your reply is not the standard one. Its catalog is the one
  get_openui_instructions describes, with two changes:
```

```
[1] - REMOVED: Form, FormControl, Input, Buttons, Button, Action, $ state variables, @
functions and expressions. This surface does not implement them; a program that uses
them renders as nothing. Never use them, and never invent components th...
```

```
[2] - ADDED: HtmlArtifact(title: string, document: string). The document is a self-
contained HTML/CSS/JavaScript page, run in a sandboxed iframe. This is the only way to
give the user something interactive on this surface.
```

```
4
```

```
5 Rules for HtmlArtifact:
```

```
[3] - Use it only when the user explicitly asks you to build something interactive: an
app, game, simulation, calculator, visualization, or similar experience. Then the whole
experience (every input, button and result) lives inside one documen..
```

07_harness_genui/step_03_trueforge_hybrid/client/genui.py

- 1 REMOVED: the guide teaches Form, Input and \$state; the page does not run them, so the model must not use them.
- 2 ADDED: HtmlArtifact, the only way to something interactive on this surface.
- 3 The exact shape of an interactive reply, one intro and one artifact; without it the model split the calculator into four.

Three additions after live runs: an example, the REMOVED / ADDED framing, and the exact reply shape. Six artifacts in six runs after the third.

The report prompt: six catalog statements, no artifact

TrueForge generative UI, catalog plus one artifact 9 chunks, 6 statements, nothing pending

Lemonade Stand Report: Top 3 Flavours

Flavour	Sales
Classic Lemon	150
Strawberry	120
Mint Lime	90

Operating Smoothly

```
root = Stack([title, table, statusTag], "column", "m")
title = TextContent("Lemonade Stand Report: Top 3 Flavours", "large-heavy")
table = Table([Col("Flavour", flavours), Col("Sales", sales, "number")])
flavours = ["Classic Lemon", "Strawberry", "Mint Lime"]
sales = [150, 120, 90]
statusTag = Tag("Operating Smoothly", null, "md", "success")
```

7,972

input tokens, two model calls

124

output tokens

- The first model call asks for the catalog through `get_openui_instructions`; the second carries it and writes the reply.
- Title, table with two columns, status tag: step 07.2's renderers, untouched.
- The instructions describe `HtmlArtifact`, and the agent did not use it here: the catalog is the default.

Prompt: Show a lemonade stand report with a table of the top three flavours and a status tag.

The calculator prompt mid-stream: the intro is drawn, the artifact shows its source growing

TrueForge generative UI, catalog plus one artifact 16 chunks, waiting for artifact, 1 artifact

Welcome to the interactive Lemonade Price Calculator! Adjust the price per cup and the number of cups sold to see your total revenue in real time.

Lemonade Price Calculator

Generating artifact ... 358 characters so far

```
<!DOCTYPE html><html><head><meta charset='UTF-8'><title>Lemonade Price Calculator
</title><style>body{font-family:system-ui;padding:1rem;max-width:320px;margin:auto}
label{display:block;margin-top:1rem;font-weight:600}input[type=range]{width:100%}
output{display:block;margin-top:.5rem;font-size:1.25rem;font-weight:bold;color:#2a9d8f}</style></head><body><label
```

```
root = Stack([intro, artifact], "column", "m")
intro = TextContent("Welcome to the interactive Lemonade Price Calculator!
Adjust the price per cup and the number of cups sold to see your total revenue
in real time.", "default")
artifact = HtmlArtifact("Lemonade Price Calculator", "<!DOCTYPE html><html>
<head><meta charset='UTF-8'><title>Lemonade Price Calculator</title><style>
body{font-family:system-ui;padding:1rem;max-width:320px;margin:auto}label{
display:block;margin-top:1rem;font-weight:600}input[type=range]{width:100%}
output{display:block;margin-top:.5rem;font-size:1.25rem;font-weight:bold;
color:#2a9d8f}</style></head><body><label
```

16

chunks in

358

characters of document so far

- The intro TextContent committed on its own line; the artifact line is still open and shows as raw source.
- The same characters are the tail of the program on the right: the server streams 40-character chunks, not lines.
- The status reads waiting for artifact: the statement is not committed until its closing quote arrives.

Prompt: Build me an interactive lemonade price calculator I can play with, with a short intro.

The artifact in its sandboxed iframe: two slider moves changed the total from \$50.00 to \$1000.00

TrueForge generative UI, catalog plus one artifact 44 chunks, 3 statements, nothing pending, 1 artifact

Welcome to the interactive Lemonade Price Calculator! Adjust the price per cup and the number of cups sold to see your total revenue in real time.

Lemonade Price Calculator 1441 characters, sandboxed iframe

Price per cup (\$):

5.00

Number of cups sold:

200

Total Revenue: \$1000.00

```
root = Stack([intro, artifact], "column", "m")
intro = TextContent("Welcome to the interactive Lemonade Price Calculator! Adjust the price per cup and the number of cups sold to see your total revenue in real time.", "default")
artifact = HtmlArtifact("Lemonade Price Calculator", "<!DOCTYPE html><html><head><meta charset='UTF-8'><title>Lemonade Price Calculator</title><style>body{font-family:system-ui;padding:1rem;max-width:320px;margin:auto}label{display:block;margin-top:1rem;font-weight:600}input[type=range]{width:100%}output{display:block;margin-top:.5rem;font-size:1.25rem;font-weight:bold;color:#2a9d8f}</style></head><body><label for='priceRange'>Price per cup ($): <output id='priceOutput'>1.00</output></label><input type='range' id='priceRange' min='0.25' max='5' value='1' step='0.05'><label for='quantityRange'>Number of cups sold: <output id='quantityOutput'>50</output></label><input type='range' id='quantityRange' min='1' max='200' value='50'><hr><div>Total Revenue: $<span id='totalRevenue'>50.00</span></div><script>const priceRange=document.getElementById('priceRange');const quantityRange=document.getElementById('quantityRange');const priceOutput=document.getElementById('priceOutput');const quantityOutput=document.getElementById('quantityOutput');const totalRevenue=document.getElementById('totalRevenue');function update(){const price=parseFloat(priceRange.value);const quantity=parseInt(quantityRange.value,10);priceOutput.textContent=price.toFixed(2);quantityOutput.textContent=quantity.toString();const total=price*quantity;totalRevenue.textContent=total.toFixed(2);}priceRange.addEventListener('input',update);quantityRange.addEventListener('input',update);update();</script></body></html>>")
```

85%

of the reply's tokens is the artifact document

1,441

characters of HTML, CSS and JS

- sandbox=allow-scripts, referrerpolicy=no-referrer, and the CSP meta first in head: the demo checks all three.
- The headless test set both sliders to their maximum inside the iframe; the document's own script updated the total.
- checkDocument() found nothing to report: no external resource, no network call, no meta refresh.

379 of 447 reply tokens are the document (463 output tokens from turn.done); the catalog reply for the report was 108 tokens.

Choosing for your own product

The report's decision framework, with the step that shows each option.
Transport first, because your product decides it. Generation mode second, because it is a cost and control trade you can revisit.

1 pick the transport

2 pick the generation mode

3 check the measurements

4 keep the three rules

If this is your situation, use this, and this step shows it

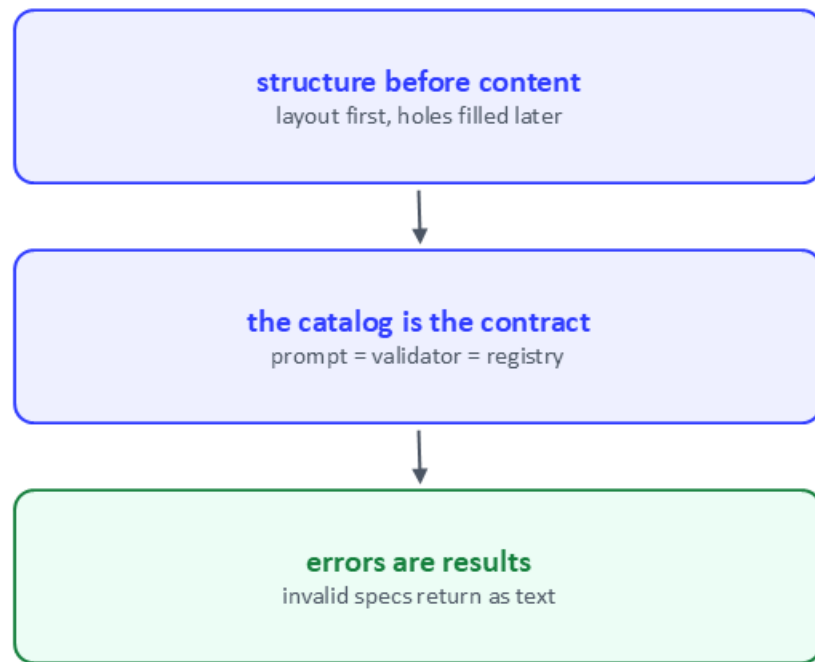
If	Then	Step
the agent runs inside a product you build	AG-UI	02.1 to 02.3
the agent must appear inside Claude, ChatGPT, VS Code or Goose	MCP Apps	06.1, 06.2
both	ship both: the same UI, two transports	06.2
the answer shapes are few and your design system is strict	static: prebuilt components, the agent fills props	01.1
layouts vary and must stay inside your design system	declarative, from a catalog	01.2 and one of 03, 04, 05
you need the lowest token cost and the earliest first paint	OpenUI Lang	04.1 to 04.3
you render on Flutter, Angular or native, or need catalog negotiation	A2UI	03.1 to 03.3
you render to PDF, email, a terminal or video as well as the browser	json-render	05.1 to 05.3
one-off or creative output is part of the product	the hybrid: a catalog with a sandboxed generated view	01.4, 04.4, 06.3, 07.3

Five questions the report raises, answered with recorded runs

Question	Measured in	Result
How much does open-ended HTML cost against a flat spec?	01.3	nine to ten times the tokens, eight times the wall time, over three runs
Does a flat element map paint earlier than a nested tree?	01.2	layout known at chunk 33 of 266, against chunk 408 of 408
Does the report's format table hold?	04.3	every cell reproduces: OpenUI Lang 4,800 tokens against 9,122 YAML, 9,948 C1 JSON, 10,180 patches
Does patch streaming help a real page?	05.2	first paint at 3.5 s against 10.1 s complete; the single-paint version took 5.8 s
How much of a hybrid reply is the open-ended part?	04.4	the artifact document was 292 of 341 completion tokens, 86%
What does a hosted harness's UI language look like on the wire?	07.2	an OpenUI Lang program of 32 lines and 7 statements inside a normal reply
Can the hybrid live inside a host you do not control, or on a hosted harness?	06.3, 07.3	yes: a generated region one sandbox deeper inside an MCP App; one artifact statement in a TrueForge reply, rendered by your own page

Three rules that held across all 23 steps

- **Structure before content.** Every format that streams well names the layout first and fills it later: flat maps, forward references, patches against ids. A nested document cannot paint until it closes.
- **The catalog is the contract.** Prompt, validation and rendering come from one definition. Change the definition and all three change. This is what keeps declarative generation safe.
- **Errors are results.** A spec that fails validation goes back to the model as text. A blocked script stays inside its iframe. The loop never ends because the agent drew something wrong.
- The harness codelab ended with the same three rules for tools. They hold for interfaces too.



Takeaway Models are commodities. The harness is the product. The interface is where the product meets the user.

Run it, then change it

- `git clone https://github.com/dlmastery/simple-coding-harness` and open `genui/README.md`: the written codelab with every snippet verified and every screenshot.
- Start with `genui/01_foundations/step_01_static_components:python demo.py` records a run and a screenshot in under a minute.
- Each README ends with a checkpoint: change the catalog, watch the prompt change; remove a prompt rule, watch the token count move.
- The harness codelab in the repository root is the other half: the loop, tools, permissions, sandbox, sessions, and the same harness on TrueForge.
- The State of Generative UI report is the map this series follows; the step READMEs cite the specification or library each one uses.

harness codelab · stages 1 to 51

the loop, tools, guardrails, context, durability, orchestration, TrueForge

generative UI series · 23 steps

static, declarative, open-ended; AG-UI, A2UI, OpenUI Lang, json-render, MCP Apps

your product

the transport your product needs, the generation mode your design system allows